

**ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ
ΠΛΗΡΟΦΟΡΙΚΗΣ**



Τομέας: Υπολογιστική Υψηλών Επιδόσεων
Εργαστήριο: Εργαστήριο Πληροφοριακών Συστημάτων
Υψηλών Επιδόσεων

**Συγκριτική Ανάλυση και Πειραματική
Αξιολόγηση Συστημάτων Παράλληλου
Προγραμματισμού Βασισμένου σε
Εργασίες**

Διπλωματική Εργασία

του φοιτητή

Γεωργίου Τσάβου του Κωνσταντίνου

Αριθμός Μητρώου: 1084606

Επιβλέπων: Παναγιώτης Χατζηδούκας, Αναπληρωτής Καθηγητής,
Πανεπιστήμιο Πατρών

Πάτρα, Ιούνιος 2026

ΠΙΣΤΟΠΟΙΗΣΗ

Πιστοποιείται ότι η διπλωματική εργασία με θέμα:

**Συγκριτική Ανάλυση και Πειραματική Αξιολόγηση
Συστημάτων Παράλληλου Προγραμματισμού Βασισμένου
σε Εργασίες**

του φοιτητή του Τμήματος Μηχανικών Ηλεκτρονικών Υπολογιστών και
Πληροφορικής

Γεωργίου Τσάβου του Κωνσταντίνου

(Α.Μ.: 1084606)

παρουσιάστηκε δημόσια και εξετάστηκε επιτυχώς από την
τριμελή εξεταστική επιτροπή στις __/__/____.

Ο Επιβλέπων

Μέλος της Επιτροπής

Μέλος της Επιτροπής

Παναγιώτης

Χατζηδούκας

Αναπληρωτής Καθηγητής

Σπυρίδων

Σιούτας

Καθηγητής

Γεώργιος Παπαδημητρίου

Επίκουρος Καθηγητής

Στοιχεία Διπλωματικής Εργασίας

Θέμα:

Συγκριτική Ανάλυση και Πειραματική Αξιολόγηση
Συστημάτων Παράλληλου Προγραμματισμού Βασισμένου
σε Εργασίες

Φοιτητής:

Γεώργιος Τσάβος του Κωνσταντίνου

Ομάδα Επίβλεψης:

Παναγιώτης Χατζηδούκας

Αναπληρωτής Καθηγητής, Πανεπιστήμιο Πατρών

Εργαστήριο:

Εργαστήριο Πληροφοριακών Συστημάτων Υψηλών Επιδόσεων

Περίληψη

Η παρούσα διπλωματική εργασία επικεντρώνεται στη συγκριτική ανάλυση και πειραματική αξιολόγηση δύο διαφορετικών συστημάτων παράλληλου προγραμματισμού βασισμένου σε εργασίες (task-based parallel programming). Καθώς η γλώσσα Python κυριαρχεί στους επιστημονικούς υπολογισμούς, η αποτελεσματική εκμετάλλευση του σύγχρονου υλικού, όπως οι πολύπλοκες αρχιτεκτονικές NUMA, παρεμποδίζεται από εγγενείς περιορισμούς του διερμηνέα, με κυριότερο το Global Interpreter Lock (GIL).

Για την αντιμετώπιση αυτών των προκλήσεων, μελετώνται και αντιπαραβάλλονται δύο πλατφόρμες με διαφορετική φιλοσοφία: η διεπαφή StarPUPy (η οποία βασίζεται στο σύστημα StarPU, σχεδιασμένο για περιβάλλοντα κοινής μνήμης και ετερογενή συστήματα) και το torcipy (ένα Python-native σύστημα που αξιοποιεί το κατανεμημένο πρότυπο MPI). Η αξιολόγηση πραγματοποιείται μέσω τριών αντιπροσωπευτικών επιστημονικών εφαρμογών: στοχαστική βελτιστοποίηση CMA-ES, ανάλυση υψής εικόνας με φίλτρα Gabor, και προσομοίωση Monte Carlo για αποτίμηση χρηματοοικονομικών παραγώγων.

Τα πειραματικά αποτελέσματα σε υπολογιστικό σύστημα 64 πυρήνων δείχνουν ότι το StarPUPy υπερέχει σημαντικά σε εφαρμογές που περιορίζονται από τον συγχρονισμό (sync-bound), χάρη στην αποδοτική διαχείριση ελαφριών νημάτων. Αντίθετα, το torcipy προσφέρει εξαιρετική κλιμάκωση και απόδοση σε εφαρμογές που περιορίζονται από τη μνήμη (memory-bound), καθώς η αρχιτεκτονική διεργασιών MPI που διαθέτει παρακάμπτει επιτυχώς το GIL και μειώνει τον ανταγωνισμό πόρων. Και τα δύο συστήματα παρουσιάζουν ικανοποιητική συμπεριφορά σε εφαρμογές αυστηρά υπολογιστικού φόρτου (compute-bound). Τέλος, η εκτέλεση με επίγνωση της τοπολογίας (NUMA-aware) έδειξε οριακές μόνο βελτιώσεις στα συγκεκριμένα πειραματικά σενάρια.

Λέξεις-κλειδιά: Υπολογιστική Υψηλών Επιδόσεων, Εργασίες, Παράλληλος Προγραμματισμός με Εργασίες, Ετερογένεια, NUMA, Συστήματα Εκτέλεσης, Python, GIL, MPI

Abstract

This diploma thesis focuses on the comparative analysis and experimental evaluation of two different task-based parallel programming systems. As Python dominates scientific computing, the efficient utilization of modern hardware, such as complex NUMA architectures, is often hindered by inherent interpreter limitations, most notably the Global Interpreter Lock (GIL).

To address these challenges, two platforms with fundamentally different design philosophies are investigated: StarPUPy (a Python interface for StarPU, built for shared memory and heterogeneous systems) and torcpy (a Python-native system built upon the distributed MPI standard). The evaluation is conducted using three representative scientific applications: stochastic optimization using CMA-ES, image texture analysis using Gabor Filters, and a Monte Carlo simulation for financial derivative pricing.

Experimental results on a 64-core computational system demonstrate that StarPUPy excels in synchronization-bound applications due to its highly efficient lightweight thread management. Conversely, torcpy provides exceptional scalability and performance in memory-bound applications, as its MPI-process architecture successfully bypasses the GIL and significantly reduces memory contention. Both systems perform adequately well in strictly compute-bound applications. Finally, NUMA-aware execution configurations yielded only marginal improvements across these specific experimental scenarios.

Keywords: High Performance Computing, Tasks, Task-based Parallel Programming, Heterogeneity, NUMA, runtime systems, Python, GIL, MPI

Ευχαριστίες

Η εκπόνηση της παρούσας διπλωματικής εργασίας σηματοδοτεί την ολοκλήρωση του κύκλου των προπτυχιακών μου σπουδών στο Τμήμα Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής του Πανεπιστημίου Πατρών.

Αρχικά, θα ήθελα να εκφράσω τις θερμότερες ευχαριστίες μου στον επιβλέποντα της εργασίας μου, Αναπληρωτή Καθηγητή κ. Παναγιώτη Χατζηδούκα, για την εμπιστοσύνη που μου έδειξε, την πολύτιμη καθοδήγησή του και τη διαρκή υποστήριξή του καθ' όλη τη διάρκεια της έρευνας και της συγγραφής.

Επίσης, θα ήθελα να ευχαριστήσω τα υπόλοιπα μέλη της τριμελούς εξεταστικής επιτροπής, τον Καθηγητή κ. Σπυρίδωνα Σιούτα και τον Επίκουρο Καθηγητή κ. Γεώργιο Παπαδημητρίου, για τον χρόνο που διέθεσαν για την αξιολόγηση της προσπάθειάς μου.

Επιπλέον, θα ήθελα να πω ένα μεγάλο ευχαριστώ και στον υποψήφιο διδάκτορα, Ανάργυρο Σοφοτάσιο, για όλη την βοήθεια με τεχνικά ζητήματα κατά την διεξαγωγή των πειραμάτων στο εργαστήριο και καθώς και στην στηριξή του κατά την συγγραφή της εργασίας.

Ένα μεγάλο ευχαριστώ οφείλω στους φίλους και συμφοιτητές μου, με τους οποίους μοιραστήκαμε τις αγωνίες, τις χαρές και τον κοινό αγώνα των φοιτητικών μας χρόνων.

Τέλος, οφείλω βαθιά ευγνωμοσύνη στην οικογένειά μου, και ειδικά στη μητέρα και την αδελφή μου, για την αγάπη και την υπομονή τους. Χωρίς τη συνεχή συμπαράστασή τους σε όλη μου την πορεία, η ολοκλήρωση αυτού του στόχου δεν θα ήταν εφικτή.

Περιεχόμενα

Συντομογραφίες	xi
Κατάλογος σχημάτων	xii
Κατάλογος πινάκων	xiii
1 Εισαγωγή	1
1.1 Σκοπός και Αντικείμενο της Εργασίας	1
1.2 Δομή της Εργασίας	2
2 Θεωρητικό Υπόβαθρο	5
2.1 Παραλληλοποίηση με Εργασίες	5
2.2 Χρονοδρομολόγηση Εργασιών	6
2.3 Αρχιτεκτονική NUMA	7
2.4 Προκλήσεις της Αρχιτεκτονικής NUMA στην παραλληλοποίηση με Εργασίες	9
2.5 Παραλληλοποίηση στην Python	10
3 Το Σύστημα StarPU	13
3.1 Τι είναι το StarPU	13
3.2 Η Αρχιτεκτονική του StarPU	14
3.2.1 Οντότητες Εργασίας	14
3.2.2 Οντότητες Εκτέλεσης	15
3.2.3 Οντότητες Δεδομένων	16
3.3 Υποσύστημα Εικονικής Κοινής Μνήμης (VSM)	16
3.3.1 Δομές και Διεπαφές Δεδομένων	17
3.3.2 Πρωτόκολλο Συνοχής MSI (Modified, Shared, Invalid)	17
3.3.3 Προηγμένες λειτουργίες πρόσβασης δεδομένων	19
3.3.4 Κατάτμηση Δεδομένων και Ιεραρχικά Φίλτρα	19
3.3.5 Ασύγχρονες Μεταφορές και Προανάγνωση (Prefetching)	20
3.4 Χρονοπρογραμματισμός Εργασιών	20
3.4.1 Μοντέλα Απόδοσης Πραγματικού Χρόνου	20

3.4.2	Πολτικές Χρονοπρογραμματισμού Εργασιών	21
3.5	Κύκλος Εκτέλεσης Εσωτερικών Εργατών	23
3.5.1	Ανάκτηση Εργασιών και Διαχείριση Καταστάσεων	23
3.5.2	Χαλαρός Συγχρονισμός και Αποφυγή Αδιεξόδων	24
3.6	Κατανεμημένη Εκτέλεση: MPI	24
3.6.1	Υποβολή Εργασιών σε Κατανεμημένο Περιβάλλον	24
3.6.2	Βελτιστοποιήσεις Επικοινωνίας MPI	25
3.7	Σημαντικά Υποσυστήματα του StarPU	25
3.7.1	Out-of-Core Εκτέλεση	26
3.7.2	Context Hypervisors και Παράλληλες Εργασίες	26
3.7.3	Ανοχή Σφαλμάτων (Fault Tolerance)	26
4	Η Διεπαφή StarPUPy	27
4.1	Τι είναι το StarPUPy	27
4.2	Ασύγχρονη Υποβολή Εργασιών και Διαχείριση Αντικειμένων Future	27
4.3	Διαχείριση Μνήμης και Πρωτόκολλο Buffer	28
4.4	Παραλληλία και Αντιμετώπιση του GIL	29
4.5	Παράδειγμα Χρήσης του StarPUPy	30
4.5.1	Αποτελέσματα Εκτέλεσης	32
4.5.2	Ολοκληρωμένος Κώδικας Παραδείγματος	33
5	Το Σύστημα torcpy	35
5.1	Τι είναι το torcpy	35
5.2	Αρχιτεκτονική του torcpy	36
5.2.1	Δομή Εργασιών και Διαχείριση Αποτελεσμάτων	37
5.2.2	Ο Ρόλος και η Λειτουργία του Νήματος Εξυπηρετητή (Server Thread)	37
5.3	Μηχανισμός Εξισορρόπησης Φόρτου	38
5.4	Προγραμματιστική Διεπαφή (API) και Υποστήριξη Κληρονομημένου Κώδικα (SPMD)	39
5.4.1	Βασικές Ρουτίνες Διαχείρισης (Library Routines)	39
5.4.2	Μοντέλο Εκτέλεσης SPMD	41
5.5	Παράδειγμα Χρήσης του torcpy	41
5.5.1	Αποτελέσματα Εκτέλεσης	43
5.5.2	Ολοκληρωμένος Κώδικας Παραδείγματος	43
6	Εφαρμογές και Μεθοδολογία	45
6.1	Πειραματικές Εφαρμογές	45

6.1.1	Στοχαστική Βελτιστοποίηση CMA-ES με Επίλυση Εξίσωσης Poisson (app00)	45
6.1.2	Ανάλυση Υφής Εικόνων με Συστοιχίες Φίλτρων Gabor (app01)	47
6.1.3	Αποτίμηση Χρηματοοικονομικών Παραγώγων με Monte Carlo (app02)	49
6.2	Μεθοδολογία Αξιολόγησης και διεξαγωγής των Πειραμάτων	52
6.2.1	Εξασφάλιση Ισοδυναμίας Εκδόσεων και Δυσκολίες Υλοποίησης	52
6.2.2	Πειραματική Διαδικασία	53
7	Πειραματική Αξιολόγηση και Συμπεράσματα	57
7.1	Δοκιμές Δυνατοτήτων του StarPUPy και torcpy	57
7.1.1	StarPUPy: MPI Server-Client	57
7.1.2	Εμφολευμένη Παραλληλοποίηση (Nested Parallelism)	58
7.2	Αποτελέσματα Πειραμάτων και Συμπεράσματα	58
7.2.1	Ανάλυση Αποτελεσμάτων Πειραμάτων	61
7.2.2	NUMA-aware εκτέλεση των εφαρμογών	62
7.3	Συμπεράσματα	65
7.4	Μελλοντική Εργασία	65
	Βιβλιογραφία	67

Συντομογραφίες

HPC	High Performance Computing
CPU	Central Processing Unit
PU	Processing Unit
GPU	Graphics Processing Unit
FPGA	Field-Programmable Gate Array
FCFS	First-Come-First-Served
RR	Round Robin
NUMA	Non-Uniform Memory Access
SMP	Symmetric Multi-Processing
UMA	Uniform Memory Access
HEFT	Heterogeneous Earliest Finish Time
GIL	Global Interpreter Lock
PEP	Python Enhancement Proposal
VSM	Virtual Shared Memory
MPI	Message Passing Interface
API	Application Programming Interface
DAG	Directed Acyclic Graph
MSI	Modified, Shared, Invalid
MESI	Modified, Exclusive, Shared, Invalid
LRU	Least Recently Used
DMA	Direct Memory Access
SPMD	Single Program Multiple Data

Κατάλογος σχημάτων

2.1	Βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες. (Πηγή: [1]) . .	6
2.2	Αρχιτεκτονική Ομοιόμορφης Πρόσβασης Μνήμης (UMA) ή Συμμετρικής Πολυεπεξεργασίας (SMP). (Πηγή: [5])	8
2.3	Αρχιτεκτονική Μη Ομοιόμορφης Πρόσβασης Μνήμης (NUMA). (Πηγή: [5])	8
7.1	Αποτελέσματα Πειραμάτων - Standard (NUMA-oblivious) τρόπος εκτέλεσης.	59
7.2	Καλύτερες ρυθμίσεις εκτέλεσης για το torcpy για κάθε εφαρμογή.	60
7.3	Παράμετρος Παραλληλικής Αποδοτικότητας (Parallel Efficiency) για κάθε εφαρμογή και για κάθε αριθμό νημάτων εργασίας.	61
7.4	Αποτελέσματα Πειραμάτων - NUMA-aware εκτέλεση των εφαρμογών σε σύγκριση με την Standard (NUMA-oblivious) εκτέλεση.	63
7.5	Καλύτερες ρυθμίσεις εκτέλεσης για το torcpy για κάθε εφαρμογή (NUMA-aware).	64

Κατάλογος πινάκων

3.1	Καταστάσεις του πρωτοκόλλου MSI	18
3.2	Μοντέλα Απόδοσης στο StarPU	21
4.1	Επιλογές για την συνάρτηση task_submit του StarPUPy	28
6.1	Μεταβλητές περιβάλλοντος για NUMA-aware και Standard εκτέλεση των StarPUPy και torcpy.	55

Κεφάλαιο 1

Εισαγωγή

1.1 Σκοπός και Αντικείμενο της Εργασίας

Η ραγδαία εξέλιξη της υπολογιστικής επιστήμης και η ανάγκη για την ανάλυση όλο και μεγαλύτερου όγκου δεδομένων έχουν καταστήσει τον παράλληλο προγραμματισμό αναπόσπαστο κομμάτι της σύγχρονης έρευνας και τεχνολογίας. Καθώς η φυσική αύξηση της συχνότητας των επεξεργαστών (ρολόι) έχει φτάσει στα όριά της, η σύγχρονη υπολογιστική ισχύς πηγάζει πλέον κυρίως από αρχιτεκτονικές πολλαπλών πυρήνων (multi-core) και συστήματα συμμετρικής ή μη-συμμετρικής πολυεπεξεργασίας. Παράλληλα, γλώσσες υψηλού επιπέδου όπως η Python κυριαρχούν στους επιστημονικούς υπολογισμούς λόγω της ευκολίας χρήσης τους και του πλούσιου οικοσυστήματος βιβλιοθηκών τους, όμως η αποτελεσματική εκμετάλλευση του σύγχρονου υλικού (hardware) μέσω αυτής αποτελεί μια μείζονα πρόκληση, κυρίως λόγω εγγενών περιορισμών της γλώσσας στον ταυτόχρονο προγραμματισμό.

Αντικείμενο της παρούσας διπλωματικής εργασίας είναι η συγκριτική ανάλυση και πειραματική αξιολόγηση δύο διαφορετικών συστημάτων παράλληλου προγραμματισμού βασισμένου σε εργασίες (task-based parallel programming). Το μοντέλο αυτό προσφέρει ένα υψηλό επίπεδο αφάιρησης, όπου το πρόγραμμα διασπάται σε ανεξάρτητες μονάδες εκτέλεσης, τις "εργασίες" (tasks). Ο προγραμματιστής ορίζει τις εργασίες και τις μεταξύ τους εξαρτήσεις, ενώ το σύστημα χρόνου εκτέλεσης (runtime system) αναλαμβάνει δυναμικά να τις δρομολογήσει στους διαθέσιμους πυρήνες, να διαχειριστεί τη μνήμη και να εξισορροπήσει τον φόρτο εργασίας, απαλλάσσοντας τον χρήστη από τον χειροκίνητο και επίπονο έλεγχο των νημάτων.

Σκοπός της εργασίας είναι να μελετηθεί πώς τα συστήματα αυτά ανταποκρίνονται στις σύγχρονες προκλήσεις του υλικού και του λογισμικού. Η πρώτη μεγάλη πρόκληση αφορά τις αρχιτεκτονικές **NUMA (Non-Uniform Memory Access)**, οι οποίες κυριαρχούν στα συστήματα υψηλών επιδόσεων (HPC). Σε αυτές, η μνήμη είναι χωρισμένη σε τοπικά τμήματα ανά επεξεργαστή, με αποτέλεσμα η πρόσβαση σε απομακρυσμένα τμήματα μνήμης να προκαλεί καθυστερήσεις και συμφόρηση. Τα task-based συστήματα πρέπει, επομένως, να έχουν επίγνωση της τοπολογίας (NUMA-awareness) για να διατηρούν την

τοπικότητα των δεδομένων. Η δεύτερη και σημαντικότερη πρόκληση στο περιβάλλον της Python είναι το **Global Interpreter Lock (GIL)**, ένας μηχανισμός που επιτρέπει την εκτέλεση κώδικα Python από ένα μόνο νήμα ανά πάσα στιγμή. Σε *cpu-bound* εφαρμογές, η χρήση απλών νημάτων οδηγεί σε *GIL thrashing* (διαρκείς εναλλαγές για την απόκτηση του lock), προκαλώντας κατακερματισμό της χρήσης της CPU (ακόμα και στο 1.5%) και ακυρώνοντας τον παραλληλισμό.

Για την αντιμετώπιση αυτών των προβλημάτων, η εργασία εξετάζει και συγκρίνει δύο πλατφόρμες με θεμελιωδώς διαφορετικές αρχιτεκτονικές φιλοσοφίες:

- **StarPU (και η διεπαφή StarPUPy):** Ένα σύστημα χαμηλού επιπέδου σε C, σχεδιασμένο για *shared memory* και ετερογενή συστήματα. Θα επικεντρωθούμε στην Python διεπαφή του, StarPUPy για την πειραματική αξιολόγηση.
- **torcpy:** Ένα Python-native σύστημα βασισμένο στο καταναμημένο πρότυπο MPI (μέσω της `mpi4py`), ιδανικό για καταναμημένα συστήματα.

Η πειραματική αξιολόγηση και η συνεισφορά της εργασίας εστιάζουν στην υλοποίηση και σύγκριση των δύο συστημάτων πάνω σε τρεις αντιπροσωπευτικές *cpu-bound* εφαρμογές: έναν αλγόριθμο βελτιστοποίησης (CMA-ES με επίλυση Poisson PDE), μια ροή επεξεργασίας εικόνας (Gabor Filters), και μια προσομοίωση Monte Carlo (Basket Options). Μέσα από αυτές τις εφαρμογές, η εργασία ποσοτικοποιεί τον χρόνο εκτέλεσης σε συνθήκες ισχυρού παραλληλισμού (64 πυρήνες), αναδεικνύοντας τα όρια και τις βέλτιστες πρακτικές χρήσης του κάθε συστήματος.

1.2 Δομή της Εργασίας

Η παρούσα διπλωματική εργασία είναι οργανωμένη σε επτά συνολικά κεφάλαια, ακολουθώντας μια λογική πορεία από τη θεωρητική θεμελίωση στην πρακτική εφαρμογή και την ανάλυση:

- **Κεφάλαιο 2 -- Θεωρητικό Υπόβαθρο:** Παρουσιάζεται το θεωρητικό υπόβαθρο γύρω από την παραλληλοποίηση με εργασίες, την ετερογένεια και τις προκλήσεις των αρχιτεκτονικών NUMA, την παραλληλοποίηση σε Python και το GIL.
- **Κεφάλαιο 3 -- Το Σύστημα StarPU:** Αναλύονται οι δυνατότητες και η αρχιτεκτονική του StarPU σε περιβάλλον C, καθώς και ο τρόπος διαχείρισης των πόρων.
- **Κεφάλαιο 4 -- Η Διεπαφή StarPUPy:** Εξετάζεται η μεταφορά του StarPU στην Python, οι μέθοδοι διαχείρισης του GIL και οι ιδιαιτερότητες του χρονοπρογραμματισμού του.

- **Κεφάλαιο 5 -- Το Σύστημα torcpy:** Αναλύεται το μοντέλο εκτέλεσης του torcpy, το API του, ο τρόπος δρομολόγησης των tasks και η χρήση του MPI για την απομόνωση διεργασιών.
- **Κεφάλαιο 6 -- Εφαρμογές και Μεθοδολογία:** Περιγράφεται συνοπτικά ο κώδικας και η μαθηματική δομή των τριών εφαρμογών (CMA-ES, Image Processing, Monte Carlo), καθώς και ο τρόπος λήψης των μετρήσεων.
- **Κεφάλαιο 7 -- Πειραματική Αξιολόγηση και Συμπεράσματα:** Παρουσιάζονται τα διαγράμματα και τα αποτελέσματα των μετρήσεων, αναλύονται τα φαινόμενα saturation και κλιμάκωσης, και συνοψίζονται τα τελικά συμπεράσματα της σύγκρισης.

Κεφάλαιο 2

Θεωρητικό Υπόβαθρο

2.1 Παραλληλοποίηση με Εργασίες

Στον χώρο του HPC, η παραλληλοποίηση με εργασίες (task parallelism) είναι μια προσέγγιση, η πλέον πιο διαδεδομένη, που επιτρέπει την εκτέλεση πολλαπλών εργασιών ταυτόχρονα, όπου κάθε εργασία μπορεί να είναι ανεξάρτητη ή να έχει εξαρτήσεις από άλλες εργασίες. Αυτή η μέθοδος είναι ιδιαίτερα χρήσιμη όταν οι εργασίες έχουν διαφορετικά μεγέθη ή απαιτούν διαφορετικούς πόρους. Αυτό έχει ως αποτέλεσμα την καλύτερη εκμετάλλευση των διαθέσιμων πόρων του συστήματος και τη μείωση του χρόνου εκτέλεσης.

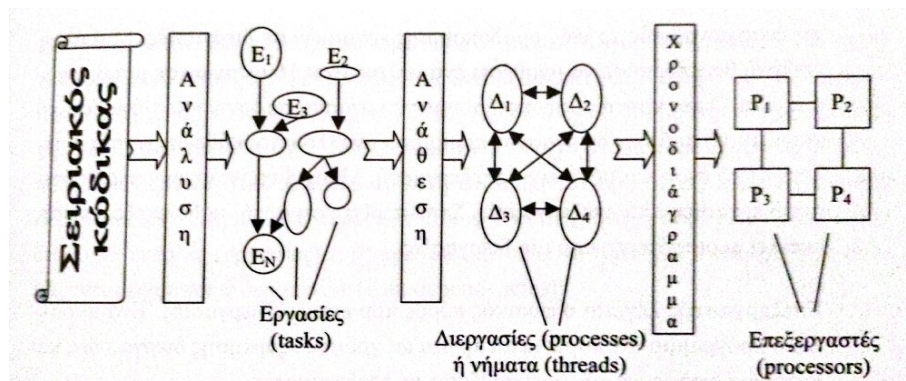
Η μετατροπή ενός αλγορίθμου για εκτέλεση σε ένα περιβάλλον παραλληλοποίησης με εργασίες απαιτεί διάφορα βήματα. Αρχικά, η υλοποίηση και η μελέτη του αλγορίθμου σε σειριακή μορφή σε μια γλώσσα προγραμματισμού θα βοηθήσει στην κατανόηση της λειτουργίας του και στην αναγνώριση των τμημάτων που μπορούν να παραλληλοποιηθούν. Ιδανικά αυτό θα θέλαμε να γίνεται αυτόματα από τον μεταγλωττιστή και το λειτουργικό σύστημα, αλλά στην πράξη απαιτείται η παρέμβαση του προγραμματιστή για να καθορίσει τις εργασίες και τις εξαρτήσεις τους [1].

Τα βήματα που πρέπει να λάβει υπόψιν ο προγραμματιστής στην διαδικασία παραλληλοποίησης ενός αλγορίθμου, είναι τα εξής [1]:

1. **Ανάλυση του Αλγορίθμου σε Εργασίες (tasks):** Ο αλγόριθμος πρέπει να αναλυθεί για να εντοπιστούν τα τμήματα που μπορούν να εκτελεστούν ανεξάρτητα. Αυτό μπορεί να περιλαμβάνει την ανάλυση των βρόχων, των συναρτήσεων και των δεδομένων που χρησιμοποιούνται.
2. **Ανάθεση σε Διεργασίες (processes) ή Νήματα (threads):** Οι εργασίες πρέπει να ανατεθούν σε διαφορετικές διεργασίες (ή νήματα) ή να εκτελεστούν σε διαφορετικούς πόρους, με βάση την διαθεσιμότητα και τις απαιτήσεις του κάθε ενός.
3. **Χρονοδρομολόγηση των Διεργασιών (scheduling):** Η χρονοδρομολόγηση των εργασιών είναι ένα σύνθετο βήμα που αναλύεται στα εξής:

- (α') **Ανάθεση κάθε Διεργασίας σε ένα Πόρο:** Κάθε διεργασία πρέπει να ανατεθεί σε έναν πόρο, όπως ένας επεξεργαστής ή ένας πυρήνας, με βάση τις απαιτήσεις της εργασίας και τη διαθεσιμότητα των πόρων.
- (β') **Απόφαση για την Εκτέλεση των Εργασιών:** Δηλαδή, πότε θα εκτελεστεί κάθε εργασία (task) στον συγκεκριμένο πόρο που του αντιστοιχίστηκε στο βήμα 3α'.

Τα παραπάνω βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες φαίνονται και στο παρακάτω Σχήμα 2.1.



Σχήμα 2.1: Βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες. (Πηγή: [1])

2.2 Χρονοδρομολόγηση Εργασιών

Όπως είδαμε στην προηγούμενη ενότητα, η χρονοδρομολόγηση των εργασιών είναι το πιο σύνθετο βήμα από τα υπόλοιπα, το οποίο στην σημερινή εποχή γίνεται όλο και πιο δύσκολο λόγω της αυξανόμενης πολυπλοκότητας των συστημάτων HPC και των εφαρμογών που εκτελούνται σε αυτά.

Η χρονοδρομολόγηση των εργασιών περιλαμβάνει την ανάθεση κάθε διεργασίας σε έναν πόρο, όπως ένας επεξεργαστής ή ένας πυρήνας, με βάση τις απαιτήσεις της εργασίας και τη διαθεσιμότητα των πόρων. Επιπλέον, πρέπει να ληφθεί υπόψη η απόφαση για το πότε θα εκτελεστεί κάθε εργασία (task) στον συγκεκριμένο πόρο που του αντιστοιχίστηκε. Αυτό καθιστά την επιλογή μιας αποτελεσματικής στρατηγικής (ή πολιτικής) χρονοδρομολόγησης κρίσιμη για την απόδοση των εφαρμογών σε περιβάλλοντα HPC.

Παλαιότερα ήταν αρκετό να γίνεται η χρήση απλών πολιτικών χρονοδρομολόγησης, όπως η First-Come-First-Served (FCFS) ή η Round Robin (RR), για να επιτευχθεί ικανοποιητική απόδοση. Ωστόσο, με την αύξηση της πολυπλοκότητας των συστημάτων και των εφαρμογών, αυτές οι απλές πολιτικές δεν είναι πλέον επαρκείς. Σήμερα, απαιτούνται πιο εξελιγμένες στρατηγικές χρονοδρομολόγησης που λαμβάνουν υπόψη παράγοντες

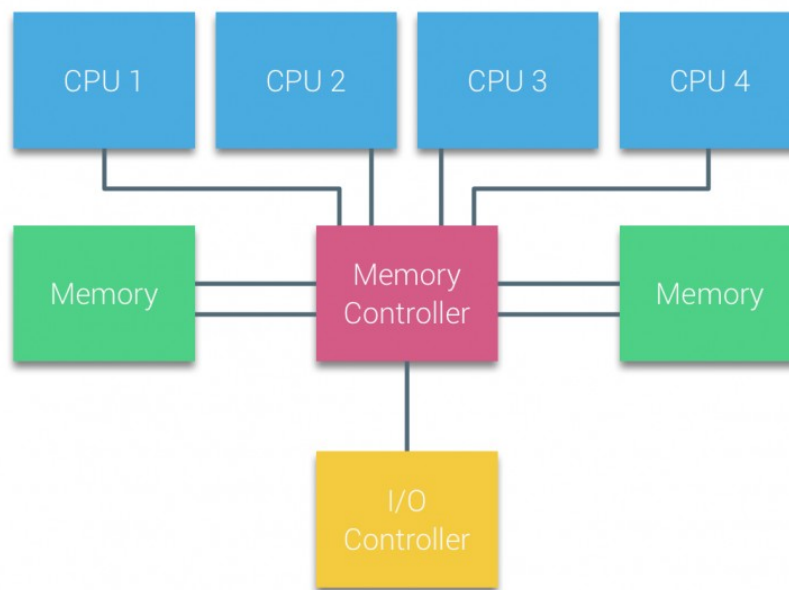
όπως οι εξαρτήσεις μεταξύ των εργασιών, η εκτίμηση του χρόνου εκτέλεσης κάθε εργασίας, η δυναμική κατάσταση των πόρων και η τοπικότητα των δεδομένων κάθε εργασίας στην μνήμη, διότι πλέον τα συστήματα είναι πολύπλοκα εισάγοντας έννοιες ετερογένειας (heterogeneity) και πολλαπλών επεξεργαστικών μονάδων, όπως οι αρχιτεκτονικές **NUMA**.

Οπότε θέλουμε η χρονοδρομολόγηση των εργασιών να γίνεται δυναμικά είτε από το λειτουργικό σύστημα είτε από το περιβάλλον εκτέλεσης (runtime environment) με βάση τις τρέχουσες συνθήκες του συστήματος και των εργασιών. Αυτό μπορεί να επιτευχθεί μέσω της χρήσης τεχνικών όπως **Work Stealing**, **HEFT** κ.α. που λαμβάνουν υπόψη τους διάφορους παράγοντες που αναφέρθηκαν προηγουμένως. Τέτοιες τεχνικές υλοποιούνται όπως θα δούμε σε παρακάτω κεφάλαια και από τη **torcpy** και τη **StarPU** [2, 3] στο επίπεδο του περιβάλλοντος εκτέλεσης.

2.3 Αρχιτεκτονική NUMA

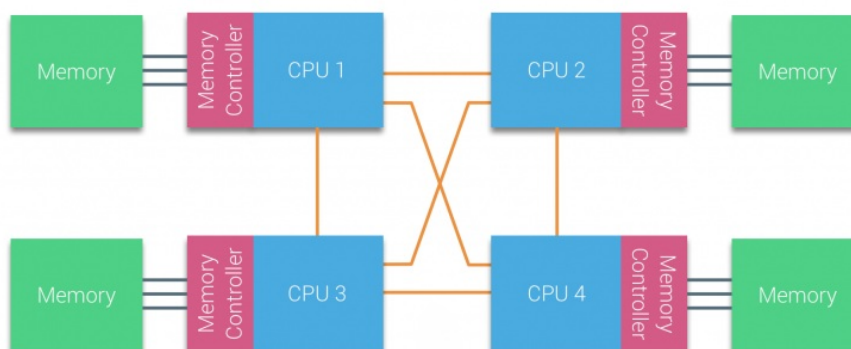
Όπως αναφέρθηκε και στην προηγούμενη ενότητα, η πολυπλοκότητα των συστημάτων **HPC** έχει αυξηθεί λόγω της διαρκούς εξέλιξης τους και ανάγκης για επίλυση πιο σύνθετων επιστημονικών και βιομηχανικών προβλημάτων. Ιστορικά, ωστόσο, το μεγαλύτερο εμπόδιο στην επίτευξη της μέγιστης δυνατής υπολογιστικής απόδοσης δεν ήταν η ταχύτητα των ίδιων των επεξεργαστών, αλλά ο περιορισμός που είναι ευρέως γνωστός ως το «εμπόδιο του Von Neumann» (Von Neumann bottleneck). Το φαινόμενο αυτό περιγράφει την εγγενή ανισορροπία μεταξύ της ταχύτητας επεξεργασίας των Κεντρικών Μονάδων Επεξεργασίας (**Central Processing Unit - CPU**) και της ταχύτητας μεταφοράς των δεδομένων από και προς την κύρια μνήμη του συστήματος, γεγονός που συχνά αφήνει τους επεξεργαστές σε κατάσταση αναμονής, στερημένους από τα απαραίτητα δεδομένα για την εκτέλεση των υπολογισμών [4].

Αρχικά, για να αντιμετωπιστεί αυτό το πρόβλημα, οι σχεδιαστές συστημάτων υιοθέτησαν μια αρχιτεκτονική που ονομάζεται Ομοιόμορφη Πρόσβαση Μνήμης (**Uniform Memory Access - UMA**), η οποία είναι γνωστή και ως Συμμετρική Πολυεπεξεργασία (**Symmetric Multi-Processing - SMP**), όπου πολλοί επεξεργαστές μοιράζονται την ίδια φυσική μνήμη, όπως φαίνεται και στο Σχήμα 2.2. Το βασικό πλεονέκτημα αυτής της προσέγγισης ήταν η απλότητά της από πλευράς προγραμματισμού. Όλοι οι επεξεργαστές λάμβαναν απολύτως ισότιμη και ομοιόμορφη πρόσβαση (ίσο χρόνο απόκρισης) σε οποιαδήποτε θέση της μνήμης. Ωστόσο, καθώς ο αριθμός των επεξεργαστών σε ένα σύστημα αυξανόταν, η συνολική απόδοση του συστήματος άρχισε να υποφέρει λόγω της αυξημένης συμφόρησης στη μνήμη [5].



Σχήμα 2.2: Αρχιτεκτονική Ομοιόμορφης Πρόσβασης Μνήμης (UMA) ή Συμμετρικής Πολυεπεξεργασίας (SMP). (Πηγή: [5])

Για την επίλυση του παραπάνω προβλήματος αναπτύχθηκε μια νέα αρχιτεκτονική που ονομάζεται Μη Ομοιόμορφη Πρόσβαση Μνήμης (**Non-Uniform Memory Access - NUMA**), όπου κάθε επεξεργαστής έχει τη δική του τοπική μνήμη, αλλά μπορεί επίσης να έχει πρόσβαση στη μνήμη άλλων επεξεργαστών. Παράδειγμα αυτής της αρχιτεκτονικής φαίνεται στο Σχήμα 2.3. Σε αυτή την αρχιτεκτονική, η πρόσβαση στη τοπική μνήμη ενός επεξεργαστή είναι ταχύτερη από την πρόσβαση στη μνήμη άλλων επεξεργαστών, γεγονός που εισάγει μια μη ομοιόμορφη συμπεριφορά στην πρόσβαση στη μνήμη. Η αρχιτεκτονική **NUMA** επιτρέπει την καλύτερη κλιμάκωση των συστημάτων με πολλούς επεξεργαστές, αλλά απαιτεί επίσης πιο περίπλοκες στρατηγικές χρονοδρομολόγησης και διαχείρισης μνήμης για να μεγιστοποιηθεί η απόδοση των εφαρμογών που εκτελούνται σε αυτά τα συστήματα [5, 6].



Σχήμα 2.3: Αρχιτεκτονική Μη Ομοιόμορφης Πρόσβασης Μνήμης (NUMA). (Πηγή: [5])

2.4 Προκλήσεις της Αρχιτεκτονικής NUMA στην παραλληλοποίηση με Εργασίες

Η φαινομενική απλότητα που προσφέρει η λογική αφαίρεση του μοντέλου των εργασιών έρχεται σε άμεση σύγκρουση με την εξαιρετικά πολύπλοκη φυσική πραγματικότητα του υλικού **NUMA**. Η εγγενής απόκρυψη των αρχιτεκτονικών λεπτομερειών από τον προγραμματιστή σημαίνει ότι το σύστημα χρόνου εκτέλεσης (runtime system) καλείται να λάβει κρίσιμες αποφάσεις σχετικά με την τοποθέτηση δεδομένων και τη δρομολόγηση των εργασιών, χωρίς πάντα να διαθέτει την πλήρη εικόνα της υποκείμενης τοπολογίας. Όταν τα αυτόματα συστήματα δρομολόγησης κατανέμουν εργασίες χωρίς επίγνωση της φυσικής τοποθεσίας των δεδομένων (**NUMA-oblivious scheduling**), οι επιπτώσεις στην απόδοση είναι καταστροφικές. Αυτή η αναντιστοιχία οδηγεί σε εκθετική αύξηση της κίνησης στους διαύλους διασύνδεσης (interconnect traffic), ακραία υποβάθμιση του εύρους ζώνης και αδυναμία γραμμικής κλιμάκωσης, διαμορφώνοντας ένα τοπίο στο οποίο η συγκριτική ανάλυση και η πειραματική αξιολόγηση των σύγχρονων συστημάτων παράλληλου προγραμματισμού καθίσταται θεμελιώδους σημασίας για την ακαδημαϊκή έρευνα [7].

Προβλήματα όπως τα παραπάνω καλούνται να λύσουν διάφορες τεχνικές που έχουν αναπτυχθεί για την βελτιστοποίηση της απόδοσης σε συστήματα **NUMA**. Μία τέτοια τεχνική είναι η **NUMA-aware Work Stealing (NUMA-WS)**, η οποία τροποποίησε τον πυρήνα του Intel Cilk Plus εφαρμόζοντας ολιστικά την αρχή του **Work-First**¹. Αντί να βασίζεται σε τεχνικές τυχαίας κλοπής που οδηγούν σε μεταναστεύσεις μεγάλου κόστους, το **NUMA-WS** προτείνει τη μηχανική ώθησης (work pushing). Το σύστημα αναθέτει στον εργάτη την προσπάθεια τοποθέτησης ενός υπολογιστικού πλαισίου (full frame) ακριβώς δίπλα στα δεδομένα του, εκβιάζοντας τη δημιουργία τοπικότητας. Το **NUMA-WS** για να αποφύγει να επιβαρύνει πολύ το τρέχων νήμα-εργάτη εφαρμόζει ώθηση δεδομένων μόνο σε τρία συγκεκριμένα σενάρια: κατά την εκτέλεση πράξεων συγχρονισμού, κατά την επιστροφή θυγατρικού νήματος σε αδρανές γονικό, και μετά από επιτυχή κλοπή εργασίας. Με αυτόν τον τρόπο, σύμφωνα με τη φιλοσοφία **Work-First**, το κόστος της βελτίωσης της τοπικότητας **NUMA** δεν επιβαρύνει ποτέ το ενεργό νήμα-εργάτη, αλλά αποσβένεται αποκλειστικά στον αδρανή νήμα-κλέφτη. Το «κατώφλι ώθησης» εγγυάται ότι οι συνολικές λειτουργίες παραμένουν σταθερές και ελεγχόμενες, αποτρέποντας τεράστιο φόρτο εργασίας. Στην πράξη, τα αποτελέσματα είναι εντυπωσιακά: ο πληθωρισμός έργου **Work Inflation**² μπορεί να μειωθεί σημαντικά στο **NUMA-WS**, με σημαντική βελτίωση της κλιμάκωσης σε συστήματα με 32 και πλέον πυρήνες [7].

Παρόμοιες τεχνικές έχουν αναπτυχθεί και για άλλες πλατφόρμες, όπως η **StarPU** [3]

¹Η αρχή του **Work-First** υποστηρίζει ότι νέες εργασίες πρέπει να εκτελούνται το συντομότερο δυνατόν.

²Το **Work Inflation** αναφέρεται στο φαινόμενο όπου ο συνολικός χρόνος εκτέλεσης που παράγουν όλα τα νήματα μαζί είναι μεγαλύτερος από την εργασία που θα εκτελούσε ένας μόνο επεξεργαστής σειριακά.

και η **torcpy** [2] που θα μελετηθούν στην παρούσα εργασία, οι οποίες υλοποιούν διάφορες στρατηγικές χρονοδρομολόγησης που λαμβάνουν υπόψη την τοπολογία **NUMA** για να βελτιώσουν την απόδοση των εφαρμογών που εκτελούνται σε τέτοια συστήματα.

2.5 Παραλληλοποίηση στην Python

Η **Python** είναι μια από τις πιο δημοφιλείς υψηλού επιπέδου γλώσσες προγραμματισμού σήμερα, γνωστή για την απλότητα, την ευκολία χρήσης της και το πλούσιο οικοσύστημα βιβλιοθηκών της.

Στην καρδιά της **Python (CPython)** βρίσκεται το **Gloabl Interpreter Lock (GIL)**, το οποίο είναι ένας μηχανισμός που επιτρέπει μόνο σε ένα νήμα λειτουργικού συστήματος να εκτελεί **Python bytecode** τη φορά, ακόμα και σε συστήματα με πολλούς επεξεργαστές. Αυτό σημαίνει ότι η παραλληλοποίηση με νήματα (threading) στην **Python** δεν μπορεί να εκμεταλλευτεί πλήρως τις δυνατότητες των πολυπύρηνων συστημάτων, περιορίζοντας την απόδοση των εφαρμογών που απαιτούν έντονη επεξεργασία [8, 9, 10]. Πιο συγκεκριμένα:

- **CPU-bound Εφαρμογές:** Για υπολογιστικά εντατικές εργασίες, το GIL λειτουργεί ως απόλυτο σημείο συμφόρησης, ακυρώνοντας εντελώς τα πλεονεκτήματα των πολυπύρηνων επεξεργαστών **NUMA**.
- **I/O-bound Εφαρμογές:** Για εργασίες που εξαρτώνται από αιτήματα δικτύου, λειτουργίες αρχείων κλπ, το GIL είναι εξαιρετικά αποδοτικό (τα νήματα απελευθερώνουν το GIL όταν περιμένουν).

Ωστόσο, υπάρχουν διάφορες βιβλιοθήκες και εργαλεία που έχουν αναπτυχθεί για να παρακάμψουν τους περιορισμούς του GIL και να επιτρέψουν την παραλληλοποίηση στην **Python** [11]. Μερικά από αυτά περιλαμβάνουν:

- **Multiprocessing Module:** Η βιβλιοθήκη **multiprocessing** επιτρέπει τη δημιουργία νέων διεργασιών, οι οποίες μπορούν να εκτελούνται παράλληλα και να εκμεταλλεύονται πολλούς επεξεργαστές, παρακάμπτοντας έτσι το GIL. Ωστόσο, η χρήση διεργασιών μπορεί να είναι πιο βαριά σε πόρους και να απαιτεί περισσότερη μνήμη σε σύγκριση με τα νήματα.
- **Threading Module:** Η βιβλιοθήκη **threading** επιτρέπει τη δημιουργία νήματων, αλλά λόγω του GIL, δεν είναι αποτελεσματική για CPU-bound εργασίες.
- **Asyncio Module:** Η βιβλιοθήκη **asyncio** παρέχει υποστήριξη για ασύγχρονο προγραμματισμό, επιτρέποντας την εκτέλεση πολλαπλών εργασιών (tasks) ταυτόχρονα χωρίς να μπλοκάρει το κύριο νήμα, αλλά δεν είναι κατάλληλη για CPU-bound εργασίες.

- **Concurrent.futures Module:** Η βιβλιοθήκη **concurrent.futures** παρέχει μια υψηλού επιπέδου διεπαφή για την εκτέλεση εργασιών (tasks) σε νήματα ή διεργασίες, επιτρέποντας την εύκολη παραλληλοποίηση, αλλά εξακολουθεί να υπόκειται στους περιορισμούς του GIL.

Κεφάλαιο 3

Το Σύστημα StarPU

3.1 Τι είναι το StarPU

Όπως έχουμε ήδη αναφέρει πολλές φορές είναι ότι τα συστήματα που χρησιμοποιούνται για εφαρμογές **HPC**, είναι πολύ περίπλοκα. Γι' αυτό το λόγο άλλωστε έχουμε αρχιτεκτονικές **NUMA**, που εισάγουν μη ομοιόμορφη πρόσβαση στη μνήμη, ενώ τα σύγχρονα HPC συστήματα συχνά συνδυάζουν και ετερογενείς (**heterogeneity**) επεξεργαστικές μονάδες (**Processing Units - PUs**). Δηλαδή ένα σύστημα NUMA μπορεί να έχει επεξεργαστικές μονάδες που είναι πέρα από **CPUs**, αλλά και **GPUs** ή **FPGAs** κ.α. Η ετερογένεια αυτή κάνει την ανάπτυξη εφαρμογών για τέτοια συστήματα πολύ δύσκολη, καθώς ο προγραμματιστής πρέπει να λάβει υπόψη του τις ιδιαιτερότητες και την υποκείμενη αρχιτεκτονική κάθε τύπου επεξεργαστικής μονάδας και να διαχειριστεί την κατανομή των εργασιών και των δεδομένων με τρόπο που να μεγιστοποιεί την απόδοση.

Την λύση σε αυτό το πρόβλημα έρχεται να δώσει το **StarPU**, ένα runtime system που παρέχει ένα ενοποιημένο περιβάλλον για την ανάπτυξη εφαρμογών σε ετερογενή συστήματα. Το StarPU επιτρέπει στους προγραμματιστές να γράφουν κώδικα που μπορεί να εκτελεστεί σε διάφορους τύπους επεξεργαστικών μονάδων χωρίς να χρειάζεται να ανησυχούν για τις λεπτομέρειες της αρχιτεκτονικής κάθε μονάδας. Παρέχει μηχανισμούς για την κατανομή των εργασιών, τη διαχείριση της μνήμης και την επικοινωνία μεταξύ των επεξεργαστικών μονάδων, επιτρέποντας έτσι την αποτελεσματική εκμετάλλευση των πόρων του συστήματος [3, 12].

Η θεμελιώδης αρχή λειτουργίας του StarPU είναι να «κάνει δυναμικά ό,τι δεν μπορεί πλέον να γίνει στατικά», καθιστώντας τις εφαρμογές φορητές και ιδιαίτερα κλιμακώσιμες χωρίς να θυσιάζεται η απόδοση χαμηλού επιπέδου [13]. Προσφέροντας ένα πλούσιο API σε C, C++ και Fortran, καθώς και υποστήριξη pragma OpenMP, το σύστημα εκτέλεσης απαλλάσσει τον προγραμματιστή από την αντιμετώπιση των λεπτομερειών υλικού χαμηλού επιπέδου, επιτρέποντάς του να επικεντρωθεί αποκλειστικά στην αλγοριθμική λογική [12].

Η παρούσα ανάλυση παρέχει μια ολοκληρωμένη και εξαντλητική παρουσίαση του τι επιτυγχάνει το StarPU και των περίπλοκων εσωτερικών μηχανισμών που χρησιμοποιεί για να το πετύχει. Η αξιολόγηση αναλύει τις βασικές αρχιτεκτονικές οντότητες, το υποσύστημα

Εικονικής Κοινής Μνήμης (**Virtual Shared Memory - VSM**), αλγορίθμους χρονοπρογραμματισμού εργασιών βασισμένους σε μοντέλα απόδοσης πραγματικού χρόνου, τον κύκλο εκτέλεσης των εσωτερικών εργασιών του συστήματος εκτέλεσης και την επέκταση αυτών των παραδειγμάτων σε περιβάλλοντα συστάδων κατανεμημένης μνήμης μέσω **MPI** [14, 15].

3.2 Η Αρχιτεκτονική του StarPU

Η αρχιτεκτονική του StarPU αποτελείται από διάφορα βασικά στοιχεία που συνεργάζονται για να παρέχουν το περιβάλλον εκτέλεσης για ετερογενείς εφαρμογές. Οι τρεις κύριες κατηγορίες οντοτήτων είναι οι εξής: Οντότητες Εργασίας, Οντότητες Εκτέλεσης και Οντότητες Δεδομένων. Αυτός ο τριμερής διαχωρισμός επιτρέπει στο σύστημα χρόνου εκτέλεσης να αποσυνδέσει τον αλγοριθμικό ορισμό ενός προβλήματος από τη φυσική εκτέλεσή του στο υλικό [16].

3.2.1 Οντότητες Εργασίας

Στην καρδιά του προγραμματιστικού μοντέλου του StarPU βρίσκονται τα **Codelets** (struct starpu_codelet). Ένα codelet ενσωματώνει έναν υπολογιστικό αλγόριθμο (ή πυρήνα υπολογισμού) και παρέχει πολλαπλές υλοποιήσεις για τις διάφορες διαθέσιμες επεξεργαστικές μονάδες (**PU**s) με βάση την υποκείμενη αρχιτεκτονική. Για παράδειγμα, ένα codelet σχεδιασμένο για την κλιμάκωση ενός διανύσματος μπορεί να περιέχει έναν απλό δείκτη συνάρτησης στην C για την εκτέλεση σε CPU, μια υλοποίηση CUDA για GPU και μια υλοποίηση OpenCL για συσκευές AMD. Η δομή του codelet επισημαίνει ποιο υλικό (hardware) υποστηρίζει μέσω μιας μάσκας bit (π.χ. STARPU_CPU | STARPU_CUDA) και υποδεικνύει τον αριθμό των buffer δεδομένων που χειρίζεται (nbuffers). Ενοποιώντας τις διάφορες υλοποιήσεις σε ένα ενιαίο codelet, το StarPU επιτρέπει στο σύστημα εκτέλεσης να επιλέξει δυναμικά την καταλληλότερη υλοποίηση με βάση τη διαθεσιμότητα των πόρων και την τοπικότητα των δεδομένων [12, 16, 17].

Η δεύτερη οντότητα εργασίας είναι τα **Tasks**. Ένα task είναι μια συγκεκριμένη εμφάνιση ενός codelet που έχει δεσμευτεί με ένα συγκεκριμένο σύνολο δεδομένων. Ένα task λειτουργεί ως μεσολαβητής μεταξύ του συστήματος χρόνου εκτέλεσης (runtime system) και της εφαρμογής. Κατά την υποβολή ενός task μέσω των βοηθητικών συναρτήσεων starpu_task_submit ή starpu_task_insert, ο προγραμματιστής καθορίζει το codelet, τα δεδομένα πάνω στα οποία θα εκτελεστεί (μέσω **Data Handles**) και τους τρόπους πρόσβασης (π.χ. STARPU_R, STARPU_W, STARPU_RW). Επιπλέον, η δομή του task υποστηρίζει προαιρετικά πεδία που παρέχουν υποδείξεις προς τον χρονοπρογραμματιστή, όπως επίπεδα προτεραιότητας, χρώμα εκτέλεσης για προφίλ και συναρτήσεις επανάκλησης

(callback_func) που ενεργοποιούνται μετά την επιτυχή ολοκλήρωση του task. Τα Tasks είναι εκ φύσεως ασύγχρονα από προεπιλογή, ωστόσο οι προγραμματιστές μπορούν να επιβάλουν σύγχρονη εκτέλεση ορίζοντας task->synchronous = 1 [12, 16, 17].

Η τελευταία οντότητα εργασίας είναι τα **Jobs**. Ένα job είναι μια δομή που χρησιμοποιεί εσωτερικά το StarPU για να αναπαραστήσει ένα task. Είναι μια χαμηλού επιπέδου οντότητα που διασχίζει το εσωτερικό γράφημα χρονοπρογραμματισμού. Ο διαχωρισμός μεταξύ tasks και jobs δίνει τη δυνατότητα για εξελιγμένες λειτουργίες όπως η παράλληλη εκτέλεση εργασιών, όπου μια μόνη δομή job μπορεί να συντονίσει πολλαπλές δομές παράλληλων tasks αντιστοιχισμένες σε μια ομάδα εργατών που χρησιμοποιούν παραλληλοποίηση σε επίπεδο νημάτων (όπως OpenMP). Η δομή job περιέχει εσωτερικά μεταδεδομένα που απαιτούνται για την ανάλυση εξαρτήσεων, την ουρά προτεραιοτήτων και την παρακολούθηση μοντέλων απόδοσης [12, 16, 17].

3.2.2 Οντότητες Εκτέλεσης

Η κεντρική οντότητα εκτέλεσης στο StarPU είναι ο **Worker**. Ένας worker είναι πάντα ένα νήμα (thread) CPU που έχει ως μοναδική ευθύνη την διαχείριση μιας συγκεκριμένης μονάδας υπολογισμού (PU). Για παράδειγμα, σε ένα σύστημα με 4 CPU και 2 GPU, το StarPU θα δημιουργήσει 6 workers: 4 για τους CPUs και 2 για τις GPUs. Ένας worker αν σχετίζεται με μια CPU είναι υπεύθυνος για την εκτέλεση των tasks. Αλλιώς στην περίπτωση που σχετίζεται με μια GPU ή άλλου είδους PU, τότε δρα σαν διαχειριστής για την συγκεκριμένη PU και δεν εκτελεί τον υπολογισμό. Δηλαδή, συνέχεια ρωτά για νέα tasks, μεταφέρει δεδομένα μέσω της διαύλου (bus) και στέλνει εντολές προς εκτέλεση στην PU που διαχειρίζεται. Αυτή η σχεδιαστική επιλογή είναι πολύ σημαντική, διότι επιτρέπει στο StarPU να διασφαλίζει ότι η διάυλος και οι ουρές εντολών της συσκευής είναι μόνιμα κορεσμένα, αποκρύπτοντας αποτελεσματικά την καθυστέρηση υλικού [16, 12, 15].

Η επόμενη οντότητα εκτέλεσης είναι οι **Drivers**. Οι drivers είναι οι διαμεσολαβητές μεταξύ των workers και του υλικού. Δίνουν στους worker ένα API ώστε να μπορούν να αρχικοποιούν το εκάστοτε υλικό, να χαρτογραφούν την μνήμη, να στέλνουν τα tasks στο εκάστοτε PU και να καθαρίζουν τα αχρείαστα δεδομένα μετά την εκτέλεση. Οι StarPU υποστηρίζει drivers για διάφορα είδη υλικού: CPU, CUDA, OpenCL, HIP, ακόμη και MPI Master-Slave διατάξης [12, 16].

Η τελευταία οντότητα εκτέλεσης είναι τα **Scheduling Contexts**. Για να μπορεί να γίνει διαχείριση των σύνθετων τοπολογιών υλικού, οι workers οργανώνονται σε scheduling contexts. Ένα scheduling context δεσμεύει μία ομάδα από workers σε ένα συγκεκριμένο αλγόριθμο χρονοπρογραμματισμού (scheduling algorithm). Για παράδειγμα, σε ένα σύστημα με 4 CPU και 2 GPU, το StarPU επιτρέπει στον προγραμματιστή να δημιουργήσει δύο scheduling contexts: ένα για τους 4 workers των CPU που χρησιμοποιούν έναν αλγόριθμο

χρονοπρογραμματισμού βασισμένο σε προτεραιότητα (priority-based scheduling) και ένα άλλο για τους 2 workers των GPU που χρησιμοποιούν έναν αλγόριθμο χρονοπρογραμματισμού βασισμένο σε απόδοση (performance-based scheduling). Αυτή η οργάνωση επιτρέπει στο StarPU να προσαρμόζει την στρατηγική χρονοπρογραμματισμού ανάλογα με τα χαρακτηριστικά του υλικού και τις απαιτήσεις της εφαρμογής [12, 16].

3.2.3 Οντότητες Δεδομένων

Η βασική οντότητα δεδομένων στο StarPU είναι το **Data Handle**. Ένα data handle είναι μια αδιαφανής αναφορά υψηλού επιπέδου σε ένα κομμάτι των δεδομένων της εφαρμογής. Λειτουργεί ως "λογική ταυτότητα" για τα δεδομένα, επιτρέποντας έτσι στο StarPU να παρακολουθεί να τα αιτήματα πρόσβασης (ανάγνωση, εγγραφή, ανάγνωση-εγγραφή) που κάνουν τα tasks. Έτσι το StarPU είναι σε θέση να αναλύει τις εξαρτήσεις μεταξύ των tasks και να φτιάχνει αυτόματα το γράφημα εξαρτήσεων (**DAG**) [12, 16].

Η δεύτερη οντότητα δεδομένων είναι το **Data Replicate**. Ένα data replicate είναι μια οντότητα χαμηλού επιπέδου που αντιπροσωπεύει ένα συγκεκριμένο φυσικό αντίγραφο των δεδομένων ενός data handle, το οποίο βρίσκεται αποθηκευμένο σε έναν συγκεκριμένο κόμβο μνήμης που τον διαχειρίζεται το StarPU. Για σε έναν σύγχρονο σύστημα HPC, κόμβοι μνήμης είναι η κύρια μνήμη RAM της CPU, η ενσωματωμένη μνήμη VRAM της GPU 0, η VRAM της GPU 1, ή ακόμα και ο σκληρός δίσκος. Ένα Data Replicate είναι η πραγματική, φυσική αποτύπωση των δεδομένων της εφαρμογής σε έναν από αυτούς τους χώρους. Τέλος, ένα data replicate δεν είναι εμφανές στην εφαρμογή αλλά είναι χρήσιμη μόνο για το StarPU όπως θα δούμε στην συνέχεια [12, 16].

3.3 Υποσύστημα Εικονικής Κοινής Μνήμης (VSM)

Σε ετερογενή συστήματα HPC, ο κύριος παράγοντας που περιορίζει την απόδοση είναι οι μεταφορές δεδομένων. Για να αξιοποιηθεί πλήρως το υλικό, ένα σύστημα χρόνου εκτέλεσης πρέπει να ελαχιστοποιήσει αυτές τις μεταφορές και να τις επικαλύψει με τους υπολογισμούς. Το StarPU το επιτυγχάνει εφαρμόζοντας την αφαίρεση της Εικονικής Κοινής Μνήμης (**Virtual Shared Memory - VSM**).

Αυτό το υποσύστημα του runtime system βλέπει όλες τις επεξεργαστικές μονάδες (PUs) και τους χώρους μνήμης τους ως διακριτούς κόμβους μνήμης. Το StarPU διαχειρίζεται αυτόματα την τοποθέτηση των δεδομένων, την μεταφορά τους και την συνοχή τους μεταξύ αυτών των κόμβων, παρέχοντας έτσι μια ενιαία, ενοποιημένη άποψη της μνήμης στην εφαρμογή.

Στην συνέχεια της υποενότητας, θα δούμε τις βασικές συνιστώσες του VSM και τις δυνατότητες που παρέχει.

3.3.1 Δομές και Διεπαφές Δεδομένων

Όπως αναφέραμε και στην υποενότητα 3.2.3, το StarPU χρησιμοποιεί δύο βασικές οντότητες δεδομένων: τα **Data Handles** και τα **Data Replicates**. Ένα data handle επιστρέφεται όταν ο προγραμματιστής εγγράφει ένα κομμάτι δεδομένων στο σύστημα χρόνου εκτέλεσης, έναντι του κλασικού τρόπου με pointers C. Η φυσική διάταξη και το αποτύπωμα μνήμης των δεδομένων περιγράφονται από μια διεπαφή δεδομένων (**Data Interface**). Το StarPU παρέχει αρκετές προκαθορισμένες διεπαφές δεδομένων για διάφορες δομές δεδομένων που συναντιούνται συχνά σε επιστημονικούς υπολογισμούς, όπως [12, 16]:

- **Variable/Scalar Interface:** Για μεμονωμένες αριθμητικές τιμές ή μικρές δομές σταθερού μεγέθους (starpu_variable_data_register).
- **Vector Interface:** Για μονοδιάστατους πίνακες με συνεχόμενα στοιχεία (starpu_vector_data_register).
- **Matrix & Block Interfaces:** Για δισδιάστατες και τρισδιάστατες πυκνές δομές δεδομένων, λαμβάνοντας υπόψη τις κύριες διαστάσεις (LD) για εξαγωγή υπο-πινάκων χωρίς περιορισμούς συνεχόμενης μνήμης (starpu_matrix_data_register).
- **Sparse Interfaces:** Ενσωματωμένη υποστήριξη για μορφές Compressed Sparse Row (CSR), Block Compressed Sparse Row (BCSR) και Coordinate (COO), ενεργοποιώντας σύνθετους επιλυτές αραιής γραμμικής άλγεβρας.
- **N-Dimensional Tensor Interface:** Για εργασίες βαθιάς μάθησης και πολυδιάστατους πίνακες.

Με την αφαίρεση των δεδομένων σε data handles, το StarPU μπορεί να παρεμβαίνει σε κάθε αίτημα πρόσβασης σε αυτά. Ένα data handle δεν αντιπροσωπεύει ένα μεμονωμένο φυσικό buffer, αλλά λειτουργεί ως ένα γενικό μηχανήμα κατάστασης που παρακολουθεί πολλαπλά data replicates. Πάνω σε αυτό βασίζεται ο μηχανισμός συνοχής MSI (Modified, Shared, Invalid) που θα αναλύσουμε στην επόμενη υποενότητα [12, 16].

3.3.2 Πρωτόκολλο Συνοχής MSI (Modified, Shared, Invalid)

Για να μπορεί το StarPU να διατηρεί την συνοχή των δεδομένων μεταξύ όλων των κόμβων μνήμης, χρησιμοποιεί ένα πρωτόκολλο συνοχής κρυφής μνήμης **MSI (Modified, Shared, Invalid)**. Αυτό το πρωτόκολλο κανονικά εφαρμόζεται συνήθως απευθείας σε επίπεδο υλικού (hardware-level), αλλά το StarPU το ανάγει σε επίπεδο λογισμικού (software-level) με την χρήση των προαναφερθέντων data handles και data replicates. Η λύση αυτή σε επίπεδο λογισμικού έναντι άλλων παρόμοιων λύσεων σε επίπεδο υλικού για συνοχή

κρυφής μνήμης (π.χ. MESI ¹ για x86 multi-core CPUs), είναι πιο αποτελεσματική και επιτυγχάνει πιο προβλέψιμη συνοχή δεδομένων, διότι πλέον το πρωτόκολλο εφαρμόζεται στο μακρο-επίπεδο δομών δεδομένων έναντι του μικρο-επιπέδου των cache-lines ² [18, 19].

Το πρωτόκολλο MSI του StarPU λειτουργεί ως εξής:

Κατάσταση	Μηχανισμός και Συνέπειες
Modified (M)	Το αντίγραφο δεδομένων σε αυτόν τον κόμβο μνήμης είναι το μοναδικό έγκυρο και ενημερωμένο. Όλα τα άλλα αντίγραφα σε άλλους κόμβους επισημαίνονται ως Invalid. Ενεργοποιείται όταν μια εργασία εκτελεστεί με STARPU_W ή STARPU_RW.
Shared (S)	Το αντίγραφο δεδομένων είναι έγκυρο και πανομοιότυπο με άλλα Shared αντίγραφα σε διαφορετικούς κόμβους. Πολλαπλές εργασίες σε διαφορετικές GPU μπορούν να εκτελούνται ταυτόχρονα, διαβάζοντας από τα τοπικά τους αντίγραφα.
Invalid (I)	Το αντίγραφο δεδομένων είναι παρωχημένο ή μη αρχικοποιημένο. Εάν μια εργασία που ανατέθηκε σε αυτόν τον κόμβο απαιτεί τα δεδομένα, το σύστημα εκτέλεσης πρέπει να εκκινήσει μεταφορά (π.χ. DMA) από τον κόμβο που κατέχει το Modified ή Shared αντίγραφο.

Πίνακας 3.1: Καταστάσεις του πρωτοκόλλου MSI

Σημαντικό είναι ότι τα δεδομένα δεν διαγράφονται επιθετικά μόλις ολοκληρωθεί μια εργασία. Τα αντίγραφα παραμένουν στην κρυφή μνήμη του κόμβου μνήμης σε κατάσταση Κοινή (Shared) ή Τροποποιημένη (Modified). Εάν μια επόμενη εργασία που έχει προγραμματιστεί στο ίδιο PU ζητήσει το ίδιο data handle, τα δεδομένα βρίσκονται ήδη εκεί, με αποτέλεσμα μηδενικό κόστος μεταφοράς μνήμης. Η διαγραφή δεδομένων συμβαίνει μόνο όταν εξαντληθεί η φυσική μνήμη της συσκευής, οπότε το StarPU χρησιμοποιεί ευριστική των Λιγότερο Πρόσφατα Χρησιμοποιημένων (LRU — **Least Recently Used**) ³ για να εκκαθαρίσει παρωχημένα δεδομένα πίσω στην κύρια μνήμη RAM ή στο δίσκο. Οι εφαρμογές μπορούν επίσης να παρέχουν υποδείξεις στο σύστημα χρόνου εκτέλεσης χρησιμοποιώντας τη συνάρτηση starpu_data_wont_use(), επιταχύνοντας την εκκαθάριση των συγκεκριμένων data handles [17, 18, 20].

¹Το πρωτόκολλο MESI (Modified, Exclusive, Shared, Invalid) είναι ένα πρωτόκολλο συνοχής κρυφής μνήμης παρόμοιο με το MSI.

²Τα cache lines είναι οι μικρότερες μονάδες δεδομένων που μπορούν να μεταφερθούν μεταξύ της κύριας μνήμης και της κρυφής μνήμης.

³Η ευριστική LRU (Least Recently Used) είναι μια μέθοδος διαχείρισης μνήμης που εκκαθαρίζει τα δεδομένα που δεν έχουν χρησιμοποιηθεί για το μεγαλύτερο χρονικό διάστημα.

3.3.3 Προηγμένες λειτουργίες πρόσβασης δεδομένων

Πέρα από τις τυπικές λειτουργίες Ανάγνωσης (Read) και Εγγραφής (Write), το StarPU εισάγει εξαιρετικά βελτιστοποιημένα παραδείγματα πρόσβασης προσαρμοσμένα για μαθηματικούς αλγόριθμους υψηλών επιδόσεων [12, 16]:

- **STARPU_REDUX:** Λειτουργώντας παρόμοια με τις reduction μεθόδους του OpenMP, αυτή η λειτουργία επιτρέπει σε πολλαπλά tasks να εκτελούνται ταυτόχρονα στο ίδιο data handle σε διαφορετικούς κόμβους. Το StarPU εκχωρεί δυναμικά buffers αρχικοποίησης ανά worker, επιτρέπει σε παράλληλα tasks να γράφουν ταυτόχρονα χωρίς κλείδωμα, και στη συνέχεια προγραμματίζει αυτόματα ένα δέντρο αναγωγής (reduction tree) από επακόλουθα tasks για τη συγχώνευση των διαφορετικών buffers στο τελικό αποτέλεσμα. Αυτό εξαλείφει πλήρως την ψευδή σειριοποίηση (false serialization) στους βρόχους συσσώρευσης.
- **STARPU_COMMUTE:** Σχεδιασμένο για μαθηματικά αντιμεταθετικές πράξεις. Τα tasks που ζητούν αντιμεταθετική πρόσβαση εγγραφής στην ίδια λαβή δεν χρειάζεται να είναι αυστηρά σειριοποιημένα. Το runtime επιτρέπεται να τα εκτελέσει με οποιαδήποτε σειρά, αυξάνοντας μαζικά τον διαθέσιμο παραλληλισμό, ενώ παράλληλα διασφαλίζει τη συνέπεια έναντι των τυπικών αναγνώσεων.
- **STARPU_SCRATCH:** Δίνει εντολή στο runtime να δεσμεύσει ένα προσωρινό, μη αρχικοποιημένο buffer στον κόμβο-στόχο αποκλειστικά για τη διάρκεια ζωής του task. Αυτό παρακάμπτει την ανάγκη του χρήστη να διαχειρίζεται προσωρινές λειτουργίες malloc ή cudaMalloc μέσα στο codelet, εξαλείφοντας τις δαπανηρές επιβαρύνσεις κατανομής.

3.3.4 Κατάτμηση Δεδομένων και Ιεραρχικά Φίλτρα

Η πυκνή γραμμική άλγεβρα και οι επιλυτές πεπερασμένων διαφορών βασίζονται σε μεγάλο βαθμό σε αλγόριθμους μπλοκ. Η καταχώρηση ενός μονολιθικού πίνακα 10GB ως ενιαία λαβή είναι υπολογιστικά καταστροφική. Μια μεμονωμένη λειτουργία εγγραφής θα ακύρωνε ολόκληρο τον πίνακα σε όλους τους επιταχυντές.

Το StarPU το επιλύει αυτό μέσω ενός προηγμένου συστήματος ιεραρχικής κατάτμησης που χρησιμοποιεί Φίλτρα (struct starpu_data_filter). Ένα κύριο data handle μπορεί να κατατμηθεί ασύγχρονα σε πολλαπλά θυγατρικά data handle. Για παράδειγμα, η starpu_matrix_filter_block() μπορεί να κόψει έναν πίνακα οριζόντια ή κάθετα, χωρίζοντάς τον σε ένα πλέγμα διακριτών υπο-πλακιδίων. Το πρωτόκολλο συνοχής MSI σέβεται αυτή την ιεραρχία. Τα tasks λειτουργούν πάντα στο πιο λεπτομερές διαθέσιμο τμήμα - τα φύλλα του δέντρου δεδομένων. Όταν ένας υπολογιστικός πυρήνας τροποποιεί ένα θυγατρικό

πλακίδιο, μόνο η κατάσταση του συγκεκριμένου μεταβαίνει σε Modified, αφήνοντας τον υπόλοιπο γονικό πίνακα απολύτως έγκυρο. Όταν η κατάτμηση αναιρεθεί μέσω της `starpu_data_unpartition()`, το StarPU συγκεντρώνει έξυπνα τα διάσπαρτα, τροποποιημένα θυγατρικά μπλοκ και τα γράφει πίσω στον συνολικό χώρο μνήμης του γονικού data handle, αποκαθιστώντας τη συνοχή [12, 16].

3.3.5 Ασύγχρονες Μεταφορές και Προανάγνωση (Prefetching)

Για να επιτευχθεί μέγιστη απόδοση υλικού, οι μεταφορές μνήμης πρέπει να αποκρύπτονται εντελώς πίσω από τον ενεργό υπολογισμό. Επειδή το StarPU διαθέτει ολιστική άποψη του γραφήματος εργασιών, διαθέτει ντετερμινιστική γνώση για το ακριβώς ποια data handles θα απαιτηθούν στο μέλλον. Μόλις ο χρονοπρογραμματιστής αναθέσει ένα μελλοντικό task σε μια PU, το υποσύστημα διαχείρισης εκδίδει αμέσως ένα ασύγχρονο αίτημα προανάγνωσης (`starpu_data_prefetch_on_node`). Μέχρι τη στιγμή που το μελλοντικό task γίνει ενεργό, η απαιτούμενη μνήμη του είναι ήδη τοπικά εγκατεστημένη, εξουδετερώνοντας πλήρως την καθυστέρηση μνήμης. Υποστηρίζεται επίσης το `STARPU_IDLEFETCH`, μια λειτουργία που βάζει στην ουρά μη κρίσιμες προαναγνώσεις όταν ο δίαυλος υλικού είναι εντελώς αδρανής [12, 16].

3.4 Χρονοπρογραμματισμός Εργασιών

Όπως αναφέραμε και στο Κεφάλαιο 2, ο χρονοπρογραμματισμός εργασιών είναι μια κρίσιμη συνιστώσα για την απόδοση των εφαρμογών σε ετερογενή συστήματα. Το StarPU παρέχει μια ευρεία γκάμα αλγορίθμων χρονοπρογραμματισμού που βασίζονται σε μοντέλα απόδοσης πραγματικού χρόνου (**real-time performance models**).

3.4.1 Μοντέλα Απόδοσης Πραγματικού Χρόνου

Αυτοί οι αλγόριθμοι χρησιμοποιούν εκτεταμένα προφίλ απόδοσης που συλλέγονται δυναμικά για να κάνουν ενημερωμένες αποφάσεις σχετικά με το πού και πότε να εκτελέσουν κάθε task [12, 16].

Παρακάτω στον Πίνακα 3.2 παρουσιάζονται τα κύρια μοντέλα απόδοσης που υποστηρίζει το StarPU, μαζί με μια σύντομη περιγραφή και τις περιπτώσεις χρήσης τους:

Τύπος Μοντέλου	Αλγοριθμικός Μηχανισμός	Βέλτιστες Περιπτώσεις Χρήσης
----------------	-------------------------	------------------------------

History-Based	Καταγράφει μέσους χρόνους εκτέλεσης σε βελτιστοποιημένο πίνακα κατακερματισμού, ευρετηριασμένο κατά μέγεθος δεδομένων και μονάδα εκτέλεσης.	Τακτικά φορτία με σταθερές διαστάσεις δεδομένων (π.χ. κατακερματισμένη γραμμική άλγεβρα).
Regression-Based	Προσαρμόζει ιστορικά δεδομένα εκτέλεσης σε μαθηματικές καμπύλες.	Φορτία με πολύ μεταβλητά μεγέθη δεδομένων.
Multiple Regression	Προηγμένη προσαρμογή πολυωνυμικής καμπύλης με πολλαπλές μεταβλητές.	Σύνθετα φορτία που εξαρτώνται από πολλαπλές διαστάσεις.
Energy Models	Παρακολουθεί κατανάλωση ισχύος (σε Joule) αντί για χρόνο εκτέλεσης.	Σενάρια που απαιτούν αυστηρό ανώτατο όριο ισχύος ή χρονοπρογραμματισμό με επίγνωση ενέργειας.

Πίνακας 3.2: Μοντέλα Απόδοσης στο StarPU

Εάν το σύστημα συναντήσει εργασία χωρίς ιστορικό στο μοντέλο απόδοσης, εκκινεί δυναμικά φάση βαθμονόμησης — εκτελεί αναγκαστικά την εργασία, τη χρονομετρεί και καταγράφει τη μετρική. Αυτά τα μοντέλα σειριοποιούνται και αποθηκεύονται στο δίσκο (συνήθως στο `.starpu/perfmodels`), βελτιώνοντας οργανικά την εξισορρόπηση φορτίου με την πάροδο του χρόνου. Οι προγραμματιστές μπορούν να ελέγξουν αυτή τη συμπεριφορά με μεταβλητές όπως η `STARPU_CALIBRATE=1` ή `STARPU_CALIBRATE=2`. Εργαλεία όπως το `starpu_perfmodel_display` επιτρέπουν την οπτικοποίηση αυτών των μετρικών εκτός σύνδεσης, ώστε να μπορούν διαπιστώσουν ότι το υλικό τρέχει όπως προβλέπεται [12, 16, 21].

3.4.2 Πολτικές Χρονοπρογραμματισμού Εργασιών

Οι παραπάνω μετρικές απόδοσης τροφοδοτούν μια ποικιλία πολιτικών χρονοπρογραμματισμού που καθορίζουν πώς το StarPU αντιστοιχίζει τα tasks στους διαθέσιμους workers. Μερικές από τις πιο σημαντικές πολιτικές περιλαμβάνουν [12, 15]:

- **Heterogeneous Earliest Finish Time - HEFT:** Το StarPU υλοποιεί μια δυναμική παραλλαγή του αλγορίθμου HEFT βασισμένο από την θεωρία στατικού χρονοπρογραμματισμού. Όταν ένα task μεταβαίνει στην κατάσταση "Ready", ο HEFT το

αξιολογεί ως προς κάθε διαθέσιμο worker. Υπολογίζει τον αναμενόμενο χρόνο εκτέλεσης χρησιμοποιώντας το μοντέλο απόδοσης που έχει ορισθεί. Υπολογίζει επίσης την προβλεπόμενη ποινή μεταφοράς δεδομένων, αξιολογώντας την τρέχουσα θέση των δεδομένων σε σχέση με τον κόμβο μνήμης του worker-στόχου. Στη συνέχεια, το HEFT αναθέτει το task στον worker που εγγυάται τον απολύτως ελάχιστο χρόνο ολοκλήρωσης. Αυτή η προσέγγιση εξισορροπεί επιθετικά την υπολογιστική ισχύ έναντι της καθυστέρησης μετακίνησης δεδομένων [3, 12, 17].

- **Deque Model Data Aware - DMDA:** Ο αλγόριθμος χρονοπρογραμματισμού DMDA βασίζεται στις αρχές του HEFT, αλλά βελτιστοποιείται για εξαιρετικά παράλληλα συστήματα. Διατηρεί ατομικές ουρές εργασιών για κάθε worker. Ταξινομεί ενεργά αυτές τις ουρές με βάση την προτεραιότητα και το κόστος μεταφοράς δεδομένων. Το DMDA μεροληπτεί σε μεγάλο βαθμό (biases) την τοποθέτηση των tasks προς τους workers των οποίων οι κόμβοι μνήμης διαθέτουν ήδη τα απαιτούμενα δεδομένα (Shared ή Modified). Μειώνοντας τις μετακινήσεις, ελαττώνει τον ανταγωνισμό του διαύλου (bus) και εξασφαλίζει υψηλότερο συνολικό κορεσμό των επεξεργαστών. Παραλλαγές όπως τα DMDAS και DMDAP ενσωματώνουν αυστηρή ταξινόμηση ανάλογα με την προτεραιότητα ή την ανάλυση κρίσιμης διαδρομής (critical path). Για παράδειγμα, σε μια παραγοντοποίηση Cholesky, χωρίς προτεραιοποίηση, θα μπορούσε να οδηγήσει σε λιμοκτονία τα κρίσιμα tasks. Το DMDAP διασφαλίζει ότι προωθούνται στο μπροστινό μέρος των ουρών [12, 14, 17].

Επίσης, το StarPU υποστηρίζει και πολιτικές χρονοπρογραμματισμού που δεν βασίζονται σε μοντέλα απόδοσης. Μερικές από αυτές είναι οι εξής [12]:

- **Eager:** Αυτός ο αλγόριθμος χρονοπρογραμματισμού χρησιμοποιεί μια κεντρική ουρά εργασιών (task queue), από την οποία όλοι οι workers αντλούν εργασίες για ταυτόχρονη εκτέλεση. Αυτό, όμως, δεν επιτρέπει την προανάγνωση (prefetching) δεδομένων, επειδή η απόφαση χρονοπρογραμματισμού λαμβάνεται αργά. Αν μια εργασία έχει προτεραιότητα διαφορετική από το 0, τοποθετείται στην αρχή της ουράς.
- **Random:** Αυτός ο αλγόριθμος χρονοπρογραμματισμού κάνει χρήση μιας ουράς εργασιών για κάθε worker και κατανέμει τυχαία τις εργασίες σε αυτούς με βάση την υποτιθέμενη γενική επίδοση του καθενός.
- **Work Stealing:** Αυτός ο αλγόριθμος χρονοπρογραμματισμού χρησιμοποιεί μια ουρά εργασιών για κάθε worker. Όταν ένας worker εξαντληθεί από εργασίες, είναι ανενεργός (idle), μπορεί να "κλέψει" εργασίες από τις ουρές άλλων workers, επιτρέποντας έτσι την δυναμική εξισορρόπηση φορτίου.

- **Heteroprio:** Αυτός ο αλγόριθμος χρονοπρογραμματισμού αναθέτει για κάθε διαφορετικό PU μια διαφορετική προτεραιότητα και κατανέμει τον φόρτο εργασίας ανάλογα με αυτήν. Απλώς αυτός ο αλγόριθμος θέλει σωστή ρύθμιση για να δουλέψει σωστά με βάση το documentation.

3.5 Κύκλος Εκτέλεσης Εσωτερικών Εργατών

Ο συνεχής κύκλος εκτέλεσης ενός νήματος worker είναι ένας εξαιρετικά βελτιστοποιημένος βρόχος ανάκτησης, εκτέλεσης και συγχρονισμού, σχεδιασμένος να μεγιστοποιεί την απόδοση του υλικού ενώ αποτρέπει την "λιμοκτονία" (starvation) των νημάτων και τα "αδιέξοδα" (deadlocks) σε περιβάλλοντα με υψηλή ταυτοχρονία (concurrency). Σε αντίθεση με έναν απλό βρόχο συμβάντων (event loop) που επεξεργάζεται ασύγχρονα μηνύματα, ο worker του StarPU λειτουργεί ως μια εξαιρετικά εξειδικευμένη μηχανή καταστάσεων.

3.5.1 Ανάκτηση Εργασιών και Διαχείριση Καταστάσεων

Όταν ένας worker είναι έτοιμος να λάβει κάποια εργασία, ρωτάει τον χρονοπρογραμματιστή που είναι συσχετισμένος με το scheduling context του. Κατά τη διάρκεια αυτής της διαδικασίας, ο worker μπαίνει σε μια κρίσιμη φάση που καθορίζεται από την συνάρτηση, `starpu_worker_enter_sched_op()`, όπου αλλάζει το εσωτερικό του flag, `state_sched_op_pending`, σε 0. Αυτή η κατάσταση σηματοδοτεί ότι ο worker αξιολογεί ενεργά την λίστα εργασιών (task queue), υπολογίζει εκτιμήσεις μοντέλων απόδοσης ή προσπαθεί να "κλέψει" δουλειά από άλλους workers [16].

Αυτό το flag κατάστασης λειτουργεί ως αυστηρό μέτρο ασφαλείας για την ταυτόχρονη εκτέλεση. Όσο είναι ενεργό το `state_sched_op_pending`, ο εργαζόμενος προστατεύεται από διαχειριστική παρέμβαση (administrative interference). Άλλα νήματα απαγορεύεται αυστηρά να αφαιρούν τον worker από το scheduling context του ή να αλλάζουν τις αναθέσεις της παράλληλης ομάδας του κατά τη διάρκεια αυτού του διαστήματος [16].

Εάν ένα εξωτερικό νήμα χρειαστεί να τροποποιήσει το πλαίσιο του worker, θέτει μια σημαία ειδοποίησης (`state_changing_ctx_notice = 10`). Αυτό σηματοδοτεί στον worker ότι δεν πρέπει να ξεκινήσει μια νέα λειτουργία χρονοπρογραμματισμού μετά την ολοκλήρωση της τρέχουσας, εμποδίζοντας το διαχειριστικό νήμα να υποστεί "λιμοκτονία". Εάν ο worker είναι ήδη βαθιά ενσωματωμένος σε μια λειτουργία χρονοπρογραμματισμού όταν φτάσει η ειδοποίηση, τίθεται μια δευτερεύουσα σημαία (`state_changing_ctx_waiting = 10`), η οποία δίνει εντολή στον worker να «ξυπνήσει» ενεργά το μπλοκαρισμένο διαχειριστικό νήμα μόλις καλέσει τη συνάρτηση `_starpu_worker_leave_sched_op` [16].

3.5.2 Χαλαρός Συγχρονισμός και Αποφυγή Αδιεξόδων

Μια βασική αρχιτεκτονική ιδέα στο σχεδιασμό του StarPU είναι η προσέγγισή του που βασίζεται στον «χαλαρό συγχρονισμό» (relaxed synchronization). Οι προηγμένοι αλγόριθμοι (όπως το Work Stealing) απαιτούν από τους workers να παρατηρούν το φορτίο, το μήκος της ουράς και την κατάσταση των ομότιμων workers τους για να λάβουν βέλτιστες αποφάσεις εξισορρόπησης φόρτου. Εάν το StarPU χρησιμοποιούσε αυστηρό κλείδωμα mutex ¹ (π.χ. ο Worker A κλειδώνει το sched_mutex του Worker B), αναπόφευκτα θα προκαλούσε αδιεξόδα (deadlocks) σε ένα αποκεντρωμένο σύστημα (ο Worker A κλειδώνει τον B, ενώ ο B προσπαθεί να κλειδώσει τον A) [16].

Ο χαλαρός συγχρονισμός επιτρέπει στους workers να διαβάζουν με ασφάλεια τα ατομικά flags κατάστασης των ομότιμών τους χωρίς να ξεκινούν ένα μπλοκάρισμα κλειδώματος. Η αναζήτηση πληροφοριών κατάστασης επιτρέπεται εάν `state_relax_refcnt > 0`, επιτρέποντας εξαιρετικά κλιμακούμενους αλγόριθμους κλοπής εργασίας και εξισορρόπησης φόρτου χωρίς κλειδώματα (lock-free) σε δεκάδες διακριτούς πυρήνες, χωρίς να σταματά η διοχέτευση εκτέλεσης. Μόλις ανακτηθεί επιτυχώς ένα task, ο worker χρησιμοποιεί τον οδηγό (driver) του για να ζητήσει από το υποσύστημα Εικονικής Κοινόχρηστης Μνήμης (VSM) να φέρει τα απαραίτητα data handles στην τοπική μνήμη. Μετά την επίλυση των μεταφορών μνήμης, ο worker καλεί τον κατάλληλο δείκτη συνάρτησης του codelet, εκτελεί τον μαθηματικό πυρήνα και επικαλείται τυχόν καταχωρημένες συναρτήσεις επανάκλησης προτού επιστρέψει στην αρχή του βρόχου [16].

3.6 Κατανεμημένη Εκτέλεση: MPI

Παρόλο που το StarPU βελτιστοποιεί την αξιοποίηση του υλικού ενός κόμβου με διάφορες ετερογενείς μονάδες επεξεργασίας (PU), η ανάγκη για περεταίρω κλιμάκωση σε πολλαπλούς κόμβους είναι εξίσου σημαντική στις σημερινές εφαρμογές HPC. Για να υποστηρίξει αυτό το σενάριο, το StarPU ενσωματώνει ένα υποσύστημα κατανεμημένης εκτέλεσης βασισμένο στο MPI (Message Passing Interface), μετατρέποντας ένα μεγάλο cluster σε ένα ενιαίο ετερογενές σύστημα [12, 16].

3.6.1 Υποβολή Εργασιών σε Κατανεμημένο Περιβάλλον

Ιστορικά, ο κατανεμημένος προγραμματισμός απαιτούσε τη χειροκίνητη παρεμβολή ρουτινών MPI_Send και MPI_Recv. Το StarPU εξαλείφει πλήρως αυτό το παράδειγμα μέσω του `starpup_mpi_task_insert`. Ο προγραμματιστής καταχωρεί τα data handles με έναν συ-

¹Ένας **mutex** (από το mutual exclusion, αμοιβαίος αποκλεισμός) είναι ένας μηχανισμός συγχρονισμού στην επιστήμη των υπολογιστών που χρησιμοποιείται για την προστασία κοινόχρηστων πόρων.

γκεκριμένο βαθμό (rank) MPI, χρησιμοποιώντας τη συνάρτηση `starpu_mpi_data_register`. Αυτό καθιερώνει την έννοια της «ιδιοκτησίας» για τα εν λόγω δεδομένα. Κατά την εκτέλεση, κάθε διεργασία MPI στο cluster εκτελεί τον ίδιο ακριβώς κώδικα εφαρμογής, ξεδιπλώνοντας το ίδιο ακριβώς καθολικό γράφημα εργασιών [12].

Όταν καλείται το `starpu_mpi_task_insert`, η μηχανή StarPU-MPI εκτελεί έναν ντετερμινιστικό αλγόριθμο συναίνεσης:

1. **Ανάθεση Κόμβου Εκτέλεσης:** Ο κόμβος MPI που κατέχει το data handle σε λειτουργία STARPU_W ή STARPU_RW επιλέγεται ως ο εκτελεστής του task. Μπορεί να παρακαμφθεί μέσω του STARPU_EXECUTE_ON_NODE.
2. **Έμμεση Δρομολόγηση Δεδομένων (Send):** Εάν ένας κόμβος MPI κατέχει δεδομένα που απαιτούνται (Ανάγνωση), αλλά δεν είναι ο κόμβος εκτέλεσης, το StarPU δημοσιεύει αυτόματα ένα ασύγχρονο MPI_Isend για τη μετάδοση.
3. **Έμμεση Δρομολόγηση Δεδομένων (Receive):** Ο κόμβος εκτέλεσης δημοσιεύει αυτόματα MPI_Irecv για τη συλλογή των δεδομένων Ανάγνωσης.
4. **Εκτέλεση και Επαναγραφή (Write-back):** Ο κόμβος εκτέλεσης λαμβάνει τα δεδομένα, επεξεργάζεται το codelet και τροποποιεί το τοπικό data handle Εγγραφής. Εάν τα δεδομένα πρέπει να ωθηθούν πίσω σε έναν ιδιοκτήτη, το StarPU χειρίζεται το MPI_Isend αυτόματα.

Το καινοτόμο με τον παραπάνω μηχανισμό είναι ότι ο βασικός αλγοριθμικός κώδικας δεν απαιτεί απολύτως καμία δήλωση επικοινωνίας MPI. Ο ίδιος ακριβώς σειριακός κώδικας κλιμακώνεται απρόσκοπτα σε χιλιάδες κόμβους [12].

3.6.2 Βελτιστοποιήσεις Επικοινωνίας MPI

Για να αποτραπεί η καθυστέρηση της CPU, το StarPU αφιερώνει ένα αποκλειστικό, καρφίτωμένο (pinned) νήμα CPU αποκλειστικά στη μηχανή προόδου του MPI. Αυτό το νήμα παρασκηνίου ελέγχει συνεχώς το δίκτυο, εκδίδοντας κλήσεις MPI_Test ασύγχρονα χωρίς να επιρεάζει του τοπικούς εργάτες (workers). Επιπλέον, για την ελαχιστοποίηση της συμφόρησης του δικτύου, το StarPU-MPI υλοποιεί μια έξυπνη κρυφή μνήμη επικοινωνίας (`starpu_mpi_cache_set`) [12].

3.7 Σημαντικά Υποσυστήματα του StarPU

Το StarPU πέρα από όλα τα βασικά συστήματα που αναλύσαμε στις προηγούμενες ενότητες, διαθέτει και άλλα σημαντικά υποσυστήματα που συμβάλλουν στην απόδοση και την ευελιξία του. Μερικά από αυτά είναι θα τα δούμε σύντομα παρακάτω.

3.7.1 Out-of-Core Εκτέλεση

Όταν το μέγεθος του προβλήματος υπερβαίνει τη διαθέσιμη κύρια RAM, το σύστημα παραδοσιακά θα κατέρρεε. Το StarPU το λύνει αυτό ενσωματώνοντας μονάδες δίσκου υψηλής ταχύτητας (NVMe) στην ιεραρχία Εικονικής Κοινόχρηστης Μνήμης (**VSM**) ως διακριτούς Κόμβους Μνήμης Δίσκου. Η πολιτική **LRU** προωθεί ασύγχρονα τα data handles απευθείας στον δίσκο. Όταν ζητηθούν ξανά, το runtime τα φέρνει πίσω στη RAM ή ξεκινά μια μεταφορά Άμεσης Πρόσβασης Μνήμης (**DMA**) απευθείας από τον αποθηκευτικό χώρο στη μνήμη της PU, παρακάμπτοντας εντελώς τη CPU [12].

3.7.2 Context Hypervisors και Παράλληλες Εργασίες

Για εξαιρετικά ακανόνιστους (irregular) φόρτους εργασίας, η στατική κατανομή workers μπορεί να οδηγήσει σε υποαπασχόληση του υλικού. Το StarPU εισάγει τον Scheduling Context Hypervisor, ο οποίος αναλύει τους χρόνους αδράνειας, τις ταχύτητες εκτέλεσης και τα μήκη των ουρών σε πραγματικό χρόνο. Χρησιμοποιώντας όρια γραμμικού προγραμματισμού, ο hypervisor αλλάζει αυτόματα το μέγεθος των πλαισίων, μεταφέροντας αδρανείς workers CPU, GPU ή γενικά PU από ένα εξαντλημένο πλαίσιο σε ένα κορεσμένο. Το StarPU υποστηρίζει εγγενώς παράλληλα tasks μέσω OpenMP (starpu_omp_init). Ένα ενιαίο **Codelet** μπορεί να ενσωματώσει μια παράλληλη περιοχή του OpenMP. Το StarPU θα αναθέσει το Job σε μια ολόκληρη ομάδα workers της CPU ταυτόχρονα, με το σωστό καρφίτσωμα πυρήνα και επίγνωση της αρχιτεκτονικής **NUMA** [12].

3.7.3 Ανοχή Σφαλμάτων (Fault Tolerance)

Σε τεράστια κατανεμημένα υπολογιστικά συστήματα, οι αστοχίες κόμβων είναι στατιστικά αναπόφευκτες. Το StarPU παρέχει ενσωματωμένη υποστήριξη μέσω λήψης σημείων ελέγχου (checkpointing) (starpu_mpi_checkpoint_template_register). Το runtime επιτρέπει τον ορισμό προτύπων σημείων ελέγχου που «παγώνουν» (freeze) την κατάσταση συγκεκριμένων data handles. Εάν ένας κόμβος αποτύχει καταστροφικά, η εφαρμογή μπορεί να επιστρέψει (roll back) σε ένα σημείο ελέγχου και να επαναπρογραμματίσει δυναμικά το χαμένο γράφημα εργασιών, εξασφαλίζοντας ότι μεγάλης κλίμακας προσομοιώσεις επιβιώνουν από τη φθορά του υλικού [12].

Κεφάλαιο 4

Η Διεπαφή StarPUPy

4.1 Τι είναι το StarPUPy

Το **StarPUPy** είναι μια διεπαφή προγραμματισμού στην γλώσσα υψηλού επιπέδου **Python** και μεταφέρει τις δυνατότητες του StarPU σε ένα φιλικό προς τον χρήστη περιβάλλον. Στόχος του είναι να επιτρέψει στους προγραμματιστές να εκμεταλλευτούν τις δυνατότητες του StarPU σε συνδυασμό με την ευκολία, την απλότητα και το πλούσιο οικοσύστημα της Python.

Ως επί το πλείστον, λειτουργεί ως ένα wrapper γύρω από τις βασικές λειτουργίες του StarPU στην χαμηλού επιπέδου γλώσσα προγραμματισμού **C**, επιτρέποντας στους επιστήμονες δεδομένων που χρησιμοποιούν αποκλειστικά Python να αξιοποιήσουν τη δύναμη του ετερογενούς χρονοπρογραμματισμού του C backend, διευκολύνοντας την ενσωμάτωση με βιβλιοθήκες όπως το **NumPy** και το **SciPy**, οι οποίες κυριαρχούν στην έρευνα [12].

4.2 Ασύγχρονη Υποβολή Εργασιών και Διαχείριση Αντικειμένων Future

Η υποβολή εργασιών στο StarPUPy βασίζεται στη χρήση της συνάρτησης `task_submit(options)(func, *args, **kwargs)`. Η κλήση αυτής της συνάρτησης δεν εκτελείται άμεσα, αλλά επιστρέφει ένα αντικείμενο **Future** της βιβλιοθήκης **asyncio** της Python. Τα αντικείμενα **Future** αποτελούν **awaitable** οντότητες που αντιπροσωπεύουν την μελλοντική ολοκλήρωση μιας ασύγχρονης εργασίας. Αυτό επιτρέπει στους προγραμματιστές να υποβάλλουν εργασίες και να περιμένουν τα αποτελέσματα από αυτές χωρίς να μπλοκάρτε το βασικό νήμα (**main thread**) εκτέλεσης [12].

Στον Πίνακα 4.1 παρουσιάζονται τα βασικά στοιχεία της συνάρτησης `task_submit` και των παραμέτρων της [12]:

Επιλογή (Option)	Τύπος Δεδομένων	Προεπιλεγμένη Τιμή Χρήσης	Λειτουργική Επίδραση
name	string	None	Ορισμός αναγνωριστικού ονόματος της εργασίας για χρήση σε εργαλεία αποσφαλμάτωσης.
synchronous	unsigned	0	Εάν οριστεί σε 1, η υποβολή μπλοκάρει την εκτέλεση μέχρι την ολοκλήρωση της εργασίας.
priority	int	0	Καθορίζει το επίπεδο προτεραιότητας της εργασίας εντός του εύρους τιμών των sched_get_min_priority() και sched_get_max_priority().
color	unsigned	None	Ορισμός χρώματος της εργασίας για την οπτικοποίηση του γραφήματος εξαρτήσεων (DAG) στο εργαλείο dag.dot.
flops	double	None	Δηλώνει το υπολογιστικό μέγεθος σε πράξεις κινητής υποδιαστολής για τη δημιουργία καμπυλών απόδοσης.
perfmodel	string	None	Καθορίζει το όνομα του μοντέλου επίδοσης για την ιστορική βαθμονομημένη χρονοδρομολόγηση.

Πίνακας 4.1: Επιλογές για την συνάρτηση task_submit του StarPUPy

Η Python 3.8+ υποστηρίζει την εκτέλεση με την παράμετρο `-m asyncio`, η οποία επιτρέπει την άμεση χρήση της εντολής `await` στο διαδραστικό περιβάλλον (REPL ¹). Επιπλέον, παρέχεται ο διακοσμητής `@starpupy.delayed`, ο οποίος μετατρέπει μια τυπική συνάρτηση σε ασύγχρονη εργασία StarPU, δεχόμενος τις ίδιες παραμέτρους χρονοδρομολόγησης [12].

4.3 Διαχείριση Μνήμης και Πρωτόκολλο Buffer

Ένα από τα σημαντικότερα χαρακτηριστικά του StarPUPy είναι η ενσωμάτωση του πρωτοκόλλου buffer (buffer protocol) της Python. Το πρωτόκολλο αυτό επιτρέπει στο StarPUPy να αποκτά άμεση πρόσβαση στα ακατέργαστα byte arrays αντικειμένων, όπως οι πίνακες NumPy, χωρίς την ανάγκη αντιγραφής ή σειριοποίησης.

Το σύστημα διατήρησης της συνοχής μνήμης του StarPU απαιτεί τη δήλωση του τρόπου πρόσβασης σε κάθε μεταβλητή. Αυτό επιτυγχάνεται με τη χρήση του διακοσμητή

¹Το **REPL (Read-Eval-Print Loop)** είναι ένα διαδραστικό περιβάλλον προγραμματισμού όπου εισάγετε μια εντολή (Read), ο υπολογιστής την εκτελεί άμεσα (Eval) και εμφανίζει το αποτέλεσμα (Print), επαναλαμβάνοντας τη διαδικασία (Loop) για την επόμενη εντολή.

@starpu.access:

- **R (Read):** Ανάγνωση δεδομένων (προεπιλογή).
- **W (Write):** Αποκλειστική εγγραφή δεδομένων.
- **RW (Read-Write):** Ανάγνωση και τροποποίηση των δεδομένων.

Όταν ένα αντικείμενο υποβάλλεται σε μια εργασία με δικαιώματα εγγραφής (W ή RW), η εφαρμογή Python δεν μπορεί να προσπελάσει τις τιμές του άμεσα, καθώς αυτές ενδέχεται να βρίσκονται σε μια διακριτή μνήμη PU. Για την προσπέλαση εκτός των εργασιών StarPU, χρησιμοποιείται το σχήμα Acquire/Release. Εάν η εφαρμογή επιχειρήσει να καταχωρίσει το ίδιο μεταβαλλόμενο (mutable) αντικείμενο Python περισσότερες από μία φορές στο σύστημα διαχείρισης δεδομένων, το StarPUPy εγείρει μια ρητή εξαίρεση ασφαλείας, *"starpu.error: Should not register the same mutable python object once more."* [12].

Επιπλέον, οι πίνακες NumPy πολλών διαστάσεων μπορούν να υποστούν κατάτμηση σε Segment Chunks για παράλληλη επεξεργασία μέσω της μεθόδου `Handle::partition(nchildren, dim, chunks_list)`. Η φυσική διεύθυνση μνήμης και το μέγεθος σε byte για κάθε υπο-πίνακα υπολογίζονται αυτόματα από το StarPUPy. Μετά την ολοκλήρωση των παράλληλων υπολογισμών στα sub-handles, η κλήση της `Handle::unpartition(handle_list, nchildren)` ενσωματώνει τα αποτελέσματα στον αρχικό πίνακα, διασφαλίζοντας τη συνοχή των δεδομένων στην κύρια μνήμη RAM [12].

Τέλος, το StarPUPy υποστηρίζει την χρήση data handles όπως και το StarPU C API, επιτρέποντας την εγγραφή σε δεδομένα που βρίσκονται σε διακριτή μνήμη PU. Αυτό επιτυγχάνεται με τη χρήση της κλάσης `Handle`¹ και την δημιουργία ενός αντικειμένου πάνω στα επιθυμητά δεδομένα. Μετά την εγγραφή, το αντικείμενο μπορεί να χρησιμοποιηθεί σε εργασίες StarPU με τα κατάλληλα δικαιώματα πρόσβασης [12].

4.4 Παραλληλία και Αντιμετώπιση του GIL

Η επίτευξη πραγματικής παραλληλίας σε περιβάλλον Python περιορίζεται παραδοσιακά από το **Global Interpreter Lock (GIL)**, το οποίο επιβάλλει τη σειριακή εκτέλεση του πηγαίου κώδικα Python. Το StarPUPy αντιμετωπίζει αυτή την πρόκληση μέσω διάφορων στρατηγικών [12]:

- **Αποδέσμευση GIL μέσω C-APIs:** Κατά την κλήση εξωτερικών βιβλιοθηκών (π.χ. πράξεις γραμμικής άλγεβρας του NumPy που βασίζονται σε βιβλιοθήκες BLAS), το

¹Η κλάση `Handle` είναι μέρος του module `starpu`.

GIL απελευθερώνεται αυτόματα. Αυτό επιτρέπει στους εργάτες CPU του StarPU να εκτελούν τις εργασίες ταυτόχρονα.

- **Multi-interpreter (Python 3.12+):** Η νεότερη αρχιτεκτονική της Python επιτρέπει την ύπαρξη ενός ανεξάρτητου GIL ανά διεργασία (PEP 0684). Το StarPUPy υποστηρίζει πλήρως αυτή τη λειτουργία μέσω της μεταβλητής `STARPU_PY_OWN_GIL=1`, αν και ορισμένες λειτουργίες, όπως τα `asyncio futures`, παρουσιάζουν περιορισμούς συμβατότητας στην τρέχουσα έκδοση.
- **MPI - Master-Slave:** Σε περιβάλλοντα με πολλαπλούς κόμβους, το StarPUPy μπορεί να χρησιμοποιήσει το **Message Passing Interface (MPI)**, που είναι υποστηριζόμενο από το backend στην γλώσσα C, για την επικοινωνία μεταξύ διεργασιών. Σε αυτό το σενάριο, ο κύριος κόμβος (master) διαχειρίζεται την υποβολή εργασιών και τη συλλογή αποτελεσμάτων, ενώ οι δούλοι (slaves) εκτελούν τις εργασίες ανεξάρτητα, παρακάμπτοντας το GIL.

Αν και οι τελευταίες δύο στρατηγικές είναι υποσχόμενες δεν υποστηρίζονται και δεν τεκμηριώνονται πλήρως στην τρέχουσα έκδοση του StarPUPy, και η υποστήριξη για αυτές τις λειτουργίες αναμένεται να βελτιωθεί σε μελλοντικές εκδόσεις.

4.5 Παράδειγμα Χρήσης του StarPUPy

Στην παρακάτω ενότητα παρουσιάζεται ένα απλό παράδειγμα χρήσης του StarPUPy, το οποίο καλύπτει την υποβολή εργασιών και την γενική φιλοσοφία της διεπαφής με βάση ότι έχουμε αναφέρει μέχρι τώρα.

Στο παράδειγμα αυτό δημιουργούνται και υποβάλλονται 8 CPU-bound εργασίες (busy loop) μέσω του StarPUPy. Η εκτέλεση των εργασιών μπορεί να παρακολουθηθεί σε πραγματικό χρόνο με την χρήση του εργαλείου `htop` σε linux σύστημα, το οποίο εμφανίζει τα νήματα/διεργασίες και τη χρήση της CPU.

Θεωρούμε ότι το StarPUPy έχει εγκατασταθεί σωστά και είναι διαθέσιμο στο περιβάλλον Python. Οδηγίες γι' αυτό μπορείτε να βρείτε εδώ [12].

Αρχικά πρέπει να εισάγουμε το module `starpup` μαζί με τις απαραίτητες βιβλιοθήκες και να ορίσουμε μια συνάρτηση που θα εκτελεί την CPU-bound εργασία. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```
import time
import asyncio
import os
from starpu import starpu
```

```
def busy_work(seconds):
    # CPU-bound busy loop to generate measurable load
    t0 = time.time()
    x = 1
    while time.time() - t0 < seconds:
        # cheap integer ops to keep CPU busy
        x = ((x << 1) ^ (x >> 3)) & 0xFFFFFFFF
    return x
```

Για την δημιουργία και υποβολή των εργασιών (tasks), που το καθένα θα εκτελεί την παραπάνω συνάρτηση `busy_work()`, χρησιμοποιούμε μια ασύγχρονη συνάρτηση, ως την πούμε `main()`, η οποία πρώτα αρχικοποιεί το StarPUPy περιβάλλον καλώντας την συνάρτηση `starpu.init()`. Στην συνέχεια μέσα σε ένα βρόχο `for` δημιουργούνται και υποβάλλονται 8 εργασίες, οι οποίες εκτελούν την συνάρτηση `busy_work()` για 8 δευτερόλεπτα η κάθε μία. Αυτό γίνεται με την επανειλημμένη χρήση της συνάρτησης `starpu.task_submit()`. Τα αντικείμενα Future που επιστρέφονται από την υποβολή των εργασιών αποθηκεύονται σε μια λίστα `futures`. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```
async def main():
    try:
        starpu.init()
    except Exception as e:
        print(f"StarPU initialization error: {e}")
        return

    print(f"PID: {os.getpid()}, ntasks=8, sec=8.0")
    print("Start submitting tasks...")

    t0 = time.time()
    futures = []
    for i in range(8):
        fut = starpu.task_submit()(busy_work, 8.0)
        futures.append(fut)
```

Στην συνέχεια, αφού έχουν υποβληθεί όλες οι εργασίες, η `main()` συνάρτηση περιμένει την ολοκλήρωση τους και συλλέγει τα αποτελέσματα από τα αντικείμενα Future. Αυτό γίνεται με την χρήση ενός δεύτερου βρόχου `for`, όπου για κάθε αντικείμενο Future καλείται η εντολή `await fut`, η οποία μπλοκάρει την εκτέλεση μέχρι να ολοκληρωθεί η αντίστοιχη εργασία. Τα αποτελέσματα αποθηκεύονται σε μια λίστα `results`. Μετά την ολοκλήρωση όλων των εργασιών, εμφανίζονται στην οθόνη ο χρόνος εκτέλεσης και τερματίζεται το περιβάλλον StarPUPy με την κλήση της συνάρτησης `starpu.shutdown()`. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```
# Wait for completion and collect results (force materialization)
results = []
```

```

for fut in futures:
    res = await fut
    results.append(res)

t1 = time.time()
elapsed = t1 - t0

print("=" * 60)
print("Framework: StarPU Python (Monitor dummy workload)")
print(f"Submitted tasks: 8")
print(f"Work per task (s): 8.0")
print(f"Elapsed time: {elapsed:.4f} seconds")
print("=" * 60)

starpup.shutdown()

```

Τέλος για την εκτέλεση της `main()` συνάρτησης χρησιμοποιούμε την εντολή `asyncio.run(main())`, η οποία εκκινεί το ασύγχρονο περιβάλλον και περιμένει την ολοκλήρωση της `main()` συνάρτησης. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```

if __name__ == '__main__':
    try:
        asyncio.run(main())
    except Exception as e:
        print(f"Error: {e}")

```

4.5.1 Αποτελέσματα Εκτέλεσης

Αν τρέξουμε το παράδειγμα που μόλις περιγράφηκε, με την παρακάτω εντολή τουλάχιστον 8 διαθέσιμους πυρήνες CPU, θα παρατηρήσουμε ότι οι εργασίες εκτελούνται παράλληλα και ο συνολικός χρόνος εκτέλεσης θα είναι περίπου 8 δευτερόλεπτα, αντί για 64 δευτερόλεπτα που θα ήταν η περίπτωση αν οι εργασίες εκτελούνταν σειριακά.

Για να εκτελέσουμε το παράδειγμα, μπορούμε να χρησιμοποιήσουμε την εντολή:

```
python3 ex_monitor_starpupy.py
```

Η έξοδος θα εμφανίσει το PID της διεργασίας, τον αριθμό των εργασιών που υποβλήθηκαν, τον χρόνο εκτέλεσης κάθε εργασίας και τον συνολικό χρόνο εκτέλεσης. Παρακάτω φαίνεται ένα παράδειγμα εξόδου:

```

PID: 173923, ntasks=8, sec=8.0
Start submitting tasks...
=====
Framework: StarPU Python (Monitor dummy workload)
Submitted tasks: 8
Work per task (s): 8.0
Elapsed time: 8.2067 seconds
=====

```

4.5.2 Ολοκληρωμένος Κώδικας Παραδείγματος

Παρακάτω παρουσιάζεται ο ολοκληρωμένος κώδικας του παραδείγματος χρήσης του StarPUPy, ο οποίος περιλαμβάνει όλα τα προηγούμενα αποσπάσματα κώδικα:

```

"""
Monitor example for StarPU Python
- Submits multiple CPU-bound dummy tasks (busy loop) via StarPU
- Use `htop` while it runs to observe threads/processes and CPU usage
"""

import time
import asyncio
import os
from starpu import starpu

def busy_work(seconds):
    # CPU-bound busy loop to generate measurable load
    t0 = time.time()
    x = 1
    while time.time() - t0 < seconds:
        # cheap integer ops to keep CPU busy
        x = ((x << 1) ^ (x >> 3)) & 0xFFFFFFFF
    return x

async def main():
    try:
        starpu.init()
    except Exception as e:
        print(f"StarPU initialization error: {e}")
        return

    print(f"PID: {os.getpid()}, ntasks=8, sec=8.0")
    print("Start submitting tasks...")

    t0 = time.time()
    futures = []
    for i in range(8):
        fut = starpu.task_submit()(busy_work, 8.0)
        futures.append(fut)

    # Wait for completion and collect results (force materialization)
    results = []
    for fut in futures:
        res = await fut
        results.append(res)

    t1 = time.time()
    elapsed = t1 - t0

    print("=" * 60)
    print("Framework: StarPU Python (Monitor dummy workload)")
    print(f"Submitted tasks: 8")
    print(f"Work per task (s): 8.0")

```

```
print(f"Elapsed time: {elapsed:.4f} seconds")
print("=" * 60)

starpu.shutdown()

if __name__ == '__main__':
    try:
        asyncio.run(main())
    except Exception as e:
        print(f"Error: {e}")
```

Κεφάλαιο 5

Το Σύστημα torcipy

5.1 Τι είναι το torcipy

Πέρα από την ανάγκη για λύσεις για την πλήρη αξιοποίηση των ετερογενών πόρων ενός υπολογιστικού κόμβου σε εφαρμογές **HPC** (π.χ. StarPU), υπάρχει και η ανάγκη για λύσεις που μεγιστοποιούν την απόδοση σε μία συστάδα (cluster) υπολογιστικών κόμβων. Μια προσέγγιση για την επίτευξη αυτού του σκοπού είναι η χρήση του **MPI** (Message Passing Interface) για την επικοινωνία μεταξύ των κόμβων. Ωστόσο, η χρήση του **MPI** μπορεί να είναι περίπλοκη και απαιτεί σημαντική προσπάθεια για την ανάπτυξη και τη συντήρηση των εφαρμογών.

Ταυτόχρονα, η συνεχής και ταχεία εξέλιξη και προσθήκη νέων βιβλιοθηκών κάνουν την γλώσσα **Python** πιο αποτελεσματική για την ανάπτυξη προηγμένων αλγορίθμων και επιστημονικών εφαρμογών. Επιπλέον, προσφέρει διεπαφές υψηλού επιπέδου που διασυνδέουν τους χρήστες με ταχύτατες βιβλιοθήκες HPC γραμμένες σε γλώσσες συστήματος όπως η C, η C++ και η Fortran.

Μία λύση που προσπαθεί να συνδυάσει τα πλεονεκτήματα της **Python** με την απόδοση του **MPI** είναι το **torcipy**. Το torcipy δεν σχεδιάστηκε από το μηδέν ως μια απομονωμένη ιδέα, αλλά αποτελεί τη φυσική εξέλιξη και την αμιγή υλοποίηση σε Python (pure Python implementation) της ισχυρής βιβλιοθήκης χρόνου εκτέλεσης TORC (Task Orchestration and Routing Code), η οποία ήταν γραμμένη σε C/C++ [2].

Το **torcipy** είναι ένα σύστημα χρόνου εκτέλεσης ανοιχτού κώδικα, αγνωστικιστικού ως προς την πλατφόρμα (platform-agnostic), το οποίο υλοποιεί προσαρμοστική εξισορρόπηση φόρτου (adaptive load balancing) και ενορχηστρώνει τον χρονοπρογραμματισμό εργασιών τόσο σε συστήματα κοινόχρηστης όσο και κατανεμημένης μνήμης [2].

Το παρόν κεφάλαιο παρουσιάζει αναλυτικά το σύστημα **torcipy**, εστιάζοντας στις βασικές του αρχές, την αρχιτεκτονική του και τις δυνατότητες που προσφέρει για την επίτευξη των στόχων που θέσαμε ως το βασικό κίνητρο για την ανάπτυξή του.

5.2 Αρχιτεκτονική του torcpy

Για την επίτευξη των προαναφερθέντων στόχων, το torcpy υλοποιήθηκε ως μια λεπτή αλλά εξαιρετικά αποδοτική στιβάδα λογισμικού πάνω από την τεχνολογία του MPI (συγκεκριμένα αξιοποιώντας το πακέτο `mpi4py`) και την τεχνολογία του multithreading της Python (αξιοποιώντας τα πακέτα `threading` και `queue`).

Η θεμελιώδης αρχιτεκτονική διάταξη μιας εφαρμογής που βασίζεται στο torcpy εδράζεται στην ανάπτυξη πολλαπλών διεργασιών MPI, οι οποίες κατανέμονται στους διαθέσιμους υπολογιστικούς κόμβους του συστήματος. Στο εσωτερικό κάθε διεργασίας MPI κατοικεί ένα εξαιρετικά βελτιστοποιημένο υποσύστημα νημάτων (thread subsystem) που αποτελείται από τρεις βασικούς πυλώνες [2]:

1. **Νήματα Εργατών (Worker Threads):** Κάθε διεργασία περιλαμβάνει ένα ή περισσότερα worker threads. Ο αποκλειστικός ρόλος τους είναι να εξάγουν (extract) εργασίες από τις διαθέσιμες ουρές και να εκτελούν τον παραγόμενο κώδικα Python.
2. **Νήμα Εξυπηρετητή (Server Thread):** Αποτελεί την καρδιά του συστήματος επικοινωνίας και τη γέφυρα με τον εξωτερικό κόσμο (άλλες διεργασίες MPI). Κάθε διεργασία διαθέτει ένα εξειδικευμένο νήμα διακομιστή (dedicated server thread), το οποίο διαχειρίζεται αποκλειστικά τις απομακρυσμένες λειτουργίες (remote operations) διαχείρισης εργασιών και δεδομένων. Όλα τα μηνύματα που αφορούν επικοινωνία μεταξύ των κόμβων ανταλλάσσονται μέσω αυτού του νήματος με διαφανή προς τον χρήστη τρόπο.
3. **Ουρές Εργασιών (Task Queues):** Ένα σύνολο από ιεραρχικές ουρές (set of queues) διατηρείται στην τοπική μνήμη της διεργασίας. Οι εργασίες υποβάλλονται σε αυτές τις ουρές, οι οποίες κατηγοριοποιούνται και οργανώνονται σύμφωνα με το επίπεδο παραλληλισμού (level of parallelism) από το οποίο αρχικά δημιουργήθηκαν. Για παράδειγμα, το αρχικό πρόγραμμα δημιουργεί εργασίες στην ουρά "επίπεδου 1". Αν μια τέτοια εργασία γεννήσει η ίδια νέες εργασίες, αυτές θα τοποθετηθούν στην ουρά "επίπεδου 2", και ούτω καθεξής.

Στην περίπτωση όπου ο χρήστης εκτελεί την εφαρμογή ως αμιγώς πολυνηματικό (multithreaded) κώδικα σε ένα μόνο μηχάνημα —δηλαδή δημιουργώντας μόνο μία διεργασία MPI η οποία φιλοξενεί πολλαπλά νήματα εργατών— το torcpy προσαρμόζεται αυτόματα. Σε αυτό το σενάριο, όλες ανεξαιρέτως οι λειτουργίες υποβολής, εξαγωγής και μεταφοράς δεδομένων πραγματοποιούνται αποκλειστικά μέσω της ταχύτατης τοπικής διαμοιραζόμενης μνήμης (shared memory), απαλείφοντας εντελώς το κόστος των κλήσεων του δικτύου MPI [2].

5.2.1 Δομή Εργασιών και Διαχείριση Αποτελεσμάτων

Μια κρίσιμη απόφαση σχεδιασμού για την εξασφάλιση της ταχύτητας του συστήματος αφορά τη δομή των ίδιων των εργασιών. Προκειμένου να αποφευχθεί το μεγάλο υπολογιστικό κόστος (overhead) που συνεπάγεται η δημιουργία, η δέσμευση μνήμης και η σειριοποίηση (serialization) ογκωδών κλάσεων, οι εργασίες υλοποιούνται αποκλειστικά ως δομές λεξικών της Python (Python dictionaries). Τα λεξικά της Python είναι δομές υψηλής ταχύτητας στο επίπεδο της γλώσσας C (cpython backend), μειώνοντας κατακόρυφα τον χρόνο που απαιτείται για τη δημιουργία της εργασίας [2].

Όταν η εργασία ολοκληρώνεται, το αποτέλεσμα της συνάρτησης αποθηκεύεται με απόλυτη διαφάνεια πίσω στον περιγραφέα της εργασίας (task descriptor ή future). Το εντυπωσιακό είναι ότι το αποτέλεσμα επιστρέφεται και αποθηκεύεται απευθείας στη διεργασία MPI η οποία δημιούργησε (spawned) την εργασία αρχικά, λύνοντας το πρόβλημα της διαχείρισης της μνήμης για τον προγραμματιστή. Ακολουθώντας πιστά το πρότυπο PEP 3148, ο τελικός χρήστης μπορεί μετέπειτα να αποκτήσει πρόσβαση στο αποτέλεσμα χρησιμοποιώντας απλώς την κλήση της μεθόδου `task.result()`. Με πανομοιότυπο τρόπο, παρέχεται η δυνατότητα πρόσβασης στα αρχικά ορίσματα της εργασίας μέσω της κλήσης `task.input()` [2].

5.2.2 Ο Ρόλος και η Λειτουργία του Νήματος Εξυπηρετητή (Server Thread)

Το Server Thread αποτελεί τον ενορχηστρωτή του συστήματος και λειτουργεί συνεχώς στο παρασκήνιο. Η λειτουργία του είναι ασύγχρονη και είναι επιφορτισμένο με τρεις κύριες, ζωτικής σημασίας ευθύνες [2]:

1. **Λήψη και Ένταξη:** Αναλαμβάνει τη λήψη νέων, εισερχόμενων εργασιών που καταφθάνουν από άλλους κόμβους (μέσω του δικτύου MPI) και αναλαμβάνει την τοποθέτησή τους (insertion) στις κατάλληλες τοπικές ουρές της διεργασίας του.
2. **Συλλογή Αποτελεσμάτων:** Παραλαμβάνει τις εργασίες που έχουν πλέον ολοκληρωθεί, μαζί με τα αντίστοιχα αποτελέσματά τους, δρομολογώντας τα σωστά στον κώδικα που τα αναμένει.
3. **Εξυπηρέτηση Αιτημάτων Κλοπής (Task Stealing Requests):** Η πιο σύνθετη λειτουργία του είναι ο χειρισμός των αιτημάτων κλοπής που καταφθάνουν από άλλες, αδρανείς διεργασίες MPI που αναζητούν φόρτο εργασίας.

Για να διασφαλιστεί ότι η επικοινωνία στο δίκτυο δεν θα μπλοκάρει την εκτέλεση του υπόλοιπου κώδικα Python (και κυρίως για να αποφευχθεί ο εγκλωβισμός του GIL

της Python από το νήμα του διακομιστή), ο σχεδιασμός επιβάλλει ρητά τη χρήση μη-μπλοκαρισμένων (non-blocking) κλήσεων του MPI. Κατά την ανάπτυξη και τη βελτιστοποίηση του torcpry, η ομάδα διαπίστωσε μέσα από τη χρήση εργαλείων αξιολόγησης από το πακέτο `mpi4py.futures` (όπως οι ρουτίνες `run_julia.py` και `perf_primes.py`) ότι οι μπλοκαρισμένες κλήσεις `MPI_Recv` δημιουργούσαν τεράστια συμφόρηση. Η λύση που υιοθετήθηκε στηρίχθηκε στην ενσωμάτωση non-blocking κλήσεων σε συνδυασμό με έναν μηχανισμό συνειδητής εκχώρησης του επεξεργαστή (`processor yielding`). Όταν το νήμα του διακομιστή δεν έχει μηνύματα να διαχειριστεί, "κοιμάται" εθελοντικά για ελάχιστο χρονικό διάστημα (π.χ. 0.01 δευτερόλεπτα), αποδεσμεύοντας πλήρως τον πυρήνα και επιτρέποντας στα νήματα των εργατών να προχωρήσουν απρόσκοπτα με την εκτέλεση του κώδικά τους. Παρά αυτή την τεχνική, παραμένει μια εγγενής πραγματικότητα: όσο αυξάνονται τα ενεργά threads που επιχειρούν να εκτελέσουν bytecode, η απόδοση θα επηρεάζεται αναπόφευκτα από το GIL [2].

5.3 Μηχανισμός Εξισορρόπησης Φόρτου

Η καρδιά της απόδοσης του torcpry σε ετερογενή και μη-προβλέψιμα υπολογιστικά περιβάλλοντα εδράζεται στον αλγόριθμο εξισορρόπησης φόρτου και στην πολιτική αναζήτησης και κλοπής εργασιών (**Work Stealing**). Ο κύκλος ζωής ενός αδρανούς νήματος-εργάτη (`idle worker thread`) ορίζεται από μια αυστηρή ακολουθία ενεργειών.

Όταν ένα νήμα-εργάτης (`Worker Thread`) ολοκληρώσει μια εργασία και βρεθεί χωρίς δουλειά, η πρώτη του κίνηση είναι να αναζητήσει νέα εργασία στον δικιά του, τοπική ουρά. Συγκεκριμένα, στοχεύει την υψηλότερη σε ιεραρχία τοπική ουρά η οποία δεν είναι κενή (`highest-level non-empty local queue`). Εάν δεν υπάρχει καμία διαθέσιμη εργασία και όλες οι τοπικές ουρές είναι άδειες, τότε ο εργάτης προσπαθεί να κλέψει κάποια εργασία από κάποια άλλη απομακρυσμένη ουρά εργασιών [2].

Σε αυτό το στάδιο, το αδρανές νήμα-εργάτης στέλνει ένα σύγχρονο αίτημα κλοπής (`synchronous stealing request`) προς το νήμα-εξυπηρετητή (`Server Thread`) της επόμενης στη σειρά διεργασίας MPI, σύμφωνα με το αναγνωριστικό κατάταξής της (`MPI rank`). Εάν η επόμενη διεργασία δεν έχει διαθέσιμη εργασία, ο εργάτης προχωρά στην επόμενη της επόμενης, δημιουργώντας ουσιαστικά μια κυκλική σάρωση (`ring topology`) όλου του MPI cluster. Η διαδικασία αυτή συνεχίζεται ακατάπαυστα μέχρι να εντοπιστεί και να επιστραφεί μια διαθέσιμη εργασία ή μέχρι να ολοκληρωθεί η σάρωση όλων ανεξαιρέτως των διαθέσιμων διεργασιών του συστήματος [2].

Από την άλλη πλευρά της εξίσωσης, όταν το νήμα-εξυπηρετητή μιας διεργασίας λάβει ένα τέτοιο αίτημα κλοπής από ένα αδρανές νήμα-εργάτη άλλου κόμβου, ακολουθεί μια εντελώς αντίστροφη λογική εύρεσης: ξεκινά την αναζήτηση για διαθέσιμη εργασία

από την κατώτερη σε ιεραρχία τοπική ουρά του (lowest-level local queue). Με άλλα λόγια, προσφέρει πρώτα τις εργασίες που βρίσκονται στο πιο εσωτερικό επίπεδο του παραλληλισμού (innermost level of parallelism) [2].

Η προεπιλεγμένη στρατηγική γέννησης εργασιών του *torcpy* (default task spawning policy) προβλέπει τη διανομή των εργασιών του "πρώτου επιπέδου" με κυκλικό τρόπο (cyclically) σε όλους τους διαθέσιμους εργάτες του συστήματος, διασφαλίζοντας την αρχική ευρεία διασπορά του φόρτου. Αντίθετα, τις εργασίες των "εσωτερικών επιπέδων" (inner-level tasks), οι οποίες προκύπτουν από τον ένθετο παραλληλισμό (nested-parallelism), τις υποβάλλει αυστηρά τοπικά στον ίδιο τον κόμβο που τις γέννησε [2].

Ο συνδυασμός αυτής της ασύμμετρης στρατηγικής δημιουργεί ένα εξαιρετικά ευφύες μοτίβο συμπεριφοράς (intelligent structural pattern). Ευνοεί ρητά την κλοπή "χονδροειδών" (coarse-grained) εργασιών μεταξύ των κόμβων (internode stealing), οι οποίες έχουν επαρκές μέγεθος υπολογισμού για να δικαιολογήσουν το χρονικό κόστος της μεταφοράς τους μέσω του δικτύου. Ταυτόχρονα, διατηρεί την εκτέλεση των "λεπτόκοκκων" (fine-grained), βαθύτερων επιπέδων παραλληλισμού στην τοπική μνήμη του εκάστοτε κόμβου, μειώνοντας δραστικά την άσκοπη επικοινωνία (communication overhead) μεταξύ των διεργασιών [2].

5.4 Προγραμματιστική Διεπαφή (API) και Υποστήριξη Κληρονομημένου Κώδικα (SPMD)

Το *torcpy* σχεδιάστηκε για να παρέχει μια Διεπαφή Προγραμματισμού Εφαρμογών (Application Programming Interface - API) που να είναι άμεσα αναγνωρίσιμη σε οποιονδήποτε προγραμματιστή έχει χρησιμοποιήσει τη βιβλιοθήκη `concurrent.futures` της Python, ακολουθώντας αυστηρά τις προδιαγραφές του PEP 3148. Πέραν αυτού, εισάγει εξειδικευμένες ρουτίνες για τη ρύθμιση του συστήματος, τον έλεγχο των καταστάσεων λειτουργίας, τον έλεγχο της κλοπής και την ανάκριση (querying) του περιβάλλοντος χρόνου εκτέλεσης [2].

5.4.1 Βασικές Ρουτίνες Διαχείρισης (Library Routines)

Το κεντρικό API περιλαμβάνει τις ακόλουθες θεμελιώδεις ρουτίνες [2]:

- **start(f)**: Αυτή είναι η λειτουργία αρχικοποίησης (initialization). Πρόκειται για μια συλλογική συνάρτηση (collective function) του MPI, που σημαίνει ότι πρέπει υποχρεωτικά να κληθεί από όλες τις διεργασίες εντός του μπλοκ `__main__` του προγράμματος. Ενεργοποιεί τη βιβλιοθήκη *torcpy* και, στη συνέχεια, εκκινεί τη συνάρτηση `f()` αποκλειστικά στη διεργασία με βαθμίδα μηδέν (rank 0). Η συνάρτηση

αυτή αναλαμβάνει το ρόλο της πρωτεύουσας εργασίας (primary application task). Όταν το δέντρο εκτέλεσης ολοκληρωθεί και η *f()* επιστρέψει, η ρουτίνα *start* φροντίζει για τον ασφαλή και καθαρό τερματισμό (shutdown) ολόκληρης της βιβλιοθήκης σε όλο το cluster.

- **submit(*f*, **args*, *qid=-1*, *callback=None*, *kwargs*):** Είναι ο πρωταρχικός τρόπος με τον οποίο ο χρήστης δημιουργεί νέες εργασίες. Υποβάλλει μία νέα εργασία (task), το οποίο αντιστοιχεί στην ασύγχρονη εκτέλεση της συνάρτησης *f()* με τα παρέχοντα ορίσματα *args*. Η παράμετρος *qid* καθορίζει ρητά το αναγνωριστικό του εργάτη στον οποίο πρέπει να υποβληθεί η εργασία. Εάν το *qid* παραμείνει στην προεπιλεγμένη τιμή του -1, η βιβλιοθήκη προβαίνει αυτόματα σε κυκλική διανομή (cyclic distribution) της εργασίας στο σύνολο των διαθέσιμων εργατών. Μια εξαιρετικά χρήσιμη παράμετρος είναι η *callback*. Όταν παρέχεται, η συνάρτηση *callback* (συνάρτηση επανάκλησης) ενεργοποιείται και εκτελείται αυτόματα στον υπολογιστικό κόμβο (rank) που είχε αρχικά γεννήσει (spawned) την εργασία, τη στιγμή που η εν λόγω εργασία ολοκληρώνεται πλήρως και τα αποτελέσματά της καταφθάνουν πίσω στον κόμβο αφετηρίας.
- **map(*f*, **seq*, *chunksize=1*):** Μια από τις πιο συχνά χρησιμοποιούμενες συναρτήσεις της Python, η *map*, παραλληλοποιείται εγγενώς. Αναλαμβάνει να εκτελέσει τη συνάρτηση *f()* σε όλα τα στοιχεία μιας τεράστιας λίστας/ακολουθίας (sequence) ορισμάτων. Επιστρέφει μια συλλογή με τα αποτελέσματα όλων των παραχθέντων εργασιών. Η παράμετρος *chunksize* (μέγεθος τμήματος) επιτρέπει την ομαδοποίηση πολλών στοιχείων της ακολουθίας σε μία μόνο εργασία, γεγονός το οποίο ελαχιστοποιεί κατακόρυφα το διοικητικό κόστος (overhead) σε περιπτώσεις όπου η εκτέλεση της συνάρτησης διαρκεί κλάσματα του δευτερολέπτου, προστατεύοντας το σύστημα από συμφόρηση.
- **wait(*tasks=None*):** Μέσω αυτής της κλήσης, η τρέχουσα εργασία αναστέλλει τη λειτουργία της και τίθεται σε αναμονή, περιμένοντας να ολοκληρωθούν όλες οι υπο-εργασίες (child tasks) που ενδεχομένως γέννησε προηγουμένως. Το πιο καινοτόμο στοιχείο της ρουτίνας *wait* στο *torcpy* είναι ότι η εργασία δεν μπλοκάρει άσκοπα τον επεξεργαστή. Το υποκείμενο νήμα εργάτη αποδεσμεύεται προσωρινά από τη συγκεκριμένη εργασία και είναι ελεύθερο να επιστρέψει στην ουρά για να εντοπίσει και να εκτελέσει άλλες διαθέσιμες εργασίες. Η στρατηγική αυτή είναι η απάντηση του *torcpy* στην πρόληψη των αδιεξόδων (deadlocks) που μαστιάζουν τα συμβατικά συστήματα.

5.4.2 Μοντέλο Εκτέλεσης SPMD

Ένα από το σημαντικά χαρακτηριστικό του API είναι η ρουτίνα `spmd(f, *args)`. Ο κλασικός παραλληλισμός εργασιών ακολουθεί το αυστηρό μοντέλο master-worker, δημιουργώντας έναν κλειστό κόσμο όπου οι εργάτες αγνοούν τη δομή του MPI. Το *torcpy* έρχεται να ανατρέψει αυτή την απομόνωση λειτουργώντας "ορθογώνια" προς το MPI [2].

Η κλήση `spmd` πυροδοτεί άμεσα και ταυτόχρονα την εκτέλεση της συνάρτησης `f()` σε όλες ανεξαιρέτως τις διεργασίες MPI του συστήματος. Αυτό μεταφράζεται σε μια δυναμική μετάβαση (dynamic switching) του περιβάλλοντος. Το σύστημα σταματά να λειτουργεί ως master-worker και μετασχηματίζεται ακαριαία στο κανονικό, θεμελιώδες μοντέλο **SPMD (Single Program Multiple Data)** που υποστηρίζει εγγενώς το MPI [2].

Αυτή η ικανότητα σημαίνει ότι οι χρήστες έχουν πλέον το ελεύθερο να ενσωματώσουν άμεσα παλαιό (legacy) κώδικα MPI, γραμμένο μέσω `mpi4py`, απευθείας μέσα στην εκτέλεση της εργασίας τους. Για παράδειγμα, δύνανται να εκτελέσουν βαριές, συλλογικές λειτουργίες του MPI (MPI collective operations) όπως οι Broadcast, Scatter, και Gather μεταξύ των κόμβων, για να προετοιμάσουν μεγάλα σύνολα δεδομένων πριν καν ξεκινήσει η αναλυτική φάση του συστήματος. Στο τέλος της εκτέλεσης της `spmd`, η εφαρμογή επιστρέφει ομαλά στο task-based μοντέλο [2].

Επιπρόσθετα, το σύστημα προσφέρει μια σειρά βοηθητικών συναρτήσεων (helper functions) για την παρακολούθηση της κατάστασης του υλικού: `torcpy.node_id()` (για να εντοπίσει το rank του κόμβου), `torcpy.worker_id()` (για το συγκεκριμένο νήμα), `torcpy.num_local_workers()`, `torcpy.gettime()` (για χρονομέτρηση ακριβείας), και φυσικά τις ρουτίνες `enable_stealing()` και `disable_stealing()` για το δυναμικό άνοιγμα ή κλείσιμο του μηχανισμού κλοπής [22].

5.5 Παράδειγμα Χρήσης του *torcpy*

Στην παρακάτω ενότητα παρουσιάζεται ένα απλό παράδειγμα χρήσης του *torcpy*, όπως και για το προηγούμενο παράδειγμα με το StarPU, με σκοπό να γίνει κατανοητή η βασική λογική λειτουργίας και χρήσης του συστήματος.

Στο παράδειγμα αυτό, δημιουργούνται 8 εργασίες που εκτελούν μια συνάρτηση `busy_work()` η οποία περιέχει έναν βρόχο που καταναλώνει CPU για 8 δευτερόλεπτα. Οι εργασίες αυτές υποβάλλονται στο σύστημα *torcpy* και εκτελούνται παράλληλα σε διαφορετικές διεργασίες και νήματα-εργάτες. Η εκτέλεση των εργασιών μπορεί να παρακολουθηθεί σε πραγματικό χρόνο με την χρήση του εργαλείου `htop` σε linux σύστημα, το οποίο εμφανίζει τα νήματα/διεργασίες και τη χρήση της CPU.. Μετά την ολοκλήρωση των εργασιών, εμφανίζεται ο συνολικός χρόνος εκτέλεσης.

Θεωρούμε ότι το σύστημα *torcpy* έχει εγκατασταθεί και είναι διαθέσιμο στο πε-

ριβάλλον Python. Οδηγίες για την εγκατάσταση του torcpy μπορούν να βρεθούν εδώ [22].

Αρχικά πρέπει να εισάγουμε το module `torcpy` μαζί με τις απαραίτητες βιβλιοθήκες και να ορίσουμε μια συνάρτηση που θα εκτελεί την CPU-bound εργασία. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```
import time
import os
import torcpy as torc

def busy_work(seconds):
    # CPU-bound busy loop to generate measurable load
    t0 = time.time()
    x = 1
    while time.time() - t0 < seconds:
        # cheap integer ops to keep CPU busy
        x = ((x << 1) ^ (x >> 3)) & 0xFFFFFFFF
    return x
```

Για την δημιουργία και υποβολή των εργασιών (tasks), που το καθένα θα εκτελεί την παραπάνω συνάρτηση `busy_work()`, χρησιμοποιούμε μια συνάρτηση, ας την πούμε `main()`. Μέσα σε αυτήν, δημιουργούμε μια λίστα `tasks` για να αποθηκεύσουμε τα αντικείμενα των εργασιών που υποβάλλουμε. Στη συνέχεια, χρησιμοποιούμε έναν βρόχο `for` για να υποβάλουμε 8 εργασίες, καθεμία από τις οποίες καλεί τη συνάρτηση `busy_work()` με όρισμα 8.0 δευτερόλεπτα. Μετά την υποβολή των εργασιών, καλούμε τη συνάρτηση `torc.wait()` για να περιμένουμε την ολοκλήρωση όλων των εργασιών. Τέλος, συλλέγουμε τα αποτελέσματα των εργασιών και εμφανίζουμε τον συνολικό χρόνο εκτέλεσης. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```
def main():
    print(f"PID: {os.getpid()}, ntasks=8, sec=8.0")
    print("Start submitting tasks...")

    t0 = time.time()
    tasks = []
    for i in range(8):
        tasks.append(torc.submit(busy_work, 8.0))

    # Wait for completion
    torc.wait()
    t1 = time.time()

    # Collect results (force materialization)
    for i, t in enumerate(tasks):
        try:
            _ = t.result()
        except Exception as e:
            print(f"task {i} error: {e}")
```

```

elapsed = t1 - t0
print("=" * 60)
print("Framework: torcpy (Monitor dummy workload)")
print(f"Submitted tasks: 8")
print(f"Work per task (s): 8.0")
print(f"Elapsed time: {elapsed:.4f} seconds")
print("=" * 60)

```

Τέλος, για την εκτέλεση της `main()` συνάρτησης χρησιμοποιούμε την εντολή `torc.start(main)`, η οποία εκκινεί το περιβάλλον του torcpy και περιμένει την ολοκλήρωση της `main()` συνάρτησης. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```

if __name__ == '__main__':
    torc.start(main)

```

5.5.1 Αποτελέσματα Εκτέλεσης

Αν τρέξουμε το παράδειγμα που μόλις περιγράφηκε, με την παρακάτω εντολή τουλάχιστον 8 διαθέσιμους πυρήνες CPU, θα παρατηρήσουμε ότι οι εργασίες εκτελούνται παράλληλα και ο συνολικός χρόνος εκτέλεσης θα είναι περίπου 8 δευτερόλεπτα, αντί για 64 δευτερόλεπτα που θα ήταν η περίπτωση αν οι εργασίες εκτελούνταν σειριακά.

Για να εκτελέσουμε το παράδειγμα, μπορούμε να χρησιμοποιήσουμε την εντολή:

```

mpirun -n 8 python3 ex_monitor_torcpy.py

```

Η έξοδος θα εμφανίσει το PID της διεργασίας, τον αριθμό των εργασιών που υποβλήθηκαν, τον χρόνο εκτέλεσης κάθε εργασίας και τον συνολικό χρόνο εκτέλεσης. Παρακάτω φαίνεται ένα παράδειγμα εξόδου:

```

PID: 173652, ntasks=8, sec=8.0
Start submitting tasks...
=====
Framework: torcpy (Monitor dummy workload)
Submitted tasks: 8
Work per task (s): 8.0
Elapsed time: 8.0513 seconds
=====

```

5.5.2 Ολοκληρωμένος Κώδικας Παραδείγματος

Παρακάτω παρουσιάζεται ο πλήρης κώδικας του παραδείγματος που περιγράφηκε παραπάνω, ο οποίος περιλαμβάνει όλα τα προηγούμενα αποσπάσματα κώδικα:

```
"""
Monitor example for torcpy
- Submits multiple CPU-bound dummy tasks (busy loop) to torcpy
- Use `htop` while it runs to observe processes/threads and CPU usage
"""

import time
import os
import torcpy as torc

def busy_work(seconds):
    # CPU-bound busy loop to generate measurable load
    t0 = time.time()
    x = 1
    while time.time() - t0 < seconds:
        # cheap integer ops to keep CPU busy
        x = ((x << 1) ^ (x >> 3)) & 0xFFFFFFFF
    return x

def main():
    print(f"PID: {os.getpid()}, ntasks=8, sec=8.0")
    print("Start submitting tasks...")

    t0 = time.time()
    tasks = []
    for i in range(8):
        tasks.append(torc.submit(busy_work, 8.0))

    # Wait for completion
    torc.wait()
    t1 = time.time()

    # Collect results (force materialization)
    for i, t in enumerate(tasks):
        try:
            _ = t.result()
        except Exception as e:
            print(f"task {i} error: {e}")

    elapsed = t1 - t0
    print("=" * 60)
    print("Framework: torcpy (Monitor dummy workload)")
    print(f"Submitted tasks: 8")
    print(f"Work per task (s): 8.0")
    print(f"Elapsed time: {elapsed:.4f} seconds")
    print("=" * 60)

if __name__ == '__main__':
    torc.start(main)
```

Κεφάλαιο 6

Εφαρμογές και Μεθοδολογία

6.1 Πειραματικές Εφαρμογές

Η αξιολόγηση προηγμένων συστημάτων παράλληλου προγραμματισμού, τα οποία βασίζονται σε μοντέλα εργασιών (task-based parallel programming) όπως τα **StarPU** και **torcpy**, προϋποθέτει τη χρήση κατάλληλων υπολογιστικών σεναρίων ελέγχου (benchmarks). Τα σενάρια αυτά δεν πρέπει να είναι απλοί, ακαδημαϊκοί μικρο-κώδικες, αλλά πραγματικές, σύνθετες επιστημονικές εφαρμογές που εγείρουν διαφορετικές προκλήσεις σε επίπεδο διαχείρισης μνήμης, υπολογιστικού φόρτου και συγχρονισμού.

Για την σύγκριση των δύο προγραμματιστικών μοντέλων, χρησιμοποιήθηκαν τρεις ξεχωριστές πειραματικές εφαρμογές. Η πρώτη εφαρμογή είναι η Στοχαστική Βελτιστοποίηση με τη μέθοδο CMA-ES (Covariance Matrix Adaptation Evolution Strategy). Η δεύτερη εφαρμογή είναι Επεξεργασία Εικόνας και Ανάλυση Υφής με Συστοιχίες Φίλτρων Gabor. Τέλος, η τρίτη εφαρμογή είναι η Αποτίμηση Χρηματοοικονομικών Παραγώγων με τη μέθοδο Monte Carlo.

Στην συνέχεια, παρουσιάζεται συνοπτικά το θεωρητικό υπόβαθρο των εφαρμογών αυτών, καθώς και οι προκλήσεις που εγείρουν σε επίπεδο διαχείρισης μνήμης, υπολογιστικού φόρτου και συγχρονισμού. Έτσι ώστε να καταστεί σαφές γιατί η χρήση αυτών των εφαρμογών είναι κατάλληλη για την αξιολόγηση των μοντέλων παράλληλου προγραμματισμού που εξετάζονται.

6.1.1 Στοχαστική Βελτιστοποίηση CMA-ES με Επίλυση Εξίσωσης Poisson (app00)

Αλγοριθμικό Πλαίσιο CMA-ES

Ο αλγόριθμος CMA-ES (Covariance Matrix Adaptation Evolution Strategy) ανήκει στην κατηγορία των εξελικτικών στρατηγικών (Evolution Strategies) και αποτελεί μέθοδο βελτιστοποίησης χωρίς παραγώγους, κατάλληλη για μη-κυρτά, μη-λεία και θορυβώδη τοπία βελτιστοποίησης όπου οι κλασικές μέθοδοι gradient-based αδυνατούν. Σε κάθε γενιά g παράγεται πληθυσμός λ υποψήφιων λύσεων μέσω δειγματοληψίας από πολυμεταβλητή

κανονική κατανομή [23, 24, 25, 26]:

$$\mathbf{x}_i^{(g+1)} = \mathbf{m}^{(g)} + \sigma^{(g)} \mathbf{y}_i, \quad \mathbf{y}_i \sim \mathcal{N}(\mathbf{0}, C^{(g)}), \quad i = 1, \dots, \lambda \quad (6.1)$$

όπου $\mathbf{m}^{(g)} \in \mathbb{R}^n$ είναι η τρέχουσα βέλτιστη εκτίμηση, $\sigma^{(g)} \in \mathbb{R}_+$ το μέγεθος βήματος (step-size) και $C^{(g)} \in \mathbb{R}^{n \times n}$ ο πίνακας συνδιακύμανσης [23, 26].

Ο πίνακας C προσεγγίζει βαθμιαία τον αντίστροφο Εσσιανό της αντικειμενικής συνάρτησης, συνδυάζοντας δύο μηχανισμούς ανανέωσης. Το **Rank- μ update** αξιοποιεί πληροφoρία χωρικής κατανομής από τον επιλεγμένο πληθυσμό της τρέχουσας γενιάς, ενώ το **Rank-1 update** συσσωρεύει ιστορική κατευθυντήρια πληροφoρία μέσω ενός διανύσματος «μονοπατιού εξέλιξης» (evolutionpath) [27, 23].

Η τελική εξίσωση ανανέωσης του πίνακα C είναι [23]:

$$C^{(g+1)} = (1 - c_1 - c_\mu) C^{(g)} + c_1 \mathbf{p}_c^{(g+1)} (\mathbf{p}_c^{(g+1)})^\top + c_\mu \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda}^{(g+1)} (\mathbf{y}_{i:\lambda}^{(g+1)})^\top \quad (6.2)$$

όπου c_1 και c_μ είναι οι αντίστοιχοι συντελεστές μάθησης. Το μέγεθος βήματος σ ελέγχεται ξεχωριστά μέσω της μεθόδου Cumulative Step-Size Adaptation (CSA). Η CMA-ES λειτουργεί αυστηρά βάσει ταξινόμησης (rank-based), προσδίδοντάς της ανοχή σε μονότονους μετασχηματισμούς της αντικειμενικής συνάρτησης [23, 26].

Αντικειμενική Συνάρτηση — Εξίσωση Poisson

Κάθε άτομο (διάνυσμα παραμέτρων θ) αξιολογείται επιλύοντας το αντίστροφο πρόβλημα της ελλειπτικής εξίσωσης Poisson [28]:

$$-\nabla^2 u(x, y) = f(x, y) \quad (6.3)$$

Το δεξί μέλος f κατασκευάζεται ως σταθμισμένο άθροισμα γκαουσιανών βάσεων $f(x, y) = \sum_k \theta_k \exp(-\|\mathbf{p} - \mathbf{s}_k\|^2 / 2\sigma_s^2)$, με κέντρα $\mathbf{s}_k$ σε προκαθορισμένες πηγές και συντελεστές θ . Η επίλυση γίνεται με επαναληπτική μέθοδο Jacobi σε κανονικό πλέγμα (128×128 κόμβοι). Η τιμή της αντικειμενικής συνάρτησης ορίζεται ως το MSE (Mean Squared Error) μεταξύ των τιμών της υπολογισμένης λύσης u σε $m = 40$ σημεία-αισθητήρες και των αντίστοιχων τιμών αναφοράς, με πρόσθετο όρο κανονικοποίησης L2 [28].

Παράλληλη Δομή και Χαρακτηρισμός

Ο παραλληλισμός στην εφαρμογή αυτή εισάγεται στο επίπεδο της **αξιολόγησης του πληθυσμού κάθε γενιάς**. Σε κάθε επανάληψη του αλγορίθμου CMA-ES, παράγεται ένας

πληθυσμός λ υποψήφιων λύσεων (ατόμων). Για κάθε άτομο i πρέπει να εκτελεστεί η ακόλουθη ακολουθία:

1. Κατασκευή του δεξιού μέλους f της εξίσωσης Poisson βάσει του διανύσματος παραμέτρων θ_i .
2. Αριθμητική επίλυση της εξίσωσης $-\nabla^2 u = f$ με επαναληπτική μέθοδο Jacobi στο πλέγμα 128×128 .
3. Δειγματοληψία της λύσης u στα $m = 40$ σημεία-αισθητήρες.
4. Υπολογισμός της τιμής καταλληλότητας (fitness) ως MSE μεταξύ της δειγματοληφθείσας και της λύσης αναφοράς.

Κάθε ένας από τους λ παραπάνω υπολογισμούς αποτελεί **μία ανεξάρτητη εργασία (task)**, καθώς δεν υπάρχει καμία εξάρτηση δεδομένων μεταξύ των ατόμων της ίδιας γενιάς [28].

Στην υλοποίηση με **StarPU**, κάθε εργασία υποβάλλεται ασύγχρονα μέσω `starpu.task_submit()(eval_candidate,  $\theta_i$ , pde, target_y)`, και ο χρονοδρομολογητής αναθέτει κάθε εργασία δυναμικά στους διαθέσιμους επεξεργαστές. Στην υλοποίηση με **torcpy**, οι λ εργασίες υποβάλλονται ομαδικά μέσω `torc.map(eval_candidate, payloads)`, όπου `payloads` είναι λίστα πλειάδων $(\theta_i, pde, target_y)$ [28].

Η παραλληλοποίηση περιορίζεται αυστηρά **εντός κάθε γενιάς**: μόλις ολοκληρωθούν όλες οι λ εργασίες, εκτελείται το σημείο φράγματος (barrier) κατά το οποίο ο αλγόριθμος CMA-ES ανανεώνει τον πίνακα συνδιακύμανσης C και παράγει τον πληθυσμό της επόμενης γενιάς. Η γενιά $g + 1$ δεν μπορεί να ξεκινήσει πριν αξιολογηθούν όλα τα άτομα της γενιάς g . Ο συνολικός αριθμός εργασιών ανά εκτέλεση είναι $\lambda \times n_{\text{gen}}$, όπου $\lambda = 16$ το μέγεθος πληθυσμού και $n_{\text{gen}} = 40$ ο μέγιστος αριθμός γενεών, δηλαδή έως 640 εργασίες συνολικά, ομαδοποιημένες σε 40 παράλληλες ομάδες των 16 [28].

6.1.2 Ανάλυση Υφής Εικόνων με Συστοιχίες Φίλτρων Gabor (app01)

Μαθηματική Διατύπωση Φίλτρου Gabor

Τα φίλτρα Gabor είναι ζωνοπερατά φίλτρα (band-pass filters) τα οποία συνδυάζουν μια γκαουσιανή περιβάλλουσα με ένα ημιτονοειδές επίπεδο κύμα, και έχειδειχθεί ότι προσομοιώνουν την απόκριση των απλών νευρικών κυττάρων του οπτικού φλοιού. Είναι ευρύτατα χρησιμοποιούμενα στην αναγνώριση υφής (texture analysis), την ανίχνευση ακμών (edge detection) και τη διαίρεση εικόνων (image segmentation) [29, 30].

Για εφαρμογή υπό γωνία θ , το σύστημα συντεταγμένων περιστρέφεται ως [29, 31]:

$$x' = x \cos \theta + y \sin \theta, \quad y' = -x \sin \theta + y \cos \theta \quad (6.4)$$

και το πραγματικό μέρος του φίλτρου (κατάλληλο για ανάλυση υφής) εκφράζεται ως [31]:

$$g(x, y) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(\frac{2\pi x'}{\lambda} + \psi\right) \quad (6.5)$$

Οι πέντε παράμετροι (μήκος κύματος λ , προσανατολισμός θ , τυπική απόκλιση σ , λόγος πλευρών γ , μετατόπιση φάσης ψ) καθορίζουν πλήρως τη συχνότητα και κατεύθυνση χαρακτηριστικών που ανιχνεύει το φίλτρο. Κατά τη διακριτοποίηση εφαρμόζεται κανονικοποίηση μέσης τιμής (zero-mean normalization) $k = g(x, y) - \text{mean}(g(x, y))$ για την εξάλειψη DC bias [28, 29, 30].

Συστοιχία Φίλτρων και FFT Convolution

Κατασκευάζεται συστοιχία $n_{\text{scales}} \times n_{\text{orients}}$ πυρήνων Gabor (ενδεικτικά: $5 \times 8 = 40$ πυρήνες), με εκθετική επιλογή κλιμάκων $\lambda = 2^{s+1}$ και ομοιόμορφο διαμερισμό προσανατολισμών $\theta_o = o\pi/n_{\text{orients}}$ στο $[0, \pi)$. Η τυπική απόκλιση κλιμακώνεται γραμμικά ως $\sigma = 0.56\lambda$ για σταθερό εύρος ζώνης μεταξύ κλιμάκων [30, 28].

Η συνέλιξη εφαρμόζεται μέσω του Θεωρήματος Συνέλιξης στο πεδίο Fourier: με κατάλληλο zero-padding στις διαστάσεις $H = H_i + H_k - 1$, $W = W_i + W_k - 1$, η χωρική συνέλιξη $\mathcal{O}(H_i W_i H_k W_k)$ μετατρέπεται σε κατά σημείο πολλαπλασιασμό φασμάτων $\mathcal{O}(N \log N)$ [28, 29]:

$$R_{\text{full}} = \text{Real}(\mathcal{F}^{-1}(F_I \odot F_K)) \quad (6.6)$$

και το αποτέλεσμα περικόπτεται στις αρχικές διαστάσεις της εικόνας.

Η ενέργεια Gabor υπολογίζεται ως [28]:

$$E(x, y) = \sqrt{\sum_{s=1}^{n_{\text{scales}}} \sum_{o=1}^{n_{\text{orients}}} R_{s,o}(x, y)^2 + 10^{-12}} \quad (6.7)$$

Ακολουθεί λογαριθμική μετατροπή $\ln(1 + E)$ για συμπίεση δυναμικού εύρους, δημιουργία ιστογράμματος n_{bins} κάδων και εξαγωγή βαθμωτού χαρακτηριστικού ως σταθμισμένο άθροισμα [28]:

$$\text{Feature Value} = \sum_{i=1}^{n_{\text{bins}}} h[i-1] \times i \quad (6.8)$$

Παράλληλη Δομή και Χαρακτηρισμός

Ο παραλληλισμός στην εφαρμογή αυτή εισάγεται στο επίπεδο της **επεξεργασίας κάθε εικόνας ανεξάρτητα**. Για κάθε αρχείο εικόνας f_j της συλλογής εκτελείται η ακόλουθη αγωγή επεξεργασίας:

1. Φόρτωση και μετατροπή σε κλίμακα του γκρι, αλλαγή ανάλυσης σε 256×256 pixels.
2. Εφαρμογή της συστοιχίας $n_{\text{scales}} \times n_{\text{orients}} = 5 \times 8 = 40$ πυρήνων Gabor μέσω FFT convolution.
3. Υπολογισμός ενέργειας Gabor $E(x, y)$ και λογαριθμική μετατροπή.
4. Κατασκευή ιστογράμματος $n_{\text{bins}} = 64$ κάδων και εξαγωγή βαθμωτού χαρακτηριστικού.

Κάθε μία από τις παραπάνω αγωγές αποτελεί **μία ανεξάρτητη εργασία (task)** αντιστοιχισμένη σε μία εικόνα. Δεν υπάρχει καμία εξάρτηση δεδομένων μεταξύ εικόνων, επομένως όλες οι εργασίες μπορούν να εκτελεστούν ταυτόχρονα χωρίς κανένα σημείο φράγματος [28].

Στην υλοποίηση με **StarPU**, κάθε εργασία υποβάλλεται ασύγχρονα μέσω `starpup.task_submit()(process_image, f_j, cfg)`, όπου `cfg` περιέχει τις παραμέτρους της συστοιχίας φίλτρων. Στην υλοποίηση με **torcipy**, οι εργασίες υποβάλλονται ομαδικά μέσω `torc.map(process_image, payloads)`, όπου `payloads` είναι λίστα πλειάδων (f_j, cfg) για κάθε εικόνα [28].

Ο συνολικός αριθμός εργασιών ισούται με τον αριθμό εικόνων της συλλογής. Κάθε εργασία είναι υπολογιστικά ακριβή (40 FFT convolutions ανά εικόνα), γεγονός που εξασφαλίζει επαρκή παράλληλο φόρτο εργασίας ανά επεξεργαστή και καθιστά τον χρόνο επικοινωνίας αμελητέο σε σχέση με τον χρόνο υπολογισμού [28].

6.1.3 Αποτίμηση Χρηματοοικονομικών Παραγώγων με Monte Carlo (app02)

Στοχαστικό Μοντέλο — Γεωμετρική Κίνηση Brown

Η εφαρμογή μοντελοποιεί την αποτίμηση ενός Δικαιώματος Προαίρεσης Καλαθιού (Basket Option) με ρήτρα Φραγμού τύπου Down-and-Out. Τα σύνθετα αυτά παράγωγα δεν διαθέτουν κλειστή αναλυτική λύση, σε αντίθεση με τα απλά Vanilla European Options που περιγράφονται από τον τύπο Black-Scholes-Merton, και αποτιμώνται αποκλειστικά μέσω στοχαστικής προσομοίωσης [32, 33].

Η εξέλιξη κάθε περιουσιακού στοιχείου S_t μοντελοποιείται μέσω Στοχαστικής Διαφορικής Εξίσωσης (SDE) Γεωμετρικής Κίνησης Brown (GBM) [33]:

$$dS_t = r S_t dt + \sigma S_t dW_t \quad (6.9)$$

Εφαρμόζοντας το Λήμμα Ito, η αναλυτική λύση για διακριτό χρονικό βήμα Δt είναι [33]:

$$S_{t+\Delta t} = S_t \cdot \exp\left(\left(r - \frac{1}{2}\sigma^2\right) \Delta t + \sigma\sqrt{\Delta t} Z\right), \quad Z \sim \mathcal{N}(0, 1) \quad (6.10)$$

όπου r το επιτόκιο χωρίς κίνδυνο, σ η μεταβλητότητα, $\mu = (r - \frac{1}{2}\sigma^2)\Delta t$ ο «drift» με διόρθωση Ito και $sd = \sigma\sqrt{\Delta t}$ ο «diffusion» [28].

Συσχέτιση και Παραγοντοποίηση Cholesky

Τα n_{assets} περιουσιακά στοιχεία του καλαθιού δεν είναι ανεξάρτητα: οι στοχαστικές διαδικασίες τους είναι συσχετισμένες μέσω πίνακα συσχέτισης $C \in \mathbb{R}^{n \times n}$, ο οποίος πρέπει να είναι Συμμετρικός και Θετικά Ορισμένος (SPD). Ο πίνακας C κατασκευάζεται τυχαία με ανάμιξη τυχαίας SPD δομής και ταυτοτικού πίνακα μέσω συντελεστή $\alpha = 0.15$ [32, 33, 28]:

$$C_{\text{blended}} = (1 - \alpha) I + \alpha C \quad (6.11)$$

Ακολουθώς παραγοντοποιείται Cholesky ως $C = LL^\top$, όπου L ο κάτω τριγωνικός «ρίζα» πίνακας. Για κάθε βήμα προσομοίωσης, ανεξάρτητα τυχαία διανύσματα Z μετασχηματίζονται σε συσχετισμένες διαταραχές $dW = L \cdot Z$ [32, 33, 28].

Μη-Γραμμικές Διαταραχές («Heavy» Monte Carlo)

Αμέσως μετά κάθε βήμα GBM εφαρμόζεται μη-γραμμική διαταραχή [28]:

$$S_t \leftarrow S_t \times (1.0 + 0.01 \sin(S_t) + 0.01 \ln(1 + |S_t|)) \quad (6.12)$$

Αυτή η διαταραχή καταργεί τη δυνατότητα macro-stepping, υποχρεώνοντας ακριβή διακριτοποίηση σε όλα τα $n_{\text{steps}} = 128$ χρονικά βήματα. Η έντονη χρήση υπερβατικών συναρτήσεων ($\exp$, $\sin$, $\ln$) αυξάνει δραστικά τις πράξεις κινητής υποδιαστολής (FLOPs), μετατρέποντας την εφαρμογή σε **CPU-bound workload** [28].

Συνθήκες Αποπληρωμής

Η αξία του καλαθιού στη λήξη ορίζεται ως ο αριθμητικός μέσος [28]:

$$\text{Basket}_T = \frac{1}{n_{\text{assets}}} \sum_{i=1}^{n_{\text{assets}}} S_{i,T} \quad (6.13)$$

Η συνθήκη φραγμού Down-and-Out ελέγχεται σε κάθε χρονικό βήμα μέσω boolean διανύσματος: αν έστω ένα στοιχείο πέσει κάτω από $B = 0.60$, το μονοπάτι «βγαίνει εκτός» (knock-out) και η αποπληρωμή μηδενίζεται [34, 28]:

$$\text{alive} \leftarrow \text{alive} \wedge \bigwedge_{i=1}^{n_{\text{assets}}} (S_{i,t} > B) \quad (6.14)$$

Η τελική αποπληρωμή ακολουθεί λογική European Call και αναπροεξοφλείται στον χρόνο $t = 0$ [34]:

$$\text{PV} = \max(\text{Basket}_T - K, 0) \cdot e^{-rT} \quad (6.15)$$

Παράλληλη Δομή — Batching και Parallel Reduction

Ο παραλληλισμός στην εφαρμογή αυτή εισάγεται στο επίπεδο της **προσομοίωσης παρτίδων μονοπατιών** (path batching). Το συνολικό πλήθος $N = 3.2 \times 10^6$ μονοπατιών κατανέμεται σε $n_{\text{tasks}} = \lceil N/\text{batch} \rceil$ παρτίδες των $\text{batch} = 50000$ μονοπατιών η καθεμία, δηλαδή 64 εργασίες συνολικά [28].

Για κάθε παρτίδα k εκτελείται η ακόλουθη αγωγή:

1. Αρχικοποίηση $n_{\text{paths}} = 50000$ μονοπατιών με τιμή $S_0 = 1.0$ για όλα τα $n_{\text{assets}} = 16$ περιουσιακά στοιχεία.
2. Επαναληπτική εξέλιξη για $n_{\text{steps}} = 128$ χρονικά βήματα: παραγωγή συσχετισμένων τυχαίων κλονισμών $dW = L \cdot Z$, ενημέρωση τιμών βάσει GBM, εφαρμογή μη-γραμμικής διαταραχής και έλεγχος συνθήκης φραγμού.
3. Υπολογισμός αποπληρωμής καλαθιού και αναπροεξόφλησης.
4. Επιστροφή **μόνο τριών** συγκεντρωτικών μεγεθών: $(\sum \text{PV}, \sum \text{PV}^2, N_{\text{batch}})$.

Κάθε παρτίδα αποτελεί **μία ανεξάρτητη εργασία (task)**, καθώς κάθε μονοπάτι χρησιμοποιεί δικό του seed τυχαίων αριθμών και δεν υπάρχει καμία επικοινωνία μεταξύ παρτίδων κατά την εκτέλεση [28].

Στην υλοποίηση με **StarPU**, κάθε εργασία υποβάλλεται ασύγχρονα μέσω `starpu.task_submit(mc_batch, n_this, seed_k, cfg, lmat)`. Στην υλοποίηση με **torcpy**, οι 64 εργασίες υποβάλλονται ομαδικά μέσω `torc.map(mc_batch, payloads)`, όπου κάθε στοιχείο της λίστας `payloads` είναι η πλειάδα $(n_{\text{this}}, \text{seed}_k, \text{cfg}, \text{lmat})$ [28].

Μόλις ολοκληρωθούν όλες οι εργασίες, το κεντρικό πρόγραμμα εκτελεί **παράλληλη αναγωγή** (parallel reduction) αθροίζοντας τα τρία μεγέθη από κάθε παρτίδα για τον υπολογισμό της τελικής εκτιμώμενης τιμής και του τυπικού σφάλματος. Η επιστροφή μόνο τριών αριθμών ανά εργασία (αντί για πίνακα 50000 τιμών) ελαχιστοποιεί δραστικά το κόστος επικοινωνίας μέσω δικτύου MPI ή διαύλου μνήμης, διατηρώντας τους επεξεργαστές στο 100% της χρήσης τους (CPU saturation) [28].

6.2 Μεθοδολογία Αξιολόγησης και διεξαγωγής των Πειραμάτων

Για κάθε μία από τις τρεις εφαρμογές που συζητήσαμε στην προηγούμενη ενότητα, υλοποιήσαμε δύο εκδόσεις: μία με το μοντέλο **StarPU**Py και μία με το μοντέλο **torcpy**.

Η αξιολόγηση των δύο μοντέλων έγινε με βάση δύο κριτήρια: (α) την απόδοση (performance) και (β) την κλιμακωσιμότητα (scalability). Η απόδοση μετρήθηκε σε επίπεδο χρόνου εκτέλεσης, ενώ η κλιμακωσιμότητα μετρήθηκε σε επίπεδο επιτάχυνσης (speedup) και μέγιστο πλήθος νήματων/διεργασιών που παραμένει υψηλή η απόδοση (parallel efficiency).

6.2.1 Εξασφάλιση Ισοδυναμίας Εκδόσεων και Δυσκολίες Υλοποίησης

Οι εκδόσεις αυτές είναι πλήρως ισοδύναμες σε επίπεδο αλγοριθμικής λογικής και αριθμητικής ακρίβειας, ώστε να διασφαλίζεται ότι οι διαφορές στην απόδοση οφείλονται αποκλειστικά στα χαρακτηριστικά των μοντέλων παράλληλου προγραμματισμού. Επίσης, για να ενισχυθεί αυτή η ισοδυναμία, χρησιμοποιήθηκαν οι ίδιες μαθηματικές βιβλιοθήκες και για τις οποίες είχαμε απενεργοποιήσει, μέσω μεταβλητών περιβάλλοντος οποιαδήποτε βελτιστοποίηση είχαν ήδη εσωτερικά (π.χ. `OMP_NUM_THREADS=1` ή `OPENBLAS_NUM_THREADS=1` που απενεργοποιεί τα εσωτερικά threads των μαθηματικών βιβλιοθηκών) [28].

Μόνο για την εφαρμογή της CMA-ES (app00) χρησιμοποιήθηκε η βιβλιοθήκη **numba**¹, ή οποία είναι κοινή και στις δύο εκδόσεις, για την επιτάχυνση του βρόχου αξιολόγησης μέσω JIT compilation. Η χρήση της **numba** ήταν απαραίτητη για να καταστεί εφικτή η εκτέλεση της εφαρμογής σε λογικά χρονικά πλαίσια. Ο λόγος που υπήρχε αυτό το πρόβλημα είναι ότι η εφαρμογή της CMA-ES περιλαμβάνει έναν βρόχο που εκτελείται για πολλές γενιές, και σε κάθε γενιά αξιολογούνται λ άτομα, όπου κάθε αξιολόγηση απαιτεί την επίλυση της εξίσωσης Poisson. Χωρίς την επιτάχυνση της **numba**, ο χρόνος εκτέλε-

¹Η **numba** είναι ένας open-source μεταγλωττιστής Just-In-Time (JIT) που μετατρέπει τον κώδικα Python και NumPy σε γρήγορο κώδικα μηχανής.

σης θα ήταν απαγορευτικός για την ολοκλήρωση των πειραμάτων. Οι υπολογισμοί αυτοί είναι σχετικά μικροί σε κόστος σε σχέση με την δέσμευση και απελευθέρωση του GIL της Python, και επειδή οι υπολογισμοί είναι πάρα πολύ σε πλήθος, κατέληγε αυτή η πολύ συχνή δέσμευση και απελευθέρωση του GIL να είναι ο κυρίαρχος παράγοντας που καθιστούσε την εκτέλεση της εφαρμογής μη πρακτική (φαίνόμενο του GIL thrashing¹). Η χρήση της `numba` επέτρεψε την εκτέλεση των υπολογισμών σε κώδικα μηχανής, παρακάμπτοντας το GIL και επιτρέποντας την αποτελεσματική εκτέλεση των βρόχων αξιολόγησης χωρίς το φαινόμενο του GIL thrashing [28].

Οι υπόλοιπες δύο εφαρμογές (Image Processing και Monte Carlo) δεν είχαν τόσο αυστηρούς χρονικούς περιορισμούς, και έτσι δεν απαιτήθηκε η χρήση της `numba` για την επιτάχυνση των υπολογισμών.

6.2.2 Πειραματική Διαδικασία

Η εκτέλεση των πειραμάτων έγινε σε ένα σύστημα με δύο NUMA-nodes (2 sockets) Intel® Xeon® Gold 6530 Processor, καθένα με 32 φυσικούς πυρήνες (64 threads με Hyper-Threading), συνολικά 251 GB RAM, μέγιστη συχνότητα ρολογιού 4 GHz και λειτουργικό σύστημα Ubuntu 24.04.4 LTS (Linux).

Κάθε εφαρμογή εκτελέστηκε, για κάθε μοντέλο παράλληλου προγραμματισμού, με διαφορετικό πλήθος νημάτων/διεργασιών (1, 2, 4, 8, 16, 32, 64) και επαναλήφθηκε 5 φορές για στατιστική αξιοπιστία κρατώντας πάντα τον μικρότερο χρόνο εκτέλεσης από κάθε συνδυασμό.

Πιο συγκεκριμένα, οι εκδόσεις των εφαρμογών για το StarPUPy εκτελέστηκαν μόνο με threads, ενώ οι εκδόσεις για το torcpy εκτελέστηκαν με συνδυασμό με διεργασίες (processes) και threads, καθώς το torcpy είναι βασισμένο πάνω στο MPI. Πάντα, όμως το συνολικό πλήθος των νημάτων/διεργασιών που χρησιμοποιήθηκε ήταν το ίδιο για κάθε μοντέλο και δεν ξεπερνούσε τον αριθμό των διαθέσιμων πόρων (64 physical cores), ώστε να είναι συγκρίσιμες οι μετρήσεις.

Επιπλέον, έγιναν δύο σεντ πειραμάτων για κάθε εφαρμογή: ένα με NUMA-aware scheduling και ένα με Standard (NUMA-oblivious) scheduling. Σε κάθε περίπτωση, γίνεται χρήση των κατάλληλων σημαιών (flags) και μεταβλητών περιβάλλοντος καθενός μοντέλου πριν την εκτέλεση των εφαρμογών. Στον Πίνακα 6.1 παρουσιάζονται συνοπτικά οι παράμετροι της πειραματικής διαδικασίας [12, 16, 22].

¹Το **GIL thrashing** συμβαίνει όταν ένα πολυνηματικό (multithreaded) πρόγραμμα στην Python αναγκάζει το Global Interpreter Lock (GIL) να απελευθερώνεται και να δεσμεύεται συνεχώς με υπερβολική συχνότητα. Αντί να επιταχύνει τις εργασίες που επιβαρύνουν τον επεξεργαστή (CPU-bound tasks), αυτή η καθυστέρηση (overhead) μειώνει δραματικά την απόδοση.

Μοντέλο	Λειτουργία	Παράμετρος	Περιγραφή
torcpy	NUMA-aware	<code>mpirun -n N</code>	Εκτέλεση με N MPI διεργασίες.
		<code>--map-by socket</code>	Αντιστοίχιση MPI διεργασιών ανά NUMA κόμβο.
		<code>--bind-to core</code>	Δέσμευση κάθε διεργασίας σε φυσικό πυρήνα.
		<code>TORCPY_WORKERS</code>	Αριθμός νημάτων εργασίας ανά MPI διεργασία, ίσος με τους πυρήνες ανά NUMA κόμβο.
	Standard	<code>mpirun -n N</code>	Εκτέλεση με N MPI διεργασίες.
		<code>TORCPY_WORKERS</code>	Συνολικός αριθμός νημάτων εργασίας χωρίς δέσμευση σε NUMA κόμβο.
StarPUPy	NUMA-aware	<code>STARPU_NCPU</code>	Αριθμός CPU εργατών, ίσος με τους πυρήνες ανά NUMA κόμβο.
		<code>STARPU_USE_NUMA=1</code>	Ενεργοποίηση χαρτογράφησης μνήμης ανά NUMA κόμβο μέσω hwloc.
		<code>STARPU_SCHED=dmda</code>	Χρονοπρογραμματιστής με επίγνωση κόστους μεταφοράς δεδομένων.
		<code>STARPU_WORKERS_GET BIND=1</code>	Σεβασμός CPU masks του MPI launcher για αποφυγή υπερφόρτωσης πυρήνων.
		<code>STARPU_LIMIT_CPU_NUMA_MEM</code>	Όριο μνήμης ανά NUMA κόμβο σε MB (τιμή: 32000).
		<code>STARPU_NCUDA=0, STARPU_NOPENCL=0</code>	Απενεργοποίηση GPU επιταχυντών.
	Standard	<code>STARPU_NCPU</code>	Συνολικός αριθμός CPU εργατών χωρίς δέσμευση NUMA τοπολογίας.

		STARPU_NCUDA=0, STARPU_NOPENCL=0	Απενεργοποίηση GPU επι- ταχυντών.
--	--	-------------------------------------	--------------------------------------

Πίνακας 6.1: Μεταβλητές περιβάλλοντος για NUMA-aware και Standard εκτέλεση των StarPUPy και torcpy.

Η εκτέλεση των δύο αυτών σετ πειραμάτων επέτρεψε την αξιολόγηση της επίδρασης του NUMA-aware scheduling στην απόδοση και κλιμακωσιμότητα των δύο μοντέλων παράλληλου προγραμματισμού, καθώς και τη σύγκριση μεταξύ τους υπό διαφορετικές συνθήκες εκτέλεσης.

Εδώ πρέπει να γίνει μια επισήμανση για τον παραπάνω πίνακα στις παραμέτρους για NUMA-aware εκτέλεση του StarPUPy. Η παράμετρος `STARPU_WORKERS_GETBIND=1` αναφέρεται μόνο για πληρότητα καθώς για το StarPUPy η εκτέλεση των εφαρμογών δεν γίνεται μέσω MPI launcher οπότε η χρήση της δεν έχει κάποια επίδραση στην εκτέλεση. Παρόλα αυτά, η παράμετρος αυτή είναι χρήσιμη για την περίπτωση που το StarPUPy εκτελείται μέσω MPI launcher, καθώς σε αυτή την περίπτωση η παράμετρος αυτή επι-τρέπει στο StarPUPy να σεβαστεί τα CPU masks που ορίζει ο MPI launcher και να μην υπερφορτώσει τους πυρήνες του συστήματος.

Κεφάλαιο 7

Πειραματική Αξιολόγηση και Συμπεράσματα

7.1 Δοκιμές Δυνατοτήτων του StarPUPy και torcpy

Πριν προχωρήσουμε στην παρουσίαση των αποτελεσμάτων των πειραμάτων, και στα συμπεράσματα που προκύπτουν από αυτά, να πούμε ότι κάναμε και κάποιες δοκιμές για να δούμε κάποιες έξτρα δυνατότητες των δύο μοντέλων παράλληλου προγραμματισμού.

7.1.1 StarPUPy: MPI Server-Client

Δοκιμάσαμε και τις δυνατότητες του StarPUPy για εκτέλεση με MPI launcher, μέσω του Server-Client (ή Master-Slave σε παλαιότερες εκδόσεις) μοντέλου, για να είναι πιο κοντά η σύγκριση με το torcpy. Αλλά δυστυχώς η εκτέλεση αυτή των εφαρμογών δεν ήταν εφικτή λόγω προβλημάτων και έλλειψη πλήρους υποστήριξης των δυνατοτήτων MPI που παρέχει κανονικά το StarPU στην γλώσσα C. Το πρόβλημα ήταν κυρίως ότι δεν υποστηρίζει την χρήση native δομών δεδομένων της Python (π.χ. dictionaries) παρά μόνο NumPy arrays, και αυτό απαιτούσε να γίνει μετατροπή των δεδομένων σε NumPy arrays για να μπορέσει να γίνει η εκτέλεση μέσω MPI launcher. Έτσι, δοκιμάσαμε να μετατρέψουμε έστω την εφαρμογή Monte Carlo (app02) να χρησιμοποιεί NumPy arrays για να εκτελεστεί μέσω MPI launcher, αλλά η μετατροπή αυτή δεν ήταν αρκετή γιατί μετά προέκυψαν άλλα προβλήματα με το Initialization του περιβάλλοντος MPI (Server-Client) σε συνδυασμό με το StarPU. Έτσι, καταλήξαμε ότι η δυνατότητες MPI δεν έχουν μεταφερθεί ακόμα πλήρως στην έκδοση της StarPU για Python και το αφήσαμε εκεί. Ο κώδικας για αυτή την έκδοση της εφαρμογής Monte Carlo (app02) υπάρχει στο αποθετήριο του έργου, αλλά δεν είναι πλήρως λειτουργικός και δεν χρησιμοποιήθηκε για την αξιολόγηση των μοντέλων παράλληλου προγραμματισμού [28].

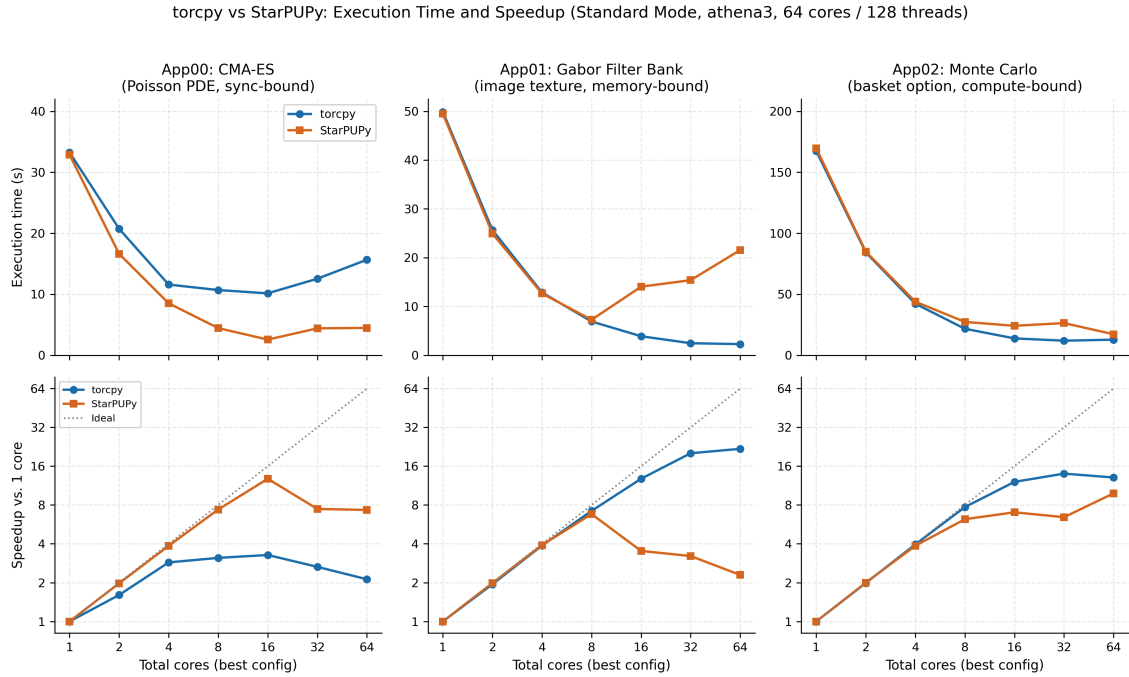
7.1.2 Εμφολευμένη Παραλληλοποίηση (Nested Parallelism)

Επιπλέον, δοκιμάσαμε την δυνατότητα εμφολευμένης παραλληλοποίησης (nested parallelism) εργασιών των δύο μοντέλων και πώς αυτά το αντιμετωπίζουν. Έτσι φτιάξαμε μια νέα εκδοχή της εφαρμογής Image Processing (app01) που εκτελείται σε δύο επίπεδα παραλληλοποίησης. Στο πρώτο επίπεδο, έχουμε δύο εργασίες που η κάθε μία παίρνει τις μισές εικόνες του συνόλου των εικόνων και ως δεύτερο επίπεδο παραλληλοποίησης, κάθε εργασία δημιουργεί τις υπο-εργασίες που εκτελούν την ανάλυση υφής με φίλτρα Gabor για τις εικόνες που τους ανατίθενται. Έτσι, έχουμε δύο επίπεδα παραλληλοποίησης, όπου στο πρώτο επίπεδο γίνεται διαμοιρασμός των εικόνων σε δύο εργασίες και στο δεύτερο επίπεδο γίνεται διαμοιρασμός των εικόνων που ανατίθενται σε κάθε εργασία σε υπο-εργασίες. Ο κώδικας για αυτή την εκδοχή της εφαρμογής Image Processing (app01) υπάρχει στο αποθετήριο του έργου, αλλά δεν χρησιμοποιήθηκε για την αξιολόγηση των μοντέλων παράλληλου προγραμματισμού [28].

Από την εκτέλεση αυτής της εκδοχής της εφαρμογής Image Processing (app01) προέκυψε ότι και τα δύο μοντέλα παράλληλου προγραμματισμού (StarPUPy και torcpy) υποστηρίζουν την εμφολευμένη παραλληλοποίηση, αλλά με μία μικρή διαφορά. Το torcpy υποστηρίζει την εμφολευμένη παραλληλοποίηση απευθείας χωρίς κάποια έξτρα ευθύνη συγχρονισμού ή εξασφάλιση αρκούντων πόρων για την εκτέλεση των υπο-εργασιών, ενώ το StarPUPy απαιτεί από τον προγραμματιστή να εξασφαλίσει ότι υπάρχουν αρκετοί διαθέσιμοι πόροι για την εκτέλεση των υπο-εργασιών. Διότι στον κώδικα που έχουμε, έχουμε στην ουσία 2 μητρικές εργασίες, η κάθε μία δημιουργεί από 50 περίπου υπο-εργασίες όσες και οι εικόνες που έχει. Αν εκτελέσουμε το πρόγραμμα με λιγότερο από 3 νήματα εργασίας, τότε το StarPUPy δεν θα μπορεί να εκτελέσει τις υπο-εργασίες και θα περιμένει για πάντα. Αντίθετα, το torcpy θα εκτελέσει τις υπο-εργασίες με τα διαθέσιμα νήματα και θα ολοκληρωθεί κανονικά. Αυτό γίνεται διότι μετά το submit των υπο-εργασιών, το torcpy δεν περιμένει να ολοκληρωθούν οι υπο-εργασίες για να συνεχίσει (non-blocking wait), ενώ το StarPUPy περιμένει να ολοκληρωθούν οι υπο-εργασίες πριν συνεχίσει (blocking wait). Έτσι, το StarPUPy απαιτεί από τον προγραμματιστή να εξασφαλίσει ότι υπάρχουν αρκετοί διαθέσιμοι πόροι για την εκτέλεση των υπο-εργασιών, ενώ το torcpy δεν απαιτεί κάτι τέτοιο.

7.2 Αποτελέσματα Πειραμάτων και Συμπεράσματα

Αρχικά, στο Σχήμα 7.1 παρουσιάζονται οι χρόνοι εκτέλεσης και οι χρονοβελτίωση όλων των εφαρμογών που αναπτύξαμε, όταν αυτές εκτελέστηκαν με το Standard (NUMA-oblivious) τρόπο εκτέλεσης.



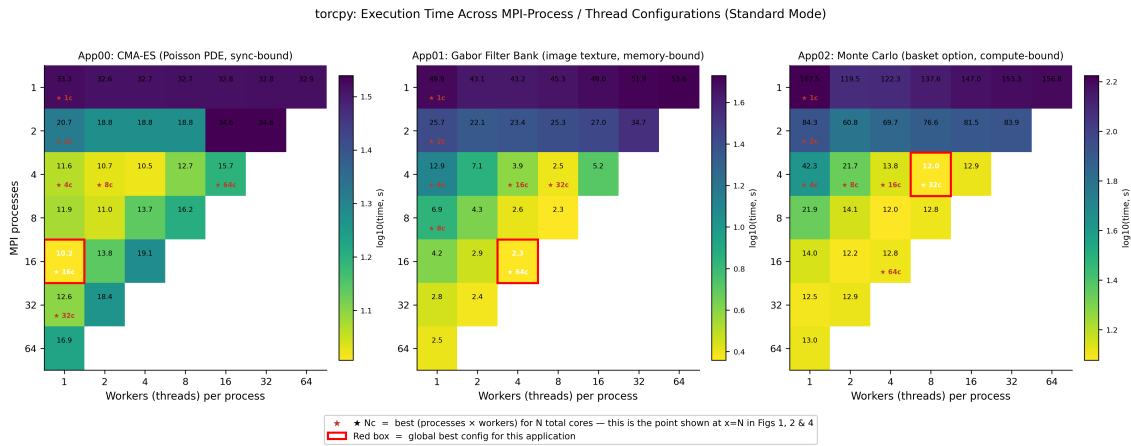
Σχήμα 7.1: Αποτελέσματα Πειραμάτων - Standard (NUMA-oblivious) τρόπος εκτέλεσης.

Τα πρώτα τρία γραφήματα του παραπάνω σχήματος παρουσιάζουν τους χρόνους εκτέλεσης των εφαρμογών. Ο άξονας y αντιπροσωπεύει τον χρόνο εκτέλεσης σε δευτερόλεπτα, ενώ ο άξονας x αντιπροσωπεύει τον αριθμό των νημάτων εργασίας που χρησιμοποιήθηκαν για την εκτέλεση των εφαρμογών. Αλλά επειδή για το torcpy έχουμε συνδυασμούς εκτέλεσης με διεργασίες και νήματα αντί καθαρά νήματα όπως στο StarPUPy, πήραμε από κάθε συνδυασμό εκτέλεσης ($\text{cores} = \text{processes} \times \text{threads}$) τον καλύτερο χρόνο. Δηλαδή, για το torcpy, αν εκτελέσαμε την εφαρμογή με 4 νήματα και 2 διεργασίες, και με 2 νήματα και 4 διεργασίες, τότε κρατήσαμε τον καλύτερο χρόνο από τους δύο συνδυασμούς. Έτσι, τα γραφήματα παρουσιάζουν τον καλύτερο χρόνο εκτέλεσης για κάθε αριθμό νημάτων εργασίας που χρησιμοποιήθηκαν.

Στα από κάτω τρία γραφήματα του Σχήματος 7.1 παρουσιάζεται η χρονοβελτίωση (speedup) των εφαρμογών για κάθε αριθμό νημάτων εργασίας σε σχέση με την σειριακή εκτέλεση (1 process x 1 thread). Ο άξονας y αντιπροσωπεύει την χρονοβελτίωση σε ποσοστό, ενώ ο άξονας x αντιπροσωπεύει τον αριθμό των νημάτων εργασίας που χρησιμοποιήθηκαν για την εκτέλεση των εφαρμογών, όπως και στα από πάνω γραφήματα. Επιπλέον, σε κάθε γράφημα υπάρχει και η θεωρητική ιδανική χρονοβελτίωση (ideal speedup) που θα είχαμε αν η εκτέλεση ήταν τέλεια παραλληλοποιημένη και δεν είχαμε κανένα είδους καθυστέρηση.

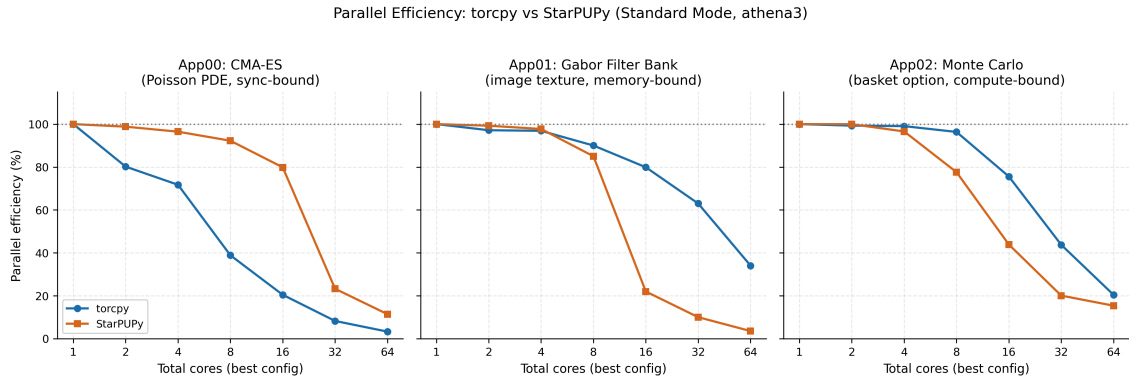
Στο Σχήμα 7.2 βλέπουμε σε ένα heatmap τις καλύτερες ρυθμίσεις εκτέλεσης για το torcpy για κάθε εφαρμογή και για κάθε αριθμό νημάτων/διεργασιών που χρησιμοποιήθηκαν. Ο άξονας y αντιπροσωπεύει τον αριθμό των διεργασιών (processes) που χρησιμοποιήθηκαν,

ενώ ο άξονας x αντιπροσωπεύει τον αριθμό των νημάτων (threads) που χρησιμοποιήθηκαν. Το χρώμα αντιπροσωπεύει τον χρόνο εκτέλεσης σε δευτερόλεπτα, με πιο ανοικτό κίτρινο χρώμα να σημαίνει μικρότερο χρόνο εκτέλεσης. Στα κελιά που αναγράφεται ένας αριθμός δίπλα από αστεράκι, σημαίνει ότι για το συγκεκριμένο πλήθος cores (processes x threads) που χρησιμοποιήθηκαν, το εκάστοτε configuration (processes x threads) ήταν το καλύτερο για την εκτέλεση της εφαρμογής. Τέλος, το κελί με κόκκινο πλαίσιο επισημαίνει πιο ήταν γενικά το γρηγορότερο configuration για την εκτέλεση της εφαρμογής.



Σχήμα 7.2: Καλύτερες ρυθμίσεις εκτέλεσης για το torcpy για κάθε εφαρμογή.

Πριν προχωρήσουμε στην ανάλυση των αποτελεσμάτων, έχουμε και το Σχήμα 7.3 όπου παρουσιάζεται η παράμετρος της παραλληλικής αποδοτικότητας (parallel efficiency) για κάθε εφαρμογή και για κάθε αριθμό νημάτων εργασίας που χρησιμοποιήθηκαν. Η παράμετρος της παραλληλικής αποδοτικότητας υπολογίζεται ως ο λόγος της χρονοβελτίωσης προς τον αριθμό των νημάτων εργασίας που χρησιμοποιήθηκαν. Δηλαδή, αν έχουμε χρονοβελτίωση 4 με 8 νήματα εργασίας, τότε η παραλληλική αποδοτικότητα είναι $4/8 = 0.5$ ή 50%. Ουσιαστικά, η παραλληλική αποδοτικότητα μας δείχνει πόσο καλά εκμεταλλευόμαστε τους διαθέσιμους πόρους (νήματα εργασίας) για την εκτέλεση των εφαρμογών.



Σχήμα 7.3: Παράμετρος Παραλληλικής Αποδοτικότητας (Parallel Efficiency) για κάθε εφαρμογή και για κάθε αριθμό νημάτων εργασίας.

7.2.1 Ανάλυση Αποτελεσμάτων Πειραμάτων

CMA-ES (app00)

Από το Σχήμα 7.1 παρατηρούμε ότι το StarPUPy υπερέχει σημαντικά του torcpy, επιτυγχάνοντας ελάχιστο χρόνο εκτέλεσης στους 16 πυρήνες, ενώ το torcpy εμφανίζει χαμηλότερο speedup (περίπου 3.5x) και αυξάνει τον χρόνο εκτέλεσης μετά τους 16 πυρήνες. Η παράλληλη αποδοτικότητα (Σχήμα 7.3) του torcpy πέφτει κατακόρυφα μετά τους 4 πυρήνες, ενώ του StarPUPy παραμένει πάνω από 80% μέχρι τους 16 επεξεργαστές.

Αυτό προκύπτει από το γεγονός ότι η εφαρμογή αυτή είναι sync-bound (περιορίζεται από συγχρονισμό). Όπως φαίνεται στον κώδικα, η παραλληλοποίηση γίνεται αυστηρά εντός κάθε γενιάς (γενιά g προς $g + 1$) με έναν βρόχο `while not es.stop()`. Το StarPUPy χρησιμοποιεί ελαφριά, ασύγχρονα νήματα (C-backend) και `asyncio.futures` που επιτρέπουν γρήγορο barrier συγχρονισμό ανά γενιά. Ενώ, το torcpy χρησιμοποιεί τη ρουτίνα `torch.map()`, η οποία σε κάθε γενιά εισάγει overhead κατανομής εργασιών μέσω διεργασιών MPI. Καθώς αυξάνονται οι πυρήνες, το κόστος συγχρονισμού του MPI barrier και το GIL thrashing για την ανάκτηση των αποτελεσμάτων ανά γενιά κυριαρχούν έναντι του μικρού υπολογιστικού έργου (πλέγμα 128×128) [28].

Image Processing - Gabor Filters (app01)

Στην δεύτερη εφαρμογή, Επεξεργασία Εικόνας - Φίλτρα Gabor (app01), παρατηρούμε ότι γίνεται το αντίστροφο σενάριο. Το torcpy εμφανίζει εξαιρετική κλιμάκωση (speedup 22x στους 64 πυρήνες) και η αποδοτικότητά του μειώνεται ομαλά. Το StarPUPy καταρρέει πλήρως μετά τους 8 πυρήνες, με τον χρόνο εκτέλεσης να αυξάνεται εκθετικά και την αποδοτικότητα να πέφτει κοντά στο 0%.

Εδώ βασικό ρόλο παίζει πάλι η φύση της εφαρμογής. Η εφαρμογή αυτή είναι memory-

bound (περιορίζεται από το εύρος ζώνης μνήμης) και εκτελεί 40 FFT convolutions ανά εικόνα. Το StarPUPy εκτελείται αποκλειστικά με threads εντός του ίδιου Python interpreter. Παρά το γεγονός ότι οι βιβλιοθήκες FFT (NumPy/BLAS) απελευθερώνουν το GIL, η ταυτόχρονη πρόσβαση δεκάδων νημάτων στην ίδια κοινή μνήμη προκαλεί ακραίο contention. Ενώ, το torcpy υπερέχει επειδή βασίζεται σε διεργασίες MPI. Κάθε worker thread ανήκει σε απομονωμένη διεργασία με δικό της χώρο διευθύνσεων (address space) αποφεύγοντας έτσι το contention.

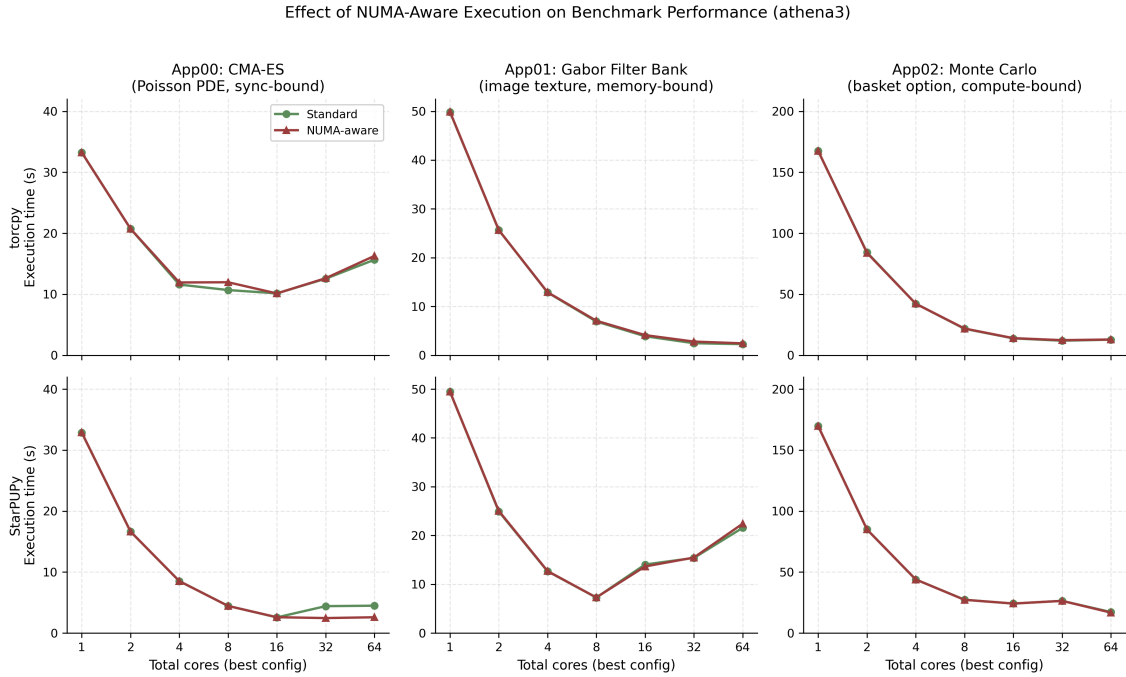
Monte Carlo (app02)

Και τα δύο συστήματα εμφανίζουν παρόμοια, ικανοποιητική κλιμάκωση μέχρι τους 8-16 πυρήνες. Μετά τους 16 πυρήνες, το torcpy καταφέρνει να διατηρήσει μια ελαφρώς καλύτερη καμπύλη speedup, ενώ το StarPUPy παρουσιάζει φαινόμενα κορεσμού (saturation).

Η εφαρμογή είναι compute-bound (έντονοι μαθηματικοί υπολογισμοί με Floops: $\exp$, $\sin$, $\log 1p$). Όπως φαίνεται στον κώδικα, η επικοινωνία είναι ελάχιστη, καθώς κάθε batch επιστρέφει μόνο τρεις scalar τιμές (`out_sum`, `out_sumsq`, `out_n`), πραγματοποιώντας ένα πολύ αποδοτικό parallel reduction στο τέλος. Η μικρή υπεροχή του torcpy στους πολλούς πυρήνες οφείλεται στην πλήρη εξάλειψη της πιθανότητας GIL thrashing λόγω των διακριτών MPI διεργασιών [28].

7.2.2 NUMA-aware εκτέλεση των εφαρμογών

Στην συνέχεια, στο Σχήμα 7.4 παρουσιάζονται οι χρόνοι εκτέλεσης όλων των εφαρμογών που αναπτύξαμε, όταν αυτές εκτελέστηκαν με το NUMA-aware τρόπο εκτέλεσης σε σύγκριση με την Standard εκτέλεση (NUMA-oblivious). Η εκτέλεση με NUMA-aware τρόπο έγινε με την χρήση των παραμέτρων που αναφέρθηκαν στο προηγούμενο κεφάλαιο, στην Υποενότητα 6.2.2.



Σχήμα 7.4: Αποτελέσματα Πειραμάτων - NUMA-aware εκτέλεση των εφαρμογών σε σύγκριση με την Standard (NUMA-oblivious) εκτέλεση.

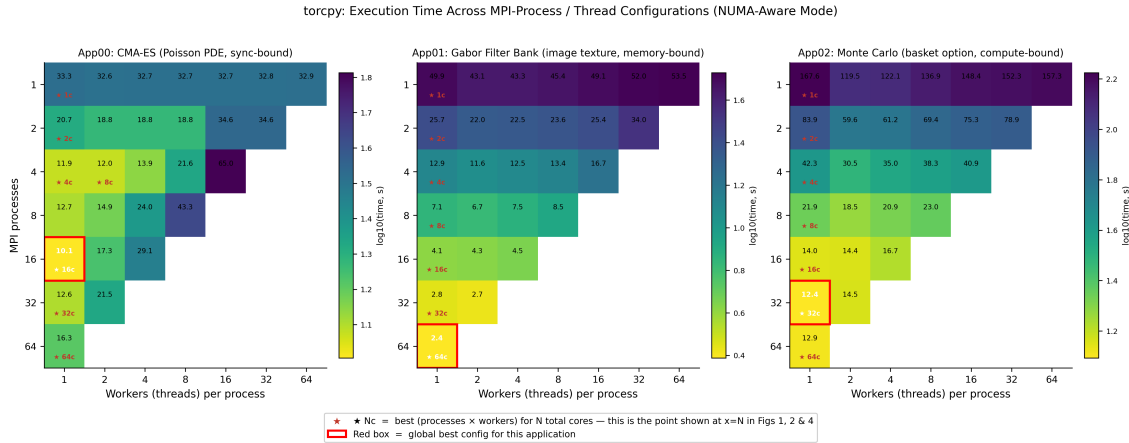
Όπως βλέπουμε στο παραπάνω σχήμα, οι τρεις πρώτες γραφικές παραστασεις είναι οι χρόνοι εκτέλεσης των εφαρμογών για κάθε αριθμό νημάτων εργασίας που χρησιμοποιήθηκαν για την torcpry και οι τρεις επόμενες είναι για το StarPUPy. Οι άξονες ορίζονται όπως και στο Σχήμα 7.1. Η πράσινη γραμμή αντιπροσωπεύει την εκτέλεση των εφαρμογών με Standard (NUMA-oblivious) τρόπο, ενώ η κόκκινη γραμμή αντιπροσωπεύει την εκτέλεση των εφαρμογών με NUMA-aware τρόπο.

Επίσης, στο Σχήμα 7.5 βλέπουμε σε ένα heatmap τις καλύτερες ρυθμίσεις εκτέλεσης για το torcpry για κάθε εφαρμογή και για κάθε αριθμό νημάτων/διεργασιών που χρησιμοποιήθηκαν όπως ήταν και για την Standard εκτέλεση.

Ανάλυση Αποτελεσμάτων NUMA-aware εκτέλεσης

Είναι προφανές από τα γραφήματα του Σχήματος 7.4 ότι η εκτέλεση των εφαρμογών με NUMA-aware τρόπο δεν βελτιώνει σχεδόν καθόλου τους χρόνους εκτέλεσης των εφαρμογών.

Αρχικά, για το torcpry η εκτέλεση με NUMA-aware τρόπο δεν βελτιώνει καθόλου τους χρόνους εκτέλεσης καμίας εκ των εφαρμογών. Αυτό οφείλεται στο γεγονός ότι το torcpry εκτελείται μέσω του MPI launcher με τις παραμέτρους `--map-by socket` και `--bind-to core`. Αυτό σημαίνει ότι το ίδιο το λειτουργικό σύστημα και το MPI δεσμεύουν (pin) τις διεργασίες μόνιμα στους φυσικούς πυρήνες του κάθε socket (NUMA node) εξαρχής. Επειδή όμως ακόμα και στο "Standard" mode, η τοπολογία είναι εν μέρει προστατευμένη



Σχήμα 7.5: Καλύτερες ρυθμίσεις εκτέλεσης για το torcpy για κάθε εφαρμογή (NUMA-aware).

από άσκοπες μεταναστεύσεις νημάτων μεταξύ sockets δεν έχουμε κάποια επίδραση στους χρόνους εκτέλεσης.

Στην συνέχεια, για το StarPUPy η εκτέλεση με NUMA-aware τρόπο ακολουθεί την ίδια λογική σε όλες τις εφαρμογές, εκτός από μια πολύ μικρή βελτίωση στους χρόνους εκτέλεσης την εφαρμογής CMA-ES (app00), όταν αυτή εκτελέστηκε με 32 και 64 νήματα εργασίας. Αυτό οφείλεται στο γεγονός ότι το StarPUPy στο Standard mode αφήνει το Linux OS να δρομολογήσει τα threads ελεύθερα. Στους 32 και 64 πυρήνες (που σημαίνει cross-socket εκτέλεση, αφού ο επεξεργαστής έχει 32 πυρήνες ανά socket), η app00 που έχει συχνούς συγχρονισμούς υποφέρει από Work Inflation¹ και απομακρυσμένη προσπέλαση μνήμης (remote memory accesses). Ενεργοποιώντας, όμως, το STARPU_USE_NUMA=1 και τον χρονοπρογραμματιστή dmda (Deque-Model Data-Aware), το StarPU χρησιμοποιεί το backend hwloc² και εφαρμόζει τοπική αναπαραγωγή δεδομένων (local data replication) και data-aware τοποθέτηση εργασιών (tasks). Οι εργασίες στέλνονται στον worker που είναι στο ίδιο NUMA node με τα δεδομένα (Modified/Shared κατάσταση του MSI πρωτοκόλλου), μειώνοντας δραστικά την κίνηση στο δίκτυο διασύνδεσης (interconnect traffic) μεταξύ των δύο sockets.

Παρόλα αυτά, η βελτίωση αυτή είναι πολύ μικρή και δεν επηρεάζει σημαντικά τους χρόνους εκτέλεσης των εφαρμογών. Αυτό οφείλεται στο γεγονός ότι οι εφαρμογές που αναπτύξαμε δεν έχουν μεγάλο όγκο δεδομένων και δεν εκτελούνται για μεγάλο χρονικό διάστημα, ώστε να επηρεαστούν σημαντικά από την τοπικότητα των δεδομένων και την

¹Στο HPC, ο όρος **Work Inflation (Διόγκωση Έργου)** αναφέρεται στον επιπλέον χρόνο CPU που καταναλώνει ένα παράλληλο πρόγραμμα σε σχέση με την ιδανική σειριακή εκτέλεσή του. Καθώς προσθέτετε επεξεργαστικές μονάδες, η αποδοτικότητα μειώνεται επειδή τα N νήματα (threads) παράγουν τελικά περισσότερο συνολικό έργο από ό,τι 1 νήμα επί N.

²Το **hwloc (Hardware Locality)** είναι εργαλείο για την αντιμετώπιση του Work Inflation που σχετίζεται με την τοπικότητα των δεδομένων και την αρχιτεκτονική NUMA.

αρχιτεκτονική NUMA.

7.3 Συμπεράσματα

Από την πειραματική αξιολόγηση προκύπτει ότι τα δύο συστήματα παρουσιάζουν διαφορετικά χαρακτηριστικά απόδοσης, τα οποία εξαρτώνται έντονα από τη φύση της εφαρμογής και από το μοντέλο εκτέλεσης. Για τις συγκεκριμένες υλοποιήσεις και ρυθμίσεις, το StarPUPy εμφανίζει καλύτερη συμπεριφορά στην εφαρμογή CMA-ES, όπου ο συχνός συγχρονισμός ανά γενιά ευνοεί έναν πιο ελαφρύ μηχανισμό υποβολής και αναμονής εργασιών. Αντίθετα, το torcpy αποδίδει καλύτερα στην εφαρμογή Gabor filters, όπου η χρήση ξεχωριστών MPI διεργασιών περιορίζει την επίδραση του GIL και μειώνει τον ανταγωνισμό εντός ενός κοινού Python interpreter. Στην εφαρμογή Monte Carlo, τα δύο συστήματα εμφανίζουν συγκρίσιμη συμπεριφορά, με μικρές διαφοροποιήσεις στους μεγαλύτερους αριθμούς πυρήνων. Τέλος, η NUMA-aware εκτέλεση επέφερε οριακές βελτιώσεις έναντι της Standard (NUMA-oblivious) εκτέλεσης.

7.4 Μελλοντική Εργασία

Με αφορμή τα αποτελέσματα των πειραμάτων που πραγματοποιήσαμε και της έρευνας που έγινε στα πλαίσια αυτής της διπλωματικής εργασίας, προκύπτουν κάποιες ιδέες για μελλοντική εργασία και έρευνα, όπως:

- Ανάπτυξη αντίστοιχων εφαρμογών σε C/C++ για να γίνει μια σύγκριση μεταξύ των δύο μοντέλων παράλληλου προγραμματισμού σε χαμηλότερο επίπεδο, με τις native εκδόσεις των δύο μοντέλων (StarPU και torc_lite) αντί για τις εκδόσεις τους για Python (StarPUPy και torcpy).
- Επέκταση της torcpy για να υποστηρίζει και κάρτες γραφικών (GPUs) για την εκτέλεση των εργασιών, ώστε να γίνει μια σύγκριση μεταξύ των δύο μοντέλων παράλληλου προγραμματισμού και σε περιβάλλοντα με GPUs μιας και είναι μια δυνατότητα του StarPUPy που υπάρχει αλλά δεν εξετάστηκε στο παρόν κείμενο.
- Ολοκλήρωση της υποστήριξης του StarPUPy για εκτέλεση με MPI, μέσω του Server-Client μοντέλου, ώστε να γίνει μια σύγκριση μεταξύ των δύο μοντέλων παράλληλου προγραμματισμού και σε περιβάλλοντα με MPI και clusters υπολογιστών.
- Ανάπτυξη εφαρμογών που επηρεάζονται από την αρχιτεκτονική NUMA και έχουν μεγάλο όγκο δεδομένων και εκτελούνται για μεγάλο χρονικό διάστημα, ώστε να

εξεταστεί η επίδραση της NUMA-aware εκτέλεσης των εφαρμογών στους χρόνους εκτέλεσης.

Βιβλιογραφία

- [1] Παπαδάκης Στυλιανός και Διαμαντάρας Κωνσταντίνος. *Προγραμματισμός και Αρχιτεκτονική Συστημάτων Παράλληλης Επεξεργασίας*. 1η έκδοση. Αθήνα: Εκδόσεις Κλειδάριθμος, 2012. ISBN: 978-960-461-446-2.
- [2] Panagiotis E Hadjidoukas κ.ά. ``torepy: Supporting task parallelism in Python". Στο: *SoftwareX* 12 (2020), σ. 100517.
- [3] Cédric Augonnet κ.ά. ``StarPU: a unified platform for task scheduling on heterogeneous multicore architectures". Στο: *European Conference on Parallel Processing*. Springer. 2009, σσ. 863–874.
- [4] Wikipedia. *NUMA (Non-Uniform Memory Access)*. 2024. URL: https://en.wikipedia.org/wiki/Non-uniform_memory_access.
- [5] Frank Denneman. *NUMA Deep Dive Part 1: From UMA to NUMA*. 2016. URL: <https://frankdenneman.ai/2016-07-07-numa-deep-dive-part-1-uma-numa/>.
- [6] Super Micro Computer, Inc. *What Is Non-Uniform Memory Access (NUMA)?* 2025. URL: <https://www.supermicro.com/en/glossary/numa>.
- [7] Justin Deters κ.ά. ``A NUMA-Aware Provably-Efficient Task-Parallel Platform Based on the Work-First Principle". Στο: *2018 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2018, σσ. 59–70. DOI: [10.1109/IISWC.2018.8573486](https://doi.org/10.1109/IISWC.2018.8573486). URL: <http://dx.doi.org/10.1109/IISWC.2018.8573486>.
- [8] RapidFork Technology. *Breaking Down Python Concurrency: The Global Interpreter Lock (GIL) and Its Effect on Multi-threading*. 2023. URL: <https://rapidfork.medium.com/breaking-down-python-concurrency-the-gil-and-its-effect-on-multi-threading-b5ab99083cb2>.
- [9] Sam Gross. *PEP 703 -- Making the Global Interpreter Lock Optional in CPython*. Python Enhancement Proposal 703. Python Software Foundation, 2023. URL: <https://peps.python.org/pep-0703/>.

- [10] William Ruys κ.ά. "A deep dive into task-based parallelism in Python". Στο: *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE. 2024, σσ. 1147–1149.
- [11] A.P. Adewole κ.ά. "Understanding and Implementing Concurrency in Python-based Systems". Στο: *International Journal of Engineering Research & Technology (IJERT)* 15.03 (2026). ISSN: 2278-0181. URL: <https://www.ijert.org/understanding-and-implementing-concurrency-in-python-based-systems-ijertv15is031148>.
- [12] StarPU Development Team. *StarPU Handbook for StarPU 1.4.12*. INRIA. 2025. URL: <https://files.inria.fr/starpu/doc/starpu.pdf>.
- [13] Cédric Augonnet, Samuel Thibault και Raymond Namyst. *StarPU: A Runtime System for Scheduling Tasks, or How to Get Portable Performance on Accelerator-Based Platforms Without the Agonizing Pain*. Presentation at NVIDIA GPU Technology Conference (GTC 2010), San Jose, USA. 2010. URL: https://www.nvidia.com/content/GTC-2010/pdfs/2160_GTC2010.pdf.
- [14] Cédric Augonnet, Samuel Thibault και Raymond Namyst. *StarPU: a Runtime System for Scheduling Tasks over Accelerator-Based Multicore Machines*. Research Report RR-7240. INRIA, Μαρ. 2010, σ. 33. URL: <https://inria.hal.science/inria-00467677>.
- [15] Cédric Augonnet κ.ά. "Data-aware task scheduling on multi-accelerator based platforms". Στο: *2010 IEEE 16th international conference on parallel and distributed systems*. IEEE. 2010, σσ. 291–298.
- [16] StarPU Development Team. *StarPU Developer Documentation*. INRIA. 2026. URL: https://files.inria.fr/starpu/doc/starpu_dev.pdf.
- [17] StarPU Development Team. *StarPU: A Unified Runtime System for Heterogeneous Multicore Architectures*. 2026. URL: <https://starpu.gitlabpages.inria.fr/>.
- [18] Alex Aiken. *StarPU -- CS315B Lecture 13*. Lecture slides, Stanford University, CS315B: Parallel Computing. URL: <https://web.stanford.edu/class/cs315b/lectures/lecture13.pdf>.
- [19] Xiaoke Qin και Prabhat Mishra. "Automated Generation of Directed Tests for Transition Coverage in Cache Coherence Protocols". Στο: *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*. 2012. URL: <https://www.cise.ufl.edu/research/cad/Publications/date12.pdf>.

- [20] Redis. *Understanding Cache Coherence in Systems*. URL: <https://redis.io/glossary/cache-coherence/>.
- [21] Cédric Augonnet κ.ά. *Getting Started with StarPU*. Tutorial at Symposium on Application Accelerators in High-Performance Computing (SAAHPC 2010), Knoxville, USA. 2010. URL: <https://starpu.gitlabpages.inria.fr/tutorials/2010-07-SAAHPC/saahpc.pdf>.
- [22] IBM. *torc_py: A Platform-Agnostic Adaptive Load Balancing Library for Task Parallelism*. GitHub Repository. URL: https://github.com/ibm/torc_py.
- [23] Nikolaus Hansen. ``Tutorial: Evolution Strategies and CMA-ES (Covariance Matrix Adaptation)". Στο: *Proceedings of GECCO 2014 (Tutorial)*. 2014. URL: <http://www.cmap.polytechnique.fr/~nikolaus.hansen/gecco2014-CMA-ES-tutorial.pdf>.
- [24] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. 2016. arXiv: 1604.00772. URL: <https://arxiv.org/abs/1604.00772>.
- [25] Nikolaus Hansen. *Covariance Matrix Adaptation Evolution Strategy (CMA-ES)*. Αδημοσίευτη ερευνητική εργασία. CMAP, École Polytechnique. URL: <http://www.cmap.polytechnique.fr/~nikolaus.hansen/copenhagen-cma-es.pdf>.
- [26] Wikipedia contributors. *CMA-ES*. 2025. URL: <https://en.wikipedia.org/wiki/CMA-ES>.
- [27] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. 2016. URL: <https://arxiv.org/pdf/1604.00772>.
- [28] George Tsavos. *Benchmark Application Source Code (app00--app02, StarPU / torcpy implementations)*. Unpublished software, University of Patras diploma thesis. 2026. URL: <https://github.com/GeorgeTsav/Comparative-Analysis-and-Experimental-Evaluation-of-Task-Based-Parallel-Programming-Systems>.
- [29] Wikipedia contributors. *Gabor filter*. 2025. URL: https://en.wikipedia.org/wiki/Gabor_filter.
- [30] ``Design of a Gabor Filter-Based Image Denoising Hardware Model". Στο: *Electronics* 11.7 (2022), σ. 1063. URL: <https://www.mdpi.com/2079-9292/11/7/1063>.
- [31] Baeldung. *How to Use Gabor Filters to Generate Features for Machine Learning*. 2024. URL: <https://www.baeldung.com/cs/ml-gabor-filters>.

-
- [32] Mark Goddard. *Monte-Carlo Option Pricing --- Correlated Random Sequences*. 2024. URL: <http://www.goddardconsulting.ca/option-pricing-monte-carlo-basket.html>.
- [33] Mike B. Giles. *Monte Carlo Methods: Geometric Brownian Motion, Correlated Brownian Motions*. 2013. URL: https://people.maths.ox.ac.uk/gilesmc/nanjing/giles_lecs-2x2.pdf.
- [34] ``A Monte Carlo Simulation Scheme for Basket Options with Barriers". Στο: *Atlantis Press*. 2016. URL: <https://www.atlantis-press.com/article/125971590.pdf>.

